

Mid-Term Report

高振明/Zhenming Gao

February 13, 2026

Contents

1 绪论	2
1.1 选题背景及意义	2
1.2 研究现状与文献综述	2
1.3 主要研究内容、预期目标与创新点	4
1.4 论述结构安排	5
1.5 本章小结	6
2 The <i>Tigris</i> language	7
2.1 词法惯例	7
2.2 值	7
2.3 标识符	8
2.4 类型表达式	8
2.5 模式	10
2.6 表达式	12
2.7 类型、类型类与实例定义	19
2.8 实现细节	22
3 Type System of the <i>Tigris</i> language	25
3.1 Formal Grammar	26
3.2 判断规则与推断规则	28
3.3 实例解析	34
3.4 System F 繁饰	37
3.5 本章小结	39
4 The <i>Lambda</i> IR, CPS, Codegen & FFI	39
4.1 <i>Lambda</i> IR	39
4.2 <i>Lambda</i> IR 的闭包转换	43
4.3 <i>Lambda</i> IR 的优化	44
4.4 CPS transformation	49
4.5 Backend: Common Lisp (SBCL) & FFI	55
5 PP: The <i>dependently-typed</i> table printer	61
6 <i>Tigrisd</i>: a lightweight theorem prover	65
6.1 The basics	65
6.2 A formal description of <i>Tigrisd</i>	67
7 Summary	73

1 绪论

1.1 选题背景及意义

随着大语言模型、前端框架 React、Vue 的兴起，作为理论计算机历史悠久的分支之一编程语言理论 & 设计实践 (Programming Language Theory, PLT & Programming Language Design & Implementation, PLDI) 与相关领域类型论 (Type Theory, TT)、范畴论 (Category Theory) 重新进入人们的视野。作为其产物之一的函数式编程范式 (functional programming) 被活用于前端设计部分，而另一产物定理证明器则与大模型结合，产生了 AI4MATH 等证明自动化项目，尝试利用大模型解决奥林匹克数学问题的捷报频传，受到数学家陶哲轩等人的青睐。然而因其理论程度高 (冷门)、对学生纯数学水平要求高的特点，许多大学并未开设该课程，推广 PLT、PLDI、类型论和范畴论的教育与学界所倡导的编程实践仍然是一项挑战。

本文旨在探索一种同时面向工程化程序开发与形式化验证的语言与编译工具链原型，同时获得可运行的编译器产出与可用的证明重写器，作为本科生或研究生科研训练与工程原型，从而成为教学、实验与工程验证的良好平台。这一原型分为两个子系统 (Tigris 语言与 Tigrisd 定理证明器，两者合称 TinyTheorem 解决方案) 按序实现：

在程序语言实践层面，本文继承并扩展 Hindley–Milner (HM) 类型系统的工程优势 (类型推断、简单语义、可扩展的类型类与谓词)，通过在工程化实践与更强语言表达力之间进行取舍，创新性地设计出高阶多态、蕴含的谓词约束、严格控制的高阶类型 HKT 构造等语言 (类型系统) 特性，系统化提升可读、可维护的中型编译器构建能力。产物包括 Tigris 语言、其形式化语言规格 (language specifications) 和一个可输出多层次结果的编译器、一个基于 IR 的解释器与 REPL。这一子系统使用 Lean 实现；

在形式化验证侧，本文设计了简化的依赖类型 (dependent types) 内核，以精简版的依赖类型 λ -代数 (λ -Calculus) 为计算模型，并采用双向 (bidirectional) 类型检查，在单一宇宙层级 (1-level type theory) 下实现蕴含的 (impredicative) 特性以降低内核复杂度，用于支持基本的证明对象与重写推理，产物包括 Tigrisd 语言 (定理证明器)、其形式化语言规格和一个可用于定理证明的 REPL。这一子系统使用 Haskell 实现。

本文借助“解耦”思路，以分离式架构将两个子系统并行推进：编程子系统坚持 HM 的相对弱的表达力与可编译性；证明子系统把依赖类型的强大表达力局限在更小而可控的内核上，从而避免将全依赖类型的复杂度与代价强加于普通程序员的日常开发环节。而项目本身，在不牺牲工程实践友好性的前提下，提供一体化但解耦的“语言 + 证明”原型，既便于教学与课程项目落地：如类型推断、闭包转换与续元传递 (continuation-passing) 化、函数式编程与依赖类型编程实践，也为进一步研究 (如更强的多态、约束求解、编译优化与证明自动化) 提供坚实基础与统一框架。

1.2 研究现状与文献综述

以下叙述本项目涉及的研究的现状。为了体现本项目的特殊之处，在每一部分的最后一段，若本文的实现与之有出入，我们加以简要说明。

需要明确的先验认识是：定理证明器也是编程语言，且一般为纯函数式语言，因为函数式语言在数理逻辑/数学上的合理性 (soundness) 最好，其中又以纯函数式为甚举例来说，数学的函数都没有副作用，这一点在 Haskell 语言中得到体现，除非是单子上 (Monadic，是受范畴论同名代数结构启发，函数式编程中用于建模副作用概念) 的函数¹。

Hindley–Milner 类型系统 由 Hindley、Milner 等奠基并由 Damas–Milner 系统化，形成“具有主类型/类型模式 (principal type-schemes)”与“算法 W (Algorithm W)”的经典框架。其核心优势为：对 λ -项的完全/全局类型推断 (complete/global type inference)、良好的语义基础与实现简洁性。该系统的后续工作围绕“向多态更强方向扩展”与“保持推断可行性”展开：

¹而即便是这样的函数，除去输入输出、多线程、STM (Software transactional memory) 这些真正与副作用相关的 IO、ST Monad 之外，其余都可以用纯函数模拟出来，本文的实现亦完全参照这一设计模式，因为 Haskell 和 Lean 都是纯函数式语言。

类型类 (typeclass) 与限定类型 (qualified types) 由 Wadler、Blott、Jones 等提出与系统化, 提供了对「重载与约束多态」的统一抽象, 使得「字典传递」成为行之有效的编译期 (compile-time) 繁饰 (elaboration) 设计策略。

高阶多态 (higher-rank polymorphism) 的推断 由 Peyton Jones 等提出, Dunfield-Krishnaswami 等人则提供了“完备且易实现的双向类型检查”解决方案, 在实践中往往以“显式标注 + 局部推断 + 刚性检查”的折中方式落地。

局部类型推断 (local type inference) 与双向类型检查 则在取强表达力与用户心智负担的中间地带: 仅在需要时 (例如函数声明, 或 2-阶及以上时) 通过少量标注引导推断, 兼顾可读性与可推断性。

在本文的“编程子系统”中, 我们借鉴上述成果, 采用“扩展 HM + 谓词 (predicates) + 类型类 + 严格一阶类型 (strict first-class) + 头等泛型 (first-class polymorphism)”的设计, 并配以“内嵌一阶模式 (Scheme) 方案 + Skolem 刚性检查 + 承担字典投影 (dictionary projection) 的 System F 繁饰”的实现策略, 使其既与 Haskell 传统保持一致性, 又便于在原型编译器框架下实现与调试。

语法与解析 (parsing) 函数式语言 (仅指 Milner 等人开创的 ML 系语言) 通常具有相仿的语法结构。其中著名者包括 Haskell、OCaml、Standard ML; 在定理证明器部分则又包括 Coq (现称 Rocq)、Lean、Agda 和 F*. 函数式语言语法的特点是灵活与统一, 既允许多种多样的用户自定义运算符 (例如使用 LR(1) ocamllyacc 实现的 OCaml), 更甚者则支持强大的元编程 (metaprogramming) 功能, 例如 GHC 的 TemplateHaskell, OCaml 的 Pre-Processor Extension (PPX) 和 Lean 的 syntax、macro、macro_rules 等语法构造: 这一特性在定理证明器中尤为明显, 因其对数学证明的需求。统一性则体现在函数式语言对语法糖 (syntactic sugar) 的缺失, 得益于这些语言的面向表达式的特性 (expression-oriented), 其表层语法 (surface syntax) 具有强通用性, 且统一使用含类型 (typed) λ -代数 (或其变体, 例如定理证明器则使用依赖类型的版本) 作为它们的核心代数 (core calculus)。从这个角度, 也可以说函数式语言几乎只有语法糖, 因为其表层语法全部被解糖 (desugar) 到前述的核心代数中, 但为了和工业语言特殊对待的“语法糖”区分, 我们不称其为语法糖。

这些特性, 除元编程之外, 都由 Tigris 继承。我们设计的表层语法综合了 Lean、OCaml、Haskell 的长处, 并通过受范畴论启发的单子式²的语法分析组合子 (Monadic Parser Combinator) 保证解析器的模块化和可维护性, 同时利用 Pratt 解析技术实现“用户可定义运算符 + 优先级/结合性”的高可扩展解析能力。Pratt 自顶向下运算符优先级解析法以其“实现简洁、扩展性强”而被广泛采用, 适合教学与原型构建。结合“分派/分组式 let 语法糖”、模式匹配 (pattern matching) 与记录 (record/struct) /归纳 (代数) 数据类型 (inductive types/algebraic datatypes) 等语言构件, 可覆盖函数式编程的主流设计模式。

类型类的繁饰与实例解析 (instance resolution) 基于 Wadler&Blott 的类型类思想, 将临时 (ad-hoc) 多态 (例如 C、Java 的函数重载) 以“字典”抽象归结为参数化 (parametric) 多态的系统化繁饰; Jones 的合格类型与“构造子类”进一步将类型构造子 (如 Functor、Applicative、Monad 等经典纯函数式设计模式) 纳入框架。在实际实现中, GHC 的 OutsideIn(X) 体系及其后续演化提出了以约束为中心的求解与实例选择流程, 并辅以 Skolem 刚性变量 (rigids)、泛化 (generalization) /实例化策略来控制推断过程的完备性与可预测性。

本文采用与之相容的思路, 但简化了 OutsideIn(X) 体系:

- System F 的繁饰以字典参数 (项层) + 类型抽象 (类型项层) 联合刻画;
- 刚性检查通过在注解与应用位点进行 Skolem 化检查, 防止不当的跨层次统一;
- 实例解析采用“基于头部匹配 + 递归解析上下文谓词”的策略 (参照 OutsideIn 风格), 并结合模式匹配决策树优化编译。

²属于函数式编程的设计模式

IR 设计与编译后端 在编译管线方面, 研究社区已就 “System F 繁饰 $\rightarrow$ Administrative-Normal Form (ANF) $\rightarrow$ 闭包转换 (closure conversion) $\rightarrow$ CPS 化” 的路径形成较为成熟的共识与方法论。这其中, Girard 提出的 System F 作为多态 λ -代数的标准中间表示, 适合承载类型抽象与字典化后的多态程序; Flanagan、Appel 等人在 ANF 与 CPS 化的经典工作明确了将控制流与求值顺序外显 (explicit) 化的工程路径, 利于优化与目标后端对接。我们在此基础上, 进一步设计了 “Lambda IR (ANF 变体) $\rightarrow$ CPS IR” 的两段式 IR, 并最终落地到 Common Lisp 以获得成熟运行时、交互环境、外部函数接口 (foreign function interface, FFI) 的支持

依赖类型内核与双向检查 在证明子系统方面, Lean、Coq、Agda 分别代表了现代依赖类型定理证明器的不同取向: Lean 与 Coq 的理论基础一致 (Calculus of Inductive Construction, CIC), 但前者强调工程机制与元编程生态, 后者强调严谨公理化³, 而 Agda 则强调程序化证明与可视化交互, 且主要用于建模 (modeling) 类型论研究前沿的同伦类型论 (Homotopy Type Theory) 与立方类型论 (Cubical Type Theory)。而它们的双向检查部分, 基本来源于前述编程语言部分类型系统的实现。

本项目采用 “简化依赖类型 + 单一宇宙 + 双向检查” 的取舍, 参考教学与研究社区中的 “pi-forall”、“LambdaPi” 等项目经验, 避免把复杂的宇宙层级、多层归纳族与高阶等式推理一次性纳入, 实现 “可运行、可实验” 的证明原型。通过与编程子系统解耦, 用户在不涉及证明时可完全在 HM 子系统内获得流畅的开发体验, 在需要轻量证明与重写时又可切换到依赖类型内核。

1.3 主要研究内容、预期目标与创新点

研究的主要内容与设计思路

- 在语言前端与解析上, 我们将实现 Lean/OCaml/Haskell 风格交融的函数式语法, 采用 Pratt 解析构建可扩展的表达式优先级框架, 支持用户自定义操作符 (优先级/结合性) 与冲突检查。表层语法大体依照 OCaml 风格, 但参考 Lean 的习惯写法提供语法糖和 Haskell 的写法提供部分语法结构 (language construct) 与模式匹配、记录/ADT、头等抽象、类型标注 (type annotation/ascription) 等语法。
- 在类型系统实现上, 构建 “扩展 HM + 谓词 + 类型类 + 第一阶 HKT” 类型系统; 支持 rank-1 方案 (TSch) 内嵌与字典化繁饰; 在语法树 (abstract syntax tree, AST) 的 Ascribe/Var/App 等关键位点使用 Skolem 刚性检查与 rank-1 方案实例 (instantiation); 提供局部双向检查与必要的显式标注通道。在类型求解上: 采用一阶合一 (unification) 的方式求解类型、参照 “字典变量在作用域内匹配/查找” 的 OutsideIn 式思路, 维护 “已访问实例” 集合避免出现环导致系统崩溃 (diverge); 在类型制导 (type-directed) 语法上, 我们令类型检查器 (typechecker) 在需要时 (譬如实例解析、解糖时的易歧义处) 要求显式类型注解, 提升错误定位与歧义时的可预期性。
- 在 IR 降级上, 我们繁饰表层语法到 System F: 新增 Proj (字典投影) 等节点便于优化与降级; 将类型抽象与字典参数显式化, 提供 TyLam/TyApp/Proj 等 System F IR 结点, 并配套 β/η -化、减少生成的 IR 噪声并向后续步骤暴露更多的优化机会, 同时保证后续变换的可维护性;
- 在后端 IR 与编译上, 我们的管线是 Lambda IR $\rightarrow$ Closure Conversion $\rightarrow$ CPS Transformation $\rightarrow$ Common Lisp; 在 IR 层实现常量折叠 (constant folding)、拷贝传播 (copy propagation)、无用代码消除 (dead code elimination)、尾调用 (tailcall/tail-recursive) 化、投影/选择消去的源优化, 最后在 CPS 层显式表达控制转移并直接下降到目标 Common Lisp 代码。
- 模式匹配检查与编译上: 在类型全数推断出来后, 我们参照 Maranget 的模式匹配冗余/欠缺检查算法, 针对缺少的/多余的模式分支向用户发出警告, 并针对具体情况向用户提出一种缺失的

³前者与后者相比, 提供了一些现代逻辑认为不可靠的公理, 例如集合论的选择公理 Axiom of Choice, 以及由之导出的排中律, 因而受人诟病; 而考虑到编程需要, 其在停机检查 termination checking 时也较松, 允许 partial 和 unsafe。因而本项目选择用其作为编程语言。

分支/立即删除多余分支；在 System F 繁饰之后，我们根据其基于决策树的编译算法，生成良好分支。

- 在证明内核部分，我们采用单一宇宙、蕴含式的依赖类型 λ -代数，并采用双向类型检查，降低判定复杂度与术语成本；参考标准的等式重写与项代换理论 (Baader&Nipkow) 提供项重写式 (term-rewriting) 解释器以支持基本等式推理与程序化证明脚本；利用 Haskell 在学术上的丰富生态，通过 Unbound 等库实现对于 λ -项的操作，变换，以及变量约束的判断；引入 Peyton Jones 提出的 Weak Head 范式 (Weak Head Normal Form, WHNF) 用于控制消去 (reduction)，将其与定义性等号 (definitional equality) 的实现结合，避免在判断两个项是否相等时的无用计算、消去、展开 (unfolding)，提升定理证明性能。
- 最后，在实验与评测上，我们以 Functor/Applicative/Monad 等典型代数结构暨函数式编程设计模式为样例，验证实例解析、字典化与投影优化的正确性；以典型函数式程序映射 (map)、折叠 (fold) 考察编译后端的性能与优化效果；以基本代数定理/列表性质作为证明例子，验证证明内核的可用性与重写效率。

课题整体采用“自顶向下的最小可用 (MVP) + 逐步精炼”的策略。首先保证语言前端、类型系统核心路径、编译与运行链路闭环 (从 Tigris 源到 Common Lisp 目标可运行)，再迭代增强类型系统细节、实例解析策略与 IR 优化规则。编程与证明两子系统以统一仓库并行推进，但保持边界清晰、依赖方向单纯：编程侧产物可独立使用，证明侧可以通过两系统相似的表层语法无痛迁移。

研究的预期目标 本课题包含三个目标 (要注意，这是最低限度的目标，实际上我们实现的功能并不止与此，见正文)：

1. 一个可运行的原型编译器与解释器：可编译到 Common Lisp (SBCL) 或直接在 REPL 交互下运行；可对带类型类与 HKT 的示例进行正确繁饰求值。
2. 整的 IR 管线与可视化输出 (基于 Wadler 的 Pretty Printing 算法)：便于教学与调试；可展示从 System F 到 Lambda IR、CPS IR 的逐步变换。
3. 证明侧最小可用内核：支持基本 λ -项、依值积/和 (dependent product/coproduct, Π/Σ) 与等式重写的简单证明脚本；在实际例子 (如代数结构的基本定理) 中可用。

1.4 论述结构安排

本文主要对课题 TinyTheorem 的两个子系统 (Tigris、Tigrisd) 做出描述和形式化说明，用于规范语言行为，同时向读者介绍两系统的使用方式。同时论述语言设计与实现上的考量、优势，以及回首复查时发现的设计失误。此外，本文不仅提倡函数式编程和依赖类型编程范式的使用，同时也是其实践者。我们还将提供一个运用于本项目的依赖类型编程的实例作为范例，并向读者介绍其思考方式、安全性与优点。具体来说，本文的正文部分可以分为六大部分：

第一部分 以语言明细/报告的 (参照 OCaml Manual 或 Haskell Language Report) 口吻用自然语言向读者全面叙述 Tigris 语言的设计以及其各部分行为，同时亦作为读者了解 Tigris 如何使用、如何工作的主要途径。因为编译是线性过程，自然，我们跳出适合一般、多层次软件描述框架，按最直观且最符合逻辑的编程语言报告的格式，根据语言对象的原子程度和编译器各 pass 的顺序作为叙述顺序，具体是：

词法惯例、值、标识符、类型表达式、模式 (模式匹配的)、表达式/定义语句、类型定义

在本部分，代码实现上的论述主要包含词法、句法分析，同时简单涉及类型检查器，即编译器的前端。这是因为类型系统和后续的编译部分和后端实现起来较为复杂，所以由它们将在后文各自独立的部分进行叙述。同时，我们认为学术语言，特别是函数式语言和工业语言，特别是常见的面向对象语言在

设计、使用、实现上都有诸多不同（例如函数式语言通常是值导向的，并且具有统一的计算模型，这在工业语言中仍然较少见）为了避免犯全知视角的写作错误，我们仍然从最基本词法惯例开始叙述。此外，这一部分亦作为读者了解函数式编程语言的窗口，因为 Tigris 语言是纯函数式的。

第二部分 专门用于给出正式的 Tigris 语言的类型系统的形式化表述。这是因为类型系统、类型论在 PL/CAV/纯数学学界，是许多重要理论的基础，十分受到重视。我们仿照 PL 论文惯例，利用 OTT 给出严谨的 Tigris 的核心代数及其定义与判断规则 (judgemental rules)、推断规则 (inference rules) 这些规则指导、规范着 Tigris 类型系统的实现，方便读者检查类型系统设计的 soundness，但阅读它们则不像阅读代码一样枯燥无味，但需要读者接受过充分的 PL/类型论的训练。此外，这一部分将更加深入的探讨 Tigris 类型系统、类型类与实例解析算法的实现及性质，最后将叙述 System F 繁饰的细节及与之相关的一些编译期优化。

第三部分 对 Tigris 编译器的中后部分的实现与设计进行叙述，包括 IR 设计、优化与生成、闭包转换、CPS 转换、后端代码生成、运行时系统及 FFI。同时，还将涉及周边工具，如编译器命令行前端的设计使用，以及基于 IR 的解析器/REPL 的实现。此外，在描述 CPS 转换时，我们给出正式的指称语义 (denotational semantics)，这些数学公式规定了 CPS 转换的对象与结果，此处需要受充分的 PL 训练的读者。这一部分应当是外行人在接触编译原理时首先想到的，其重要性不言而喻，且体现了编译的字面义：没有了这些管线，使用 Tigris 编写的程序终究无法工作。

第四部分 撷取 Tigris 实现过程中使用的一个依赖类型编程范例，以身作则向读者倡导、帮助读者理解这一编程范式的思考过程和优势。可以看作是对依赖类型编程的一个简单描述，或一份教程。同时这一部分还起到承上启下的作用：为读者提供即将叙述的 Tigrisd 一些先验知识，使读者能够更确切的理解 Tigrisd 的一些设计。

第五部分 主要给出 Tigrisd 定理证明器的自然语言介绍以及正式的形式化描述。由于 Tigrisd 在表层语法上与 Tigris 相仿，而且采用相似的 parser combinator 的句法分析实现方式，我们主要对两者的不同之处、与定理证明器特有的部分进行描述，主要包括枚举族 (indexed family) 定义、函数、模式匹配与依赖积的消去 (elimination)、证明/类型无关性、模块设计、内置命令以及类型宇宙 (universe)。而最重要的 Tigrisd 内核 (kernel, 包含 typechecker 和类型论，依惯例称定理证明器的类型系统为类型论)、与实现相关的 WHNF 和定义性相等则通过与第二部分非常相近的格式通过 OTT 给出形式化描述。对内核的形式化描述规定了定理证明器的行为与计算法则 (computational rule)。阅读这一部分需要读者接受过充分的 PL/类型论的训练。

第六部分 总结本课题并作展望，指出本项目以后可能的研究和发展方向，并提出一些改进与语言功能、配套编辑器插件上的增加。

最后，读者需要注意到，由于本课题自身属于编程语言设计与实现，正文中不可避免的会出现 Tigris 和 Tigrisd 的代码。我们希望读者不要默认其枯燥无味，因为这些代码是样例，而并非用于凑字数的具体实现的源代码；它们用于形象地演示当前正在描述的问题。对于任何接触过编程的读者来说，我们认为相比于用自然语言描述问题，使用样例代码可以更清晰的展现问题。同时，为了方便读者分辨样例代码与实现的源代码，在排版上对前者我们开启语法高亮，对后者我们在引用代码的左侧加上代码行数。

本文由英文原文翻译而来。

1.5 本章小结

(略)

符号	关键字									
→ ;; ⇒	mutual	infixl	infixr	match	then	let	fun			
, _ : ∀	extern	class	forall	data	prefix	postfix	in			
	type	with	instance	else	and	rec	if			

Table 1: 保留符号及关键字

2 The *Tigris* language

Tigris 是一个注重可读性与语法统一性的灵活的，纯函数式静态类型的语言。它从 Lean、OCaml 和 Haskell 汲取灵感。

2.1 词法惯例

在下文我们用一套修改版的 BNF (Backus-Naur Normal Form, 误和 ABNF 混淆) 来表示表层语法。修改过的 BNF 可以在其右手侧 (RHS) 接受 POSIX 正则表达式中的 `+[^]`。

文本编码 Tigris 源代码应当是 UTF-8 编码的文本文件。

空白字符 包含空格 (spaces)、制表符 (tabs)、CR/LF。空白字符会被忽略，因而不影响程序的语义。⁴ 通过使用空白字符，相邻的标识符、字面量以及关键字可以被隔开，而不被算入同一个标识符中。

注释 指以 `--` (Lean/Haskell/Lua 风格) 或以 `NB.` (J 风格) 开头的单行注释。与空白字符相同，注释也会被词法分析器忽略。注意目前并不支持多行注释。

标识符 几乎和 OCaml/Haskell/Lean 一致：

```

<ident>      ::= (<letter> | _) <rest>*
<upper-ident> ::= <uppercase> <rest>*
<lower-ident> ::= <lowercase> <rest>*
<letter>     ::= <upper-letter> | <lower-letter>
<rest>       ::= <letter> | 0..9 | _ | ' | ! | ?
<lower-letter> ::= a..z
<upper-letter> ::= A..Z

```

为了化简语法分析，标识符是否由大写字母开头具有语义（后述），而以 `!?` 结尾的标识符则不做区分。后者是为了允许一些常见的代码风格，例如以 `?` 结尾表示一个可空的值/函数 (**Option** a)、以 `!` 结尾表示一个函数可能抛出错误或造成非本地返回 (non-local exiting)。

整数字面量 `<intlit>` ::= `("-" | "+")? 0..9+`，十进制且支持大数。

字符串字面量 `<strlit>` ::= `"[^"]+"`

保留符号及关键字 见 **Table 1**。词法分析的歧义按照最长匹配原则进行处理。

2.2 值

值 (value) 是不能够再消去 (化简) 的表达式，可以理解为一个范式。用大步操作语义 (big-step operational semantics) 表示为 $\langle v, \sigma \rangle \Downarrow \langle v \rangle$ 其中 v 是以下其中之一：

⁴在定理证明器一侧并非如此。

整数 整数的范围取决于后端。在我们的实现中，无论是 SBCL 后端，还是 Lean 编写的解释器，都支持可变精度数字，亦即大数运算，因而不存在相对底层语言的 u32、i64 等等数据类型。

字符串 是字符的无穷序列。其长度限制取决于后端。在我们的实现中，SBCL 后端支持长达 2^{45} 的字符串。

元组 n -元组 (tuple) 通常写作 $(v_1, \dots, v_n)$.⁵

结构体 亦称记录 (record) 是归纳类型/代数数据类型 (inductive/algebraic datatype, ADT) 的语法糖，字面量写作 $\{field_1 = v_1, \dots, field_n = v_n\}$.⁶ 考虑到潜在的大重构问题，当前版本的 Tigris 只实现了伪类型制导句法分析。因而对具有相同栏位 (field) 的结构体来说，无论其类型如何，都会抛出一个句法分析歧义警告 (ambiguous parsing warning)，但该警告可被类型标注 (包括后述的模式亦是如此) 消除：⁷

```
type Point = {x : Int, y : Int} -- Point' 定义和 Point 相同
let _ = {x = 1, y = 2} -- [WARN] ambiguous record literal (using default 'Point'),
                        -- candidates are [(Point, #[x, y]), (Point', #[x, y])]
let _ = ({x = 1, y = 2} : Point') -- 成功
```

对于栏位有重叠的部分，我们用最长匹配规则来区分：

```
type Point = {x : Int, y : Int} ;; type Point3D = {x : Int, y : Int, z : Int}
let (x, y) = ( {x = 1, y = 2}           -- x : Point
              , {x = 1, y = 2, z = 3} ) -- y : Point3D
```

构造值 指的是通过某一归纳类型的值构造子 (value constructor, 简称构造子, constructor) 构造出的值，英文亦作 *variant*。构造子类似函数，但他们也可以不接收任何参数：对某构造子 C_i ，其构造值一定是 C_i 或 $C_i \text{ fid}_1 \dots \text{fid}_n$ 。以下若干构造值是内置的：

构造子	类型
true	Bool
false	Bool
()	Unit

Functions 指从值到值的映射，细节后述。

2.3 标识符

标识符用于指代某一语言对象：Figure 1a

2.4 类型表达式

类型表达式具有以下语法：

```
<type-expr> ::= <type-app> (-> <type-expr>)?           (arrow type)
<type-atom> ::= <ident>
               | "(" <type-expr> ")"                   (parenthesized)
               | <type-ctor>
```

⁵元组在实现中是特殊对待，并非由常规的 Prod 编码。

⁶而类型类的实例又是结构体的语法糖。

⁷因为它是由句法分析器处理而非类型检查器，该类型标注必须置于结构体字面量上。

类别	首字母	运算符	结合性
值	任意	类型应用	左
构造子	大写	积类型 (*)	右
类型构造子	大写	箭头类型 ($\rightarrow$)	右
结构体栏位	任意		
约束的类型变量	小写		

(a) 不同类别标识符的大小写差异

(b) 类型运算符结合性

Figure 1: 不同类别标识符的大小写差异与类型运算符结合性

	sepBy(* ×, <type-expr>)	(product type)
	<type-args>	
<type-app>	::= <type-app>? <type-atom>	(type application)
<type-ctor>	::= <upper-ident>	
<bound-ty>	::= <lower-ident>	
<type-binder>	::= <bound-ty>	(type binder)
	"(" <bound-ty> : <intlit> ")"	
<type-args>	::= <bound-ty> <type-expr>	(type argument)
<qualified>	::= "[" sepBy1(",", (<bound-ty> <type-ctor>) <type-expr>+) "]"	
<scheme>	::= { FORALL <type-binder>+ }? <qualified>? ", " <type-expr>	
FORALL	::= forall ∀	

需要注意，对于并列式⁸的类型应用 (juxtapositional type application)⁹、积类型和箭头类型，我们有以下优先级（越高者越先）与结合性表，见Figure 1b。

类型变量 亦称 TV (type variables)，是小写字母开头的标识符。类型变量的类型，即类型的类型 (kind) 是由用户显式提供的，因为目前我们并未实现 TV 的类型推断 (kind inference)。一个记作 $(\alpha : n)$ 的 TV 有类型 $\text{Type} \rightarrow \text{Type}$ ，否则默认为 Type 。但因目前我们不允许柯里化的高阶类型存在， $(\alpha : 2)$ 实际上是 $\text{Type} \times \text{Type} \rightarrow \text{Type}$ 。因而在称谓上，我们应当使用元数 (arity) 一词而非 kind。在数据类型定义中，TV 必须是被约束 (bound) 的，其约束位置可以是类型构造子的参数位置，或是右手侧的全称量词 $\forall$ (forall) 中。读者应当注意到，类型表达式的合法性检查 (validity checking) 基本在句法分析阶段完成，也就是说，若存在某一 TV 是自由变量 (free variable)，编译器会抛出句法分析错误。

在实现中，编译器会用到以下两种具有特殊含义的 TV：

- 元变量 (metavariables)，亦称 MV，记作 ?m.N 其中 N 是计数器的数字。元变量指代某一未知类型，且必须被某一类型所实例化 (instantiation)¹⁰、或在离开作用域时被 HM 类型系统的自动泛化机制所泛化。无论如何，MV 不应当在最终的模式 (scheme)¹¹ 中出现。
- Skolem TV¹²，亦称 skolems，记作 ?sk.a 其中 a 指代原始 TV。Skolems 由 skolemization 过程

⁸并列式指语法上不需要括号的函数应用/调用。例如 $f\ 1\ 2$ 表示将 1, 2 两参数传给 f 并调用之。这一语法差异看似简单，实际上有重大差异：函数式语言具有的柯里化 (后述) 特性使得 $f\ 1\ 2$ 比 $f(1, 2)$ 更符合语义，因为后者等于向 f 传一个元组 (1, 2)。这是 ML 系函数式语言的共性，但在 Ruby、Perl/Roku 等语言中也有出现。读者可以把其类比为 Shell 命令的调用。而从句法分析的角度考虑，并列式语法的实现与中缀运算符一致，但其“算符”是空白字符，因而有下方的优先级表。

⁹在实现上，类似 Haskell 2010 Report 中的 a/b-type。

¹⁰“实例化”一词在本文中经常使用，但该词的具体含义并没有清晰的定义：有时，我们用这词指代特殊化 (specialization)，指 TV 被实际 (concrete) 类型所实例化；有时，我们只用其指代 TV 的替换 (substitution) 操作。

¹¹与上面的“实例化”一词相同，“模式”一词，既指代模式匹配中的模式 (pattern)，亦指代公理模式 (此处为类型模式) 中的模式 (scheme)。虽然这容易造成歧义，但两个概念相隔甚远，读者应当能自行区分其具体意思。

¹²纪念挪威数学家 Thoralf Skolem，主要研究数理逻辑与集合论。

产生，这些 TV 实际上由类型构造子编码¹³，并且绝对不应被实例化。这使得类型检查器能够查出更多的类型错误：

```
instance : Functor Option = { map --  $\forall \alpha, \beta. (\alpha \rightarrow \beta) \rightarrow \text{Option } \alpha \rightarrow \text{Option } \beta$ 
    | f, None  $\Rightarrow$  None
    | f, Some a  $\Rightarrow$  Some a } -- [ERROR] Can't unify type ?sk.a with ?sk.b.
```

上述例子是非法的：根据 map 的模式含义，类型变量 a 和 b 应当相互独立，即 $a \not\sim b$ ，其中 $(\sim)$ 算符表示合一化：在 map 函数体内，两者被 skolemize，不能相互用自身实例化对方。这里，从 map 正确的语义来看，我们猜想用户应当是忘记应用 f，即第二个分支应当是 Some (f a)。即便我们允许这样的代码，其语义也不正确：但现在我们可以通过 skolems 在编译期就捕捉到这一错误并回报给用户。Skolems 所具有的特殊性质称为刚性 (rigid)，在后文我们会见到不仅仅是 Skolems 具有刚性，但其是最具代表性的。

函数类型 类型表达式 $ty_1 \rightarrow \dots \rightarrow ty_n$ 表示一个从 $ty_1, \dots, ty_{n-1}$ 到 ty_n 的函数的类型。

积类型 类型表达式 $ty_1 \times \dots \times ty_n$ 表示一个具有类型为 $ty_1 \times \dots \times ty_n$ 的元素的元组的类型。

构造类型 与构造值相似，可看做类型层面的值。¹⁴ 正如同 TV，类型构造子同样具有元数，而元数为 0 的类型构造子其自身即是类型，构成一个类型表达式。但与构造子不同，类型构造子必须完全应用 (fully applied)，这一特性与工业语言中的函数相似。

不定递归类型 会被类型检查器拒绝，因此类型检查器无法推断程序 `fun x -> x x` 的类型，且一个特殊的错误信息会被抛出。这是为了提高类型系统的合理性，因为不定递归类型的方程式无解，或者说其解是一个无限增长的树形结构。允许这种类型的存在，就可以构造不动点组合子 (fixpoint combinator)，其中又以 Y 组合子最为著名 (以下是其中一种实现)：

$$Y f = f (Y f) : \forall \alpha. (\alpha \rightarrow \alpha) \rightarrow \alpha,$$

将其应用到恒等函数 (identity) 函数上 $Y \text{id}_\alpha$ 得到类型 $\forall \alpha. \alpha$ 。出现这一类型的值是非常危险的，因为该类型可以被实例化到任意一个类型上，即，该值具有任意类型，可以是整数，也可以是字符串。更危险的问题来源于后文 Tigrisd 定理证明器的理论基础 Church-Howard 同构 (isomorphism)，在逻辑上，存在这样的类型 (这样的命题成立) 可以导出任意命题成立，这显然是大错特错的。而即便我们只禁止这种情况，诸如 $\alpha \sim \tau$ 其中 $\alpha \in \tau$ 的合一化也是不合理的，因而我们不允许这种类型的存在。

2.5 模式

模式匹配的模式是一个指令模板，用于解构 (destructure) 某种形状的数据结构。如果用户自定义运算符 (后述) 被绑定到构造子上，则该运算符亦自动作为新的模式 (语法糖) 出现，提高代码可读性。无论何种模式，在句法分析期其都将被解糖至以下集中基本模式中：

<code><pat></code>	<code>::= <pat-left> <infix-op> <pat></code>	infix operator enriched
	<code><pat-left></code>	
	<code><prefix-op> <pat></code>	prefix operator enriched
	<code><pat> <postfix-op></code>	postfix operator enriched
<code><pat-left></code>	<code>::= <pat-atom></code>	
	<code><ctor> many1(<pat-atom>)</code>	variant
<code><pat-atom></code>	<code>::= <lower-ident></code>	variable

¹³在实现中，我们用 TCon 编码，避免错误的合一化。

¹⁴在这里我们仍然清晰地分类型与值，但这一界限在后文的基于依赖类型的定理证明器中愈发模糊。

运算符	结合性
构造子应用	左
前缀算符	
后缀算符	
中缀算符	左
中缀算符	右

Table 2: 算符在模式中的优先级

<code><ctor></code>	constructor
<code>" _"</code>	wildcard
<code>"{" sepBy1(", ", <pat-field>) "}"</code>	record pattern
<code>"(" sepBy1(", ", <pat>) ")"</code>	product pattern
<code>"(" <pat> : <type-expr> ")"</code>	ascribed pattern
<code><literal></code>	
<code><pat-field> ::= <field> = <pat></code>	

注意某一构造子模式的元数必须与其子模式 (sub-pattern) 的数目相等, 即在这个上下文的构造子必须是完全应用。Mixfix (即前缀、中缀、后缀) 运算符具有以下优先级 (越高者越先): [Table 2](#)。

下面叙述各基本模式。

变量与通配符模式 将一个标识符与值关联起来, 例如 `let fst (x, _) = x`. 与其他语言不同, 模式不一定是线性的, 即单个变量在一个模式中 can 绑定多次, 且新绑定覆盖旧绑定。但是, 应当注意可能的语义问题:

```
let prodEq : ∀ a b, a × b → Bool
  | (x, x) ⇒ true
  | (x, _) ⇒ false -- [WARN] Found redundant alternatives at 2) #[(x, _)]
```

第一个分支并非测试元组两个元素是否相等, 这是显然的: 给定任意模式, 没有固定的方法测试某一数据结构的两部分是否相等。要测试相等, 应当利用与类型类 Eq 相关联的 (=) 方法 (method)。

字面量模式 可用于测试字面量是否相等:

```
let strToBool : String → Bool = fun
  | "true" ⇒ true
  | "false" ⇒ false -- [WARN] Partial pattern matching, an unmatched candidate is [_]
```

括号括住的模式 在混有运算符的模式中起作用:

```
type List a = Nil | Cons a (List a)
infixr 90 "::" ⇒ Cons
let f1 (x :: y :: _) = (x, y)
let f2 ((x :: y) :: _) = (x, y)
```

注意 f1 与 f2 相差甚远 (因为用户自定义的 (::) 算符是右结合), 这在编译器自动生成的 *Tigris*' System F IR 很显然:

```
let (::) : ∀ α, α → List α → List α = (λ α. Cons@α)
let f1 : ∀ α, List α → α × α =
  (λ α. fun ?x₀ : List α ⇒ match ?x₀ with | Cons x (Cons y _) ⇒ (x, y))
let f2 : ∀ α, List (List α) → α × List α =
  (λ α. fun ?x₀ : List (List α) ⇒ match ?x₀ with | Cons (Cons x y) _ ⇒ (x, y))
```

就如同数学中使用括号提高某个项的运算优先级，自然，在模式中使用括号亦如此。

构造模式 有格式 $C_i p_1, \dots, p_n$ ：它匹配所有自 C_i 构造出，且其参数又满足子模式 $p_1, \dots, p_n$ 的构造值。当参数与子模式数量不匹配时，编译器会抛出元数不匹配错误。

元组模式 有格式 $(p_1, \dots, p_m)$ ，因为元组可以看成是由二元逗号构造子 $(-, -)$ 构成的构造模式，而该二元构造子又可看作是右结合的中缀运算符，因而元组、元组模式都是右结合的。

结构体模式 与结构体字面量相似，遇到具有相同栏位的模式时会产生句法分析歧义并报警。

```
type Point = {x : Int, y : Int} -- Point' 定义和 Point 相同
let sum {x, y} = x + y -- [WARN] ambiguous record pattern (using default 'Point'),
                        -- candidates are [(Point, #[x, y]), (Point', #[x, y])]
let sum' ({x, y} : Point') = x + y
```

结构体是归纳类型的语法糖，结构体模式亦同，因而，可以用常规的构造模式去解构之：

```
class Functor (m : 1) = {map : forall a b, (a → b) → m a → m b}
let getMap (Functor f) = f
```

2.6 表达式

表达式是 *Tigris* 的核心。非正式地，为了方便后文的描述，我们定义以下元变量：

- p_i 表示模式匹配中的第 i 个模式；
- C_i 表示归纳类型的第 i 个构造子；
- e, e', e_i 表示任意表达式，其值视给定的上下文（环境）而定；
- v, v', v_i 表示值；
- $a, b, c, d, e, x, y, z, w$ 表示标识符；
- f, g, h 表示标识符，在语义上，表示函数；
- $\llcorner$ 是黑板风格字体，在元语言（metalanguage）中普通分界符（delimiter）用光时用作分界符；
- 希腊字母 α 基本表示类型。若字符上方有一横杠 $\bar{\alpha}$ 则表示一系列这样的字母，并有隐式编号 $1, \dots, i$ 。

任一 *Tigris* 表达式，总是将经过足够多的变换操作而被翻译到其核心代数 **Expr** 中。以下给出表达式的 BNF 语法规格。

<expr>	::= <expr-infix> (<expr-infix> : <type-expr>)	ascription
<expr-infix>	::= <expr-left> <infix-op> <expr-infix> <prefix-op> <expr-left> <postfix-op> <expr-left> <expr-left>	infix operator prefix operator postfix operator
<expr-left>	::= fun (<expr-binder>+ ARROW <expr> <expr-match>) let "rec"? <pat'> = expr in <expr> let "rec"? <let-binder> sepBy("and", <let-binder>) in <expr> if <expr> then <expr> else <expr>	lambda abstraction pattern matching let let(rec) expression conditional

	rec <expr>	fixpoint; deprecated
	match sepBy1(",", <expr>) with <expr-match>	pattern matching
	<expr-funapp>	
<expr-funapp>	::= <expr-funapp>? <expr-atom>	function application
<expr-atom>	::= <literal>	
	<expr-ctor>	
	<lower-ident>	variables
	"(" sepByN(",", <expr>) ")"	unit
		N = 0
		N = 1
		N ≥ 2
	"{" sepBy1(",", <expr-field>) "}"	record literal
<expr-ctor>	::= <upper-ident>	constant constructors
<expr-match>	::= (" " sepBy1(",", <pat>) ARROW <expr>)+	pattern match body
<expr-field>	::= <field> = <expr>	
<expr-binder>	::= <id> <pat'>	binders
<pat'>	::= "(" <pat> ")"	parenthesized <pat>
	<literal>	except literal patterns
<let-binder>	::= <pat'> = <expr>	single-pattern binding
	<ident> <expr-binder>* <type-expr>? <let-rhs>	value/function binding
<let-rhs>	::= "=" <expr> <expr-match>	let binding RHS
<literal>	::= <intlitt> <strlitt> "true" "false"	
ARROW	::= "⇒" "→"	

读者应当注意，词法分析器 ARROW 同时接受 $\Rightarrow$ 与 $\rightarrow$ ，但在实际编程中，我们只用前者，这是因为后者的增加主要是为了提供一定程度上的 ML 系向后兼容性。

基本表达式 指的是在 <expr-atom> 中列出的所有语法，加上函数简写的语法糖。

函数简写语法 指的是受 Lean 启发的括号语法糖，在括号内，所有的下划线 ($_$) 或中点 ($\cdot$) 会按语法树的前序遍历 (preorder, 即 DFS) 被翻译到函数的绑定位置 (function binder)。这类似 Haskell 的运算符分割 (operator sectioning) 语法糖。我们的语法糖仍遵守运算符的先后顺序，如 Figure 2.

$(_ + 1)$	<code>fun x ⇒ x + 1</code>
$(_ + 1 * 2 / _)$	<code>fun x y ⇒ x + 1 * 2 / y</code>
$(id _)$	<code>fun x ⇒ (id : forall a, a → a) x</code>

(a) 函数简写语法糖

(b) 解糖结果

Figure 2: 函数简写语法糖的对比

被括号括起的表达式亦可以添加类型标注，如上面的例子所示。

柯里化的函数应用与定义 函数形式的值 (functional value) 是柯里化的¹⁵，也就是说，函数的部分应用 (partial application) 是允许且在函数式编程实践中经常被使用的。请读者回想前面我们提到的并列式函数应用语法，体会这一语法之于柯里化的必要性：常规语法 $f(1,2)$ 其实隐含地告诉用户该函数必须完全应用，而 $f \ 1 \ 2$ 则淡化了这一信息，用户使用部分应用 $f \ 1$ 也不会感觉违和。而实际上，Tigris 函数定义可以看作是普通工业语言的超集，函数 $f(x, y)$ 表示一个接收 2-元组的一元函数，在语义上等价于需要完全应用，可用于模拟工业语言的函数定义与应用，虽然这种模式并不常用。

¹⁵构造子与函数构成所有函数形式的值集合的划分。

函数应用 靠左结合, 使用多个表达式并列的语法。此外, *Tigris* 是按值调用 (call-by-value, CBV), 因而要求表达式 $e\ e'$, 首先求 e 并确保其是一函数形式的值, 其次求 e' 。

Lambda 抽象 具有格式 $\text{fun } p_1 \cdots p_n \Rightarrow e$ 。

函数定义 具有以下两种形式:

$$\begin{array}{l} \text{fun} \\ | p_{1,1}, \dots, p_{1,m} \Rightarrow e_1 \\ \vdots \\ | p_{n,1}, \dots, p_{n,m} \Rightarrow e_n \end{array} \quad (1)$$

$$\text{fun } p_1 \cdots p_m \Rightarrow e \quad (2)$$

而上述两种形式无非是以下形式的语法糖, 且与之恒等:

$$\begin{array}{l} \text{fun } x_1 \Rightarrow \cdots \text{fun } x_n \Rightarrow \\ \text{match } x_1, \dots, x_m \text{ with} \\ | p_{1,1}, \dots, p_{1,m} \Rightarrow e_1 \\ \quad \text{这之下仅适用形式 (1)} \\ | \overbrace{p_{2,1}, \dots, p_{2,m} \Rightarrow e_2} \\ \vdots \\ | p_{n,1}, \dots, p_{n,m} \Rightarrow e_n \end{array}$$

语法上的直觉感受告诉我们, 最简形式 (canonical) 的 Lambda 抽象只接受一个参数, 多参数函数本质上是多个单参数函数的嵌套, 这就是柯里化的要义。

Let-绑定 & 表达式 与大多数语言区分值定义与函数定义、全局定义与局部定义的做法不同, 作为函数式语言的 *Tigris* 将函数视作头等公民, 与其他值待遇相同: 无论是传统意义上的值或函数、全局与否的定义, 全部通过 Let 或 Letrec 进行绑定——语法构造 Let/Letrec 将标识符与值关联起来, 作用域为本地或全局。

命名规则 (Nomenclature) 我们规定, “Let-绑定” (let-biding) 描述上述 BNF 语法中的 $p_n = e_n$ 部分, and-分支则是隐含存在的 (除非显式说明不存在); “Let-表达式” (let-expression) 则在这之上包含程序主体部分 in <expr-body>。

本地非递归定义 语法构造 $\text{let } p_1 = e_1 \text{ and } \cdots \text{and } p_n = e_n \text{ in } e$ 首先并行¹⁶求出 $e_1 \cdots e_n$ 并将结果与模式 $p_1 \cdots p_n$ 作匹配, 若全部成功, 则在由上述模式匹配中产生的绑定充盈的词法作用域中求 e , 并以该值作为整个 Let-表达式的返回值。¹⁷在实现上, 这样的 Let 绑定会直接被翻译为基本语法构造 **match .. with**。

上述的最简 Let-绑定形式可以用于直接绑定 (或者在传统意义上, 定义) 函数形式的值, 但 *Tigris* 也提供了语法糖用于定义函数。在一个 Let-表达式中, 函数可以通过 $\text{let } f \mid p_1 \cdots p_n = e$ 、模式矩阵 (pattern matrices) 式的 $\text{let } f \mid p_{i,j} \Rightarrow e$, 或这两种形式的混合 $\text{let } f \mid \bar{p} \mid \bar{p} \Rightarrow e$ ¹⁸ 定义函数, 例如:

¹⁶理论上的 (在类型检查器中); 实际行为取决于后端。Cf. 嵌套 Let。

¹⁷读者应当注意, 绑定是并行进行的。绑定到 e_i 的标识符一定在 e_{i+1} 中无定义。

¹⁸回忆: 在某一对象上加横线表示一系列这样的对象。

```

let prodSum (x, y) z w = x + y + z + w
and x = 1 and y = 2 in prodSum (x, y) 3 4

let (x, y) = (1, 2) in x + y ;; let (true) = false -- 报错, true 不等于 (无法匹配) false

let compose f g x = f (g x) in -- 函数复合操作符
let add20 = (_ + 20) and minus30 = fun x => x - 30
in compose add20 minus30 50

```

第三个例子 ($0 = 1$) 会触发类型检查器的报警:

```
[WARN] Partial pattern matching, possible cases such as [false] are ignored
```

这说明类型检查器能够检查某个模式是否匹配全面 (exhaustive)。这是显然的, 对于一个匹配不全面的模式, 如果右手侧出现了无法匹配的值, 匹配失败, 程序在该点的行为即是未定义的。我们规定, 如果 runtime system 在运行期检测到匹配失败, 将通过 Common Lisp 的条件系统 (condition system) 抛出一个低级错误, 上面的例子为例, 运行期系统会抛出无法恢复 (nonrecoverable) 的 MATCH-FAILURE 错误, 由 LISP debugger 中止正在运行的程序, 等待用户输入:

```

debugger invoked on a MATCH-FAILURE in thread
#<THREAD .. RUNNING ..>: No branch can be matched against #[Toplevel]
Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.
restarts (invokable by number or by possibly-abbreviated name):
  0: [ABORT] Exit debugger, returning to top level.

```

本地递归定义 通过 Letrec 语法构造引入:

$$\text{let rec } f_1 p_1 \cdots p_n = e_1 \text{ and } \cdots \text{ and } g \cdots = e_n \text{ in } e$$

Letrec 语法结构的功能, 若过分精简地概述, 即: 在求 e_1 到 e_n 的同时, 类型检查器将等号左手侧的名字视为已经绑定。在实际使用中, 这一构造的重要之处不言而喻: 在 *Tigris* 中唯一可以提供 (互) 递归语言功能的语法结构。我们以一个稍微复杂的例子 (树与森林) 体现互递归的函数、类型 (后述) 之间的配合使用:

```

mutual type Tree a   = Empty | Node a (Forest a)
      type Forest a = Nil    | Cons (Tree a) (Forest a) ;;
let rec countTree | Empty    => 0
                  | Node _ f => 1 + countForest f
and countForest  | Nil       => 0
                  | Cons t f => countTree t + countForest f

```

其中, countTree 用于计算某棵树连带的森林里的树的数目 (包括自身); countForest 用于计算一片森林里的所有树的数目。

刚接触函数式编程的用户可能更希望用更“函数式一点”的风格来定义上述例子中出现的函数, 但可能会犹豫因为他们不知道下面这样的定义是否能够被 *Tigris* 接受: **let rec $f = \text{fun} \cdots$ and $\cdots$ and $g = \text{fun} \cdots$ in e** ——答案是确定的, 正如同前述的非递归 Let, Letrec 的最简式即为上面的形式。我们之前叙述的形式都无非是语法糖。这一形式定义 f, g 为在 e 内有绑定的互递归函数。这一特性在核心代数中有直接体现: 无论某个 Letrec-绑定为递归与否, 其一定会被翻译到 AST 的 Let 结点; 唯一区别是 Letrec-绑定的右手侧按其形状, 有可能被 Fix 结点包住。

Letrec 与 Let 在语义上的区别是微小但复杂的。我们基本上按照 OCaml 的实现但做了更多的简化且更严格。

Remark (启发式区分算法 Partition heuristic). 一个 Letrec-绑定的右侧必须具有 fun .. 的形状; 否则照 Let-绑定论处。

利用这一简单¹⁹ 的启发式算法²⁰, 我们将一个 Letrec-绑定组划分为两组: 一组严格递归²¹, 另一组则成为普通的 Let-绑定。除此之外, 我们还要求 Letrec-绑定的左侧必须是标识符 (变量), 这与 Haskell 的实现不同, 后者将之翻译为不动点组合子的应用, 搭配模式匹配。这些要求已经足够严格, 甚至能够将一些原本合法的程序算作不合法:

```
let rec x = (1, x) and y = (2, x)
let rec x = (1, y) and y = 1
```

范例一在 CBV 求值语义中是不合法的, 因为其构造了互相无限循环的两个元组; 而范例二则是合法, 但因为上述限制, 在 *Tigris* 中属于不合法。但我们认为这些限制无足轻重, 虽然范例二是强行设计出来的, 从编程实践的角度看这一代码风格并不实用且可读性差; 此外, 笔者也无法在找到/想到更实用的, 且必须要用上述写法的例子, 因而我们认为这些不足为道的小例子难以成为需要更精确的 Letrec-绑定限制的理由。

全局定义 与本地定义几乎一致, 但如其名, 仅限于顶层 (toplevel)。顶层的 Let(rec)-表达式不必再拥有其程序体, 因而退化为一个独立的定义。多个顶层 Let 可以有选择的用 ; 作分隔, 提高可读性。

类型推论与泛化 发生在许多地方。之所以放在 Let 的小标题下, 是因为 HM 的自动泛化机制与 Let 构造有相关 (通称 let-polymorphism)。

类型推论将用户从手动添加类型标注的繁琐工作中解放出来, 同时保证程序的类型正确性 (前提程序本身正确)。与其他一样具有类型推论的语言不同, 得益于类型系统的小心设计, *Tigris* 具有全局类型推断, 这使得类型检查器能够从零类型标注 (字面义) 推断整个程序的类型。具体来说, 这是因为我们的类型系统具有主类型 (principal types), 即能够推断最泛化的类型的能力。换句话说, 从 Python/Ruby/LISP 或任何一个不需要类型标注的动态语言迁移过来的用户都能保持他们不写类型标注的习惯 (只是语法上), 但同时又能享受到静态类型带来的程序安全保障。

泛化的对象是模式 (scheme) 且发生在 Let-绑定处。泛化与类型推论是一起工作的, 与其他语言需要利用特殊语法构造实现多态不同, *Tigris* 充分利用自动泛化机制, 保证 “能泛化的程序一定会被自动泛化且无需类型标注” 这一性质。

搭配这两个不同于一般语言的特性, 在进行函数式编程实践时, 与常见的工业语言相比, 用户能获得异常好的可读性。我们不妨同 Typescript (使用渐进类型系统, gradual typing)²² 作一比较:

控制结构 是用于调整语句执行顺序的语法结构。 *Tigris* 具备以下数种控制结构:

顺序执行结构 目前没有内置, 用户可以使用嵌套 Let 表达式替代。

条件语句结构 具有以下格式: if e then e_1 else e_2 ; 其值为 e_1 当且仅当 e 返回 Bool 类型的构造子 true, 否则为 e_2 。

¹⁹不具备自由变量检查 (free occurrence checking), 或 OCaml Manual 第 12 章第 1 节中的大多数条件。

²⁰OCaml 的 静态可构造 (statically constructive) 的条件之一。

²¹指其中不应出现非函数形式的值的递归定义。

²²依笔者意见, 是语法设计不当, 噪音比较大, 可读性较差的代表性语言之一。

```

function k<T, U>(t: T, _: U): T {return t}
const id: <T>(t: T) ⇒ T = (t) ⇒ t
function map<T, U>(f: (t: T) ⇒ U, xs: T[]): U[] {
  let xss = []
  for (let x of xs) {
    xss.push(f(x))
  }
  return xss
}

let sns = map((n) ⇒ n.toString(), [1, 2, 3]);

```

(a) Gradual typing w/ limited local type inference

```

let k x _ = x
let id x = x
let rec map f
  | Nil ⇒ Nil
  | x :: xs ⇒ f x :: map f xs

let sns = map toString $ 1 :: 2 :: 3 :: Nil

```

(b) Static typing w/ global inference/generalization

Figure 3: Typescript 与 Tigris 进行函数式编程实践的比较

分情况讨论 (case analysis) 结构 亦称模式匹配, 表达式

```

match e1, ..., em with
| p1,1, ..., p1,m ⇒ e1
| p2,1, ..., p2,m ⇒ e2
|
| pn,1, ..., pn,m ⇒ en

```

将行向量表达式 $e_1, \dots, e_m$ 与模式矩阵按并行求值顺序相匹配²³, 并返回第一个成功的分支的值。在编译期, 类型检查器通过一个高效的, 矩阵模型的算法, 对模式矩阵开展完全、冗余 (redundancy) 性检查, 其中前者用于检查某一模式矩阵是否能够匹配所有情况, 后者用于检查这一矩阵中的任意两个行向量是否重复, 若重复则像用户报错, 并立即删除多余的行向量。直观上这一行为是正确的: 若出现多个重复的行向量, 则依先来先得原则取第一个分支; 在运行期, 当所有分支都匹配失败时, 程序会抛出运行时错误 MATCH-FAILURE。

```

let (1, 2) = (1, 2) -- [WARN] Partial pattern matching, possible cases such as [(_, _)] are ignored
let f | (1, 2) ⇒ 1 + 2 -- [WARN] Found redundant alternatives at 3) #[(_, y)]
    | (x, _) ⇒ x | (_, y) ⇒ y

```

读者需要注意, 完全性检查与运行时的模式匹配毫不相干: 考虑上方范例一, 根据右手侧的值, 我们很清楚左侧模式一定匹配成功, 但同样的结论并不能适用于更复杂的右手侧。模式匹配仅在运行时执行, 编译期的完全性检查仅仅是根据左侧模式及对应表达式的类型作出完全性判断。——具体的行为与代码由编译器的模式匹配编译模块决定和生成。

模式匹配在语义和实现上可以看作是命令式语言中的 switch 语法结构的超集, 它允许用户使用声明式 (declarative) 的写法来优雅的解构某数据结构, 是函数式编程的核心功能之一。

对“声明式”一词的理解, 读者不妨联想大量使用这一思考方式的数学。考虑概率的公理化定义, 这就是典型的声明式表述: 它仅告诉读者应当满足何种条件, 才能算作“概率”, 而不是从如何计算“概率”本身定义“概率”。同理, 当我们利用模式匹配处理数据结构时, 并未显式地写出各种投影 (例如数组访问) 操作, 而仅仅是告诉编译器: “满足这一模式的数据结构才能通过”²⁴

²³Cf. 积模式则是按前序遍历顺序匹配的。

²⁴简单地说, 声明式只表述程序应该做什么 (what); 命令式只表述程序该怎么做 (how)。

运算符	优先级	类型	对应实现
*	70	infixl (中缀左结合)	原生整数乘法
/	70	infixl	原生整数除法
+	65	infixl	原生整数加法
-	65	infixl	原生整数减法
=	50	infixl	原生整数等号
\$	1	infixr (中缀右结合)	函数应用
@@	1	infixr	函数应用

Table 3: 预定义的运算符

运算符 是特殊的，被绑定到某个表达式（实现）上的字符串。运算符可以被句法分析器识别，在代码中串连使用。

Tigris 与其他语言最明显的差别是其允许用户从给定的无穷字符串集合里定义运算符，并同时设置这些运算符的优先级（precedence）与结合性（associativity）。

用户自定义的运算符是一 4-元组 ($op, prec, assoc, impl$) 或 3-元组 ($op, prec, impl$)，这些信息是通过以下几种运算符声明语句获取的：

```

<infixl-decl> ::= infixl ":"? <op> <prec> ARROW <expr> ; ⇒ (<op>, <prec>, L, <expr>)
<infixr-decl> ::= infixr ":"? <op> <prec> ARROW <expr> ; ⇒ (<op>, <prec>, R, <expr>)
<prefix-decl> ::= prefix ":"? <op> <prec> ARROW <expr> ; ⇒ (<op>, <prec>, <expr>)
<postfix-decl> ::= postfix ":"? <op> <prec> ARROW <expr> ; ⇒ (<op>, <prec>, <expr>)

```

其中 $\langle op \rangle$ 表示运算符对应的字符串，且为 $\langle strlit \rangle$ 的子集。

Definition (运算符字符串候选 Operator candidate strings). 与 Haskell 或 OCaml 从给定的一个集合内选取可用运算符字符串不同，类似 Lean, *Tigris* 将 $\langle op \rangle$ 定义为任意字符串，除去以下几种特殊情况：

- 由数字开头的，
- 包含或由以下字符开头的： $\llbracket, ()[]\{\}\llbracket$ ，
- 除开头以外，包含 $\langle blank \rangle$ 的，
- 与此关键词表中的任意符号相等的：Table 1.

此外，运算符字符串周围的空白字符会被自动剔除。

三种类型（前/中/后缀）的运算符的优先级（再加上函数应用）与模式中的相同：Table 2. 虽然存在无穷多的候选字符串可供用户选择用来声明运算符，但我们也用同种方式内置了一些常见的运算符（其定义仍然可被覆盖）^{25 26}：

这其中，\$ 与 @@ 的使用是两个经常在函数式编程实践中出现的小技巧，通过巧妙地运用优先级/结合性自定义机制——在最低优先级下，我们可以利用这些算符来避免右结合的括号组，也就是说，下面三种写法等价，但后两种显然可读性更强：

```
f (g (h (x + y))) ;; f $ g $ h $ x + y ;; f @@ g @@ h @@ x + y
```

²⁵虽然在实际应用中，更常见的思路是将运算符绑定到类型类的方法上以方便调用接口方法，但此处我们去繁从简；此外，目前而言我们的系统中也确实不包含序章定义（prelude，即包含某一语言最基本定义的源文件，由该语言本身写成）。

²⁶按常规做法，通过硬编码手段，语法构造的优先级应小于任意运算符，也就是说，`let n = 10 in n + x` 应当是 `let n = 10 in (n + x)`，而非 `(let n = 10 in n) + x`。

对于某些函数体较大的函数来说，用户可以使用该技巧来避免在函数体两侧加括号，提升代码编写的流畅性；同时，这一技巧也被广泛的运用在本项目的实现当中。

不仅如此，对于前缀/后缀算符而言，我们可以把一个字符串同时声明为这两种类型的运算符，例如（其中 `fact` 是阶乘，而 `not` 是取反）：

```
postfix 65 "!" ⇒ fact ;; prefix 65 "!" ⇒ not
let main = (50!, !true)
```

但读者需要注意，目前无法将同一字符串同时声明为前缀/中缀或后缀/中缀。常见的用法是将列表（广义表、内存中是链表）的 `Cons` 构造子绑定到 `(::)`，使得其可以同时表达式和模式中使用²⁷：

```
type List a = Nil | Cons a (List a) ;; infixr 67 " :: " ⇒ Cons
let car (x :: _) = x ;; let cdr (_ :: xs) = xs
```

顶层运算符声明可以有选择的用 `;;` 作分隔，提高可读性。

外部声明 利用了 *Tigris* 的 FFI 功能来声明外部函数，“外部”即这些标识符所绑定的值的定义对编译器不可见，因为其并非在本语言内部定义的。

```
<extern-decl> ::= extern <ident> <extern-symb> : <scheme>
<extern-symb> ::= <strlit>
```

外部声明引入了一个不透明（opaque，即无定义的）的标识符 `<ident>`，其类型为 `<scheme>`，被绑定到外部函数（以名字引用的）`<extern-symb>` 上。虽然 *Tigris* 是一个纯函数式语言，但与 Haskell 或 Lean 不同，其并不强求用户——若要直接引入具有副作用的函数，就可以利用 FFI. 我们认为这是一个高级功能，对此感兴趣的读者参见后文更细致的描述：subsection 4.5.

2.7 类型、类型类与实例定义

数据类型定义 用于引入新类型。其将声明的类型构造子绑定到两种类型定义之一：归纳类型或结构体类型；虽然后者不过是前者的语法糖而已。

```
<type-decl> ::= mutual <type-decl1>+ ";;" (mutual retype definition)
              | <type-decl1>
<type-decl1> ::= TYPE <type-ctor> <type-binder> = <type-constrs>
<type-field> ::= <ident> : <type-expr>
<type-constrs> ::= <type-record>
                  | ("|" <type-ind>)+
<type-record> ::= "{" sepBy1(",", <ident> : <type-expr>) "}"
<type-ind> ::= <expr-ctor> <type-atom>+ (inductive type branch)
TYPE ::= type | data
```

读者注意，TYPE 词法分析器同时匹配 `type`、`data`，但应使用前者，后者只是为增加 Haskell 的兼容性。

归纳类型 在工程上亦称代数数据类型，是函数式编程特征之一的发达的类型系统带来的独特机制。——和上文所说的模式匹配一样，对 C 熟悉的读者可以将其看作 `union` 和 `enum` 关键字的一个超集。

从理论的角度讲，结构体类型与元组类型无异，且后者可由前者编码（实现），前者无非是带名字的后者——它们表示的是“且”（严谨地说，积）关系；而表示“或”（严谨地说，和）关系的类型就称为不交并类型（disjoint union）、带标签并类型（tagged union）——这一类型具有多个分支，每一个分支由一个构造子（即标签）标识。要想构造这一类型的数据，必须要从数个分支（构造子中）选取一个进行构造，也就是说，任一构造值不可能同时由多个构造子产生，因而称之不交并。前面说过，

²⁷`car` 与 `cdr` 是两个十分著名的列表访问函数，起源于 LISP.

构造子是函数形式的值，因为构造子可携带多个不同类型的数据（可以定义多个匿名栏位），这亦可以看作一种元组类型，所以在每个“或”类型分支的内部也大量存在着“且”类型。

“或”类型与“且”类型统称归纳类型。之所以称其“归纳”，是因为“或”类型的每个分支从证明论的角度而言可以看作是引入该类型（命题）的值（命题的证明）的一种情况，而所有分支则构成该类型构造方式的穷举（能够证明该命题的所有情况，即数学证明中的分情况讨论），因而称之归纳²⁸；而称其为代数数据类型，是因为这一概念的理论基础来源于范畴论的自函子²⁹代数（algebra for an endofunctor, F-algebra），而其中“且”类型即 categorical product $((\times), \text{积})$ ，“或”类型即 categorical coproduct/sum $((+), \text{余积/和})$ 。同时，从这一定义中可以导出任意归纳类型的消去子（eliminator），亦称计算法则，属于 catamorphism³⁰的一种，其代表如 fold/foldr。实际上，各大定理证明器亦确实会为所有归纳类型（在证明器的上下文中是 inductive family/indexed family）导出消去子，作为最基础的计算规则。具体地说，任意消去子的参数都应该包含

- 动机 (motive)，即消去子的目标类型（想要证明的命题）。从操作语义的角度考虑，这是一个续点 (continuation)，会被定理证明器的内核应用；
- 小前提 (minor premise)，每一分支（构造子）都是一个小前提（除去空类型³¹，因为其没有分支）；
- 大前提 (major premise)，即类型本身的一个证明。

但这些内容已超出本文讨论范围，感兴趣的读者可了解递归论（计算理论）、证明论和范畴论。

归纳类型具有较强的理论特性，根植于数学，但这并不妨碍其在工程上具有的重大意义，一些现代的主流语言，如 Rust、Swift 亦已支持这一特性，且广泛运用于编程实践中。我们不妨先了解如何定义一个归纳类型。

在 *Tigris* 中，归纳类型定义具有以下格式：

$$\text{type } T \bar{\alpha} = C_1 \sigma_{1,1} \cdots \sigma_{1,c_1} \mid \cdots \mid C_n \sigma_{n,1} \cdots \sigma_{n,c_n}.$$

这一定义引入 n -元新类型构造子 T 和值构造子 $C_1, \dots, C_n$ ，且该类型构造子必须完全应用，如前文所言。以下式子给定一个值构造子的类型：

$$C_i : \sigma_{i,1} \rightarrow \cdots \rightarrow \sigma_{i,c_i} \rightarrow T \bar{\alpha}.$$

其中小写的 c_i 表示 C_i 的元数。³²

归纳类型的工程意义在于其用严谨的理论编码工业语言中出现的一些现象：例如 JVM 语言（例如 Java）的对象可以是 NULL。这从函数式语言的角度看来并不严谨，因为具有类型 τ 的值就应该是这一类型的值，而不是空指针。而空对象通常被某些出错的函数返回，为此，在函数式编程实践中通常用 Option 编码可空对象：`type Option a = None | Some a`，而许多语言具有的错误处理，无非表示“某一函数能够返回一个错误或正确值”这件事，因而也可以被编码为：`type Except e a = Error e | Ok a`；搭配前述的模式匹配，就能以类型安全的手段在编译期处理所有例外情况（Error 或 None）；而搭配纯函数式的函子/单子编程设计思想，就可以实现工业语言的遇错短路等控制流语义。虽然本项目完全使用纯函数式设计模式，但本文并非函数式编程教程，因而我们略过具体的叙述。

²⁸事实上，各大定理证明器在定义命题、谓词时使用的即是这种解读。

²⁹从类型论作为范畴论逻辑 (categorical logic) 的范畴论语义 (categorical semantic) 解读看来，编程语言中的任一高阶类型都是单一类型宇宙（范畴 **Type**）上的自函子；但在 Lean 这样具有非累加（与 Coq 不同）的 (cumulative) 无穷有序的类型宇宙中则不一定，因为存在泛类型宇宙 (universe polymorphic) 的函子定义。

³⁰cata-态射，cata- 是希腊语词根，表示某样事物的崩坏（消去）。

³¹即 empty type。许多编程语言的 void、nothing 类型实际上都是空类型。这在类型论的角度看来，是存在逻辑问题的：空类型即没有分支、不能被证明（构造出来）的类型。在一个合理的逻辑系统中，如果出现空类型的值，说明系统存在矛盾；在编程实践中，通常用其表示无法停机的程序、抛出错误而崩溃的程序类型。现代语言通常使用 Unit（幺类型）替代 void，表示任何可以忽略的值，指任何基底集（当把类型作为代数结构来看待时）都是单元素集合 (singleton) 的类型，即这一类型只有一个值。

³²我们要求构造子的结果类型 (result type) 必须和正在定义的类型相匹配，因为目前并不支持枚举族（亦称泛化代数类型，generalized ADT, GADT）的定义，这一扩展特性已经存在于后续的 *Tigris*d 定理证明器且支持更完整。

前面强调多次，结构体类型是归纳类型的语法糖，这一解糖的指称语义是

$$\llbracket \text{type } T \text{ args}^* = \{(fid : \sigma)^*\} \rrbracket := \llbracket \text{type } T \text{ args}^* = T \sigma^* \rrbracket$$

其中 tm^* 表示 tm 作为序列结构的展开/绞接 (splice)。结构体定义可以与归纳类型定义混用：

```
type RGB = {r : Int, g : Int, b : Int}
type Color = Red | Green | Blue | Yellow | Black | White | Custom RGB
type Tree a = Lf | Br a (Tree a) a
```

同义类型（等价约束）和上下文（谓词）暂时不被归纳类型定义所支持，且 *Tigris* 目前并不做严格正向/逆变位置检查 (strict positiveness/contravariant position checking)。

Definition (正向位置). 一个位置是正向的，当且仅当

1. 其不在函数的参数位置，
2. 其不是除归纳类型的之外的类型构造子的参数（在 *Tigris* 中已经满足）。

因为 *Tigris* 是编程语言，从设计角度看来我们也不准备加入这一限制因为其过强，将部分合法的类型定义算作不合法。

类型类定义 类型类是存在于函数式编程中与面向对象编程的接口相呼应的概念，但前者是良定义且更高效、强大的，体现于其允许临时/参数多态 (ad-hoc/parametric polymorphism) 在函数式编程中以严谨的方式存在；体现于方法是静态分配的，在编译期即完成工作；体现于其能够在多个类型参数上做分派；还体现于其搭配约束量化可以通过一个实例为多个具有共性的类型实现同一类型类。因为两者在软件工程方法学上的意义相似，往后除非特指，我们亦用“接口”一词表示类型类。

```
<class-decl> ::= class <class-ident> <type-binder> = <type-record>
<class-ident> ::= <type-ctor>
```

读者注意，我们使用 `class` 关键字定义类型类不代表这和面向对象范式的类有任何关系，恰巧相反：后者的“类”用于表示一群具有相同方法、栏位的对象；前者则表示一群具有共性（实现了某个接口）的类型。实际上，函数式的“类”一词直接来源于集合论中的 `class`：集合的集合并非集合（根据正则公理），称作类。

类型类在其他语言中亦有出现，虽然其拓展功能可能有差异，例如 Rust 的 `trait`，Swift 的 `protocol`，即便是 C++，也在最新版本中引入了 `concept`，类型类实现的一种。

一个类型类定义引入一个新类与其附带的，全局可用的新方法。类与结构体在实现上相似，因为前者基于字典传递 (dictionary passing)，即通过把每一个类实例（由结构体编码）传递给调用对应方法的函数实现方法分派。定义格式为：

```
class C  $\bar{\alpha}$  = <type-record>
```

其方法的类型由以下式子给定：

$$f_i : \forall \bar{\alpha}, \bar{\beta}. \sigma_i$$

其中 $\bar{\beta}$ 是方法层级额外绑定的 TV³³， σ_i 则是 f_i 的签名。考虑

```
class Eq a = {eq : a -> a -> Bool} ;; class HEq a b = {heq : a -> b -> Bool}
class Functor (f : 1) = {map : forall a b, (a -> b) -> f a -> f b}
let main = (map, eq, heq)
```

`eq` 具有类型 $\forall a [\text{Eq } a]. a \rightarrow a \rightarrow \text{Bool}$ ，`map` 具有类型 $\forall f [\text{Functor } f]. \forall a b. (a \rightarrow b) \rightarrow f a \rightarrow f b$ ，`heq` 具有类型 $\forall a b [\text{HEq } a b]. a \rightarrow b \rightarrow \text{Bool}$ ；而 `main`，即某个调用了上面所有方法且没有提供任何具体类型去精练 (refine) 类的程序，具有以下复杂的类型：

$$\forall \alpha \beta \gamma \delta [\text{Functor } \alpha, \text{Eq } \beta, \text{HEq } \gamma \delta] \forall a b. ((a \rightarrow b) \rightarrow \alpha a \rightarrow \alpha b) \times (\beta \rightarrow \beta \rightarrow \text{Bool}) \times (\gamma \rightarrow \delta \rightarrow \text{Bool}).$$

我们称出现在类型模式中的类为上下文、谓词或约束。³⁴同时还称被这样的约束限制的量化为有界量

³³这是通过多态栏位 (polymorphic field) 实现的。

³⁴读者应当注意不要同“约束求解”的约束混淆。

化 (bounded quantification)。³⁵ 读者需要注意, 默认方法 (default method) 与超类 (superclass) 目前不被类型类支持。

实例定义 正如面向对象编程中接口总是与其实实现分不开一样, 类型类亦如此。我们称一个类型类的实现为实例。

```
<inst-decl>      ::= instance ":"? <inst-scheme> = "{" <inst-method-decl> "}"
<inst-scheme>    ::= (FORALL <type-binder>+)? <inst-context>? ", " <class-ident> <type-expr>+
<inst-context>   ::= "[" sepBy1(", ", <qualified>) "]"
<inst-method-decl> ::= sepBy(", ", <let-binder>)
```

考虑到语法统一性, 实例定理的右手侧在语法上与结构体字面量一致, 但其栏位赋值可以使用类似 Let 定义的语法。

```
instance HEq String Bool = {heq | "true", true ⇒ true | "false", false ⇒ true | _, _ ⇒ false}
instance ∀a b [HEq a b], HEq b a = {heq y x = heq x y}
instance ∀a [Eq a], HEq a a = {heq = eq}
```

实例定义可以量化 TV, 与谓词搭配, 就可以表示逻辑上的蕴含 (implication) 关系 (如例二、三)。这里, 我们量化 a, b 与一个实例 $\text{HEq } a \ b$, 并用之作为 $\text{HEq } b \ a$ 的实现来表达异性等号 (heterogenous equality) 的交换律 (commutativity), 也用来表达二元关系同性等号是异性等号的子集这一事实。

关于编译器是如何在程序中寻找实例这一问题, 我们将在后文介绍实例解析 (查找) 算法, 目前只需明确以下若干规律:

1. 某一类型不能多次作为同一个类的实例;
2. 重叠实例会抛出编译期歧义错误;
3. 由于方法应用产生的歧义类型会阻断类型类繁饰并抛出编译期歧义错误。

```
class HAdd a b c = {hadd : a -> b -> c} ;; instance HAdd Int Int Int = {hadd = add}
-- HEq ?m6 Int: typeclass elaboration is stuck because of
-- metavariable(s) [?m6] induced by a call to heq.
let f' = heq (hadd 2 3) 5
```

在上面的例子中, $\text{hadd } 2 \ 3$ 被类型检查器推断为含有 MV, 导致 HEq 的实例查找失败。这是显然的, 因为异性加法的结果类型是独立的, 仅仅给定其参数并不能推出具体的结果类型。我们需要加上类型标注如 $f' : \text{Int}$ 才能令编译器找到第一行定义的实例。

虽然类型检查器能够在用户不提供任何谓词的情况下根据实现代码推断谓词, 我们仍然要求用户显式完整地给出上下文。顶层的实例定义可以有选择的用 `;;` 作分隔, 提高可读性。

2.8 实现细节

前文的部分论述涉及到了部分模块的实现细节, 下面我们将在这一部分集中的对 *Tigris* 的词法、句法分析, 类型检查做一些更细致的描述。

³⁵ 相较而言, 支持泛型的面向对象语言所具有的有界量化即为子类型 (subtype), 或者用绝大多数程序员都知道的概念说明: 继承 (inheritance)。

词法、句法分析 编译器的词法、句法分析阶段是融合实现的。我们利用函数式的单子化句法分析设计模式，典型工具是 句法分析组合子，搭配 Pratt 优先级爬坡算法，实现各种语言构件及自定义运算符的句法分析。

在实践中，组合句法分析类似递归下降分析器 (recursive descent parser)，或者说类似手写的分析器。组合子对应的文法与解析表达文法 (parsing expression grammar, PEG) 相似：两者遇到歧义语句时总是选择第一条匹配的生成规则 (production rule)。但目前而言，不同于 Lean 的实现，*Tigris* 的句法分析模块还不是 Packrat 分析器 (PEG 的其中一种)，后者能够在线性时间内完成分析。此外，因为使用单子设计模式，我们的实现仍然存在大量的回溯 (backtracking) 操作。我们定义，一个句法分析单子 (parser monad) 类型为

```
1 abbrev TParser  $\sigma$  := SimpleParserT Substring Char $ StateRefT (PEnv  $\times$  String) (ST  $\sigma$ )
```

其中 σ 是一形式参数，用于传入基于 ST 的句法分析单子。这样，我们能够在纯函数式语言中储存真正可变 (mutable)、高效的状态，因为 ST 是基于 threadlocal 的实现，能够赋予计算环境 (computational context) 真实的可变副作用；相比利用栈空间的 StateT 更高效。此外，上述定义中的单子转换栈 (monad transformer stack) 还附带了其余两个单子转换器 SimpleParserT、StateRefT，这使得我们可以在 ST 中利用类似 State 的接口方法以流畅的编写代码，同时 Substring 的使用允许通过一对指针 (分别指向字符串始/终点) 在同一字符串上做计算 (句法分析) 而不产生任何中间字符串数据，保证内存使用的高效性。

关于组合子本身是如何实现的，我们首先考虑任意两个句法分析器 P, P' ，则可以定义组合子

- *alternative* 组合子 $p \oplus q (j) = P j \cup Q j$
- *sequence* 组合子 $(p \otimes q) (j) = \bigcup \{Q k \mid k \in P j\}$

这即为函数式编程中常见的几个类型类 MonadExcept、Alternative、Seq 对应方法在句法分析组合子下的实现：这些实例，与基本的句法分析框架及常见的组合子已由 F. G. Dorais 的现有库 lean4-parser 完成，我们不再造轮子。但除此之外，我们还定义了数个重要但缺失的组合子实现函数应用、运算符的分析，例如 chainl、chainr：

```
1 partial def chainl1 (p : ParserT  $\varepsilon$   $\sigma$   $\tau$  m  $\alpha$ )
2   (op : ParserT  $\varepsilon$   $\sigma$   $\tau$  m ( $\alpha \rightarrow \alpha \rightarrow \alpha$ )) : ParserT  $\varepsilon$   $\sigma$   $\tau$  m  $\alpha$  := do
3     let x  $\leftarrow$  p; rest x where
4       rest x := (do let f  $\leftarrow$  op; let y  $\leftarrow$  p; rest (f x y))  $\triangleleft$  pure x
5 partial def chainr1 (p : ParserT  $\varepsilon$   $\sigma$   $\tau$  m  $\alpha$ )
6   (op : ParserT  $\varepsilon$   $\sigma$   $\tau$  m ( $\alpha \rightarrow \alpha \rightarrow \alpha$ )) : ParserT  $\varepsilon$   $\sigma$   $\tau$  m  $\alpha$  := do
7     let x  $\leftarrow$  p; rest x where
8       rest x := (do let f  $\leftarrow$  op; chainr1 p op  $\langle \& \rangle$  f x)  $\triangleleft$  pure x
```

类型检查器 *Tigris* 代码库中有两个类型检查器，其一较为老旧但简单且高效的 Algorithm W 实现；其二是目前正在使用的，利用约束求解算法的实现。选择后者的原因是其设计模式更适合于实现类型类功能、量化类型。在本文我们只探讨该实现。类型检查器运行在 InferC 单子中：

```
1 abbrev InferC  $\sigma$  := StateRefT CState (EST TypingError  $\sigma$ )
```

为了处理不当合一化，我们参照 GHC 的 TV 分层机制实现了一个简化的版本；此外，对于编译器而言，类型信息是宝贵的，在后续的 System F 繁饰与 IR 降级中皆有使用。因而检查器不仅检查、推断类型，还负责从原始 AST (Expr) 中产生带类型的 AST, TExpr 并将其传递给繁饰模块用于 System F 的繁饰，最后产生 System F IR (FExpr)。FExpr 同 Expr 即核心代数的定义十分相似，我们在此只列出其差异部分：

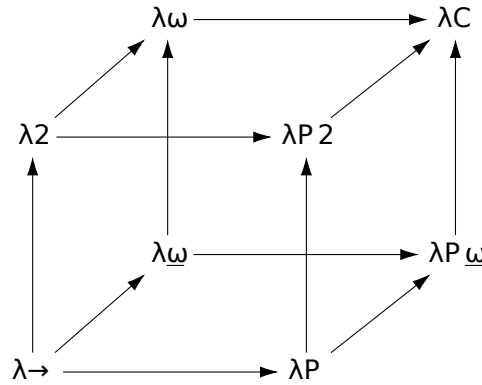


Figure 4: Lambda 方块

```

1  inductive FExpr where -- (... 其余结点略...)
2    | TyLam (a : TV) (body : FExpr)                -- 类型抽象
3    | TyApp (f : FExpr) (arg : MLType)              -- 类型应用
4    | Proj (src : FExpr) (mname : String) (idx : Nat) (ty : MLType) -- 字典投影
5  deriving Repr, Inhabited

```

应当认识到这一 AST 的表达力是超出目前 *Tigris* 的，因为其属于 Girard 提出的 System F 计算系统，允许项依赖于类型，支持 Arbitrary-rank Polymorphism 和 Impredicative types，所以亦称二阶 λ -代数。关于不同类型论的表达力，我们可以在 Barendregt 提出的 Lambda Cube 中清晰比较。从左下角到右上角，这一立方体代表的类型论的表达力逐渐增强，更具体的说，x 轴的箭头 $\rightarrow$ 表示那些类型可以依赖值的类型论，属于依值类型；³⁶ y 轴的箭头 $\uparrow$ 表示项可以依赖类型，即具备多态；z 轴的箭头 $\nearrow$ 则表示类型可以依赖类型，即具备类型构造子。

在这一立方体中， $\lambda_{\rightarrow}$ 表示简单类型 λ -代数 (simply typed λ -calculus)、 λ_2 表示 System F、 λ_{ω} 表示 System F_{ω} ³⁷，而位居表达力顶端的类型论则是 Coquand 提出的 Calculus of (inductive) Construction (λ_C)，被广泛的运用于各个定理证明器中，包括 Coq 和 Lean。

而 *Tigris* 使用的扩展 HM 系统则处于 $\lambda_{\rightarrow}$ 和 F_{ω} 之间，这是因为我们虽然支持直谓多态 (predicative polymorphism，亦称 rank-1 polymorphism)，我们并不支持 arbitrarily ranked 多态，亦不支持非直谓多态 (impredicative polymorphism)，不满足 F；此外其虽然支持 HKT，但并不支持柯里化的类型构造子，因而最相似的类型论应当是 F_{ω} 。但是即便如此，我们认为 *Tigris* 的类型系统的表达力已经胜过绝大多数具有多态的语言，因为无论是 Arbitrary-Ranked Polymorphism、Impredicative types 还是 Higher-Kinded Types，都还停留在 PL 学术界阶段，并未得到大规模的语言的支持。此外，*Tigrisd* 的类型系统是 $\lambda_{\Pi\omega}$ 与 $\lambda_{\Pi 2}$ 的简化混合体，因为我们并未完全实现主流定理证明器所用的 CIC。

然而，类型论表达力的增强并非没有代价。Hindley-Milner 类型系统之所以经典，不仅仅是因为其开创性，更因为它小心的设计。事实上，System F 及以上表达力的系统其类型推论为不可判定 (undecidable) 且必须依赖用户的类型标注，对这些系统的推论/检查工作必须在本地通过双向算法进行 (local bidirectional type inference/checking) ——即便是类型系统发达的 Haskell，也必须在使用上述高级特性时切换到另一个类型检查器——更不消说依赖类型论。所以，即便 *Tigris* 失去了很大程度的（逻辑上）的表达力，但考虑到本课题的编程子系统并不负责数学证明，更不希望失去招牌的强大类型推论功能，我们认为这一表达力牺牲是完全值得的。

回到 IR 本身，之所以仍然选用 System F IR，是基于软件工程的认识：考虑本课题的未来展望，即便之后希望加上上方的这些功能，我们也不需要耗费精力开展大型重构，可以在现有框架下直接进行补充。

同时，为了方便对编译器做 debug，我们利用与 Wadler 相似的 pretty-printing 算法打印 FExpr

³⁶即依赖类型，此处为和下文区分做调整。

³⁷Haskell (GHC) 使用这一系统，同时加上 ADT 和 Typall Equality。

AST，在前文的样例中亦有出现：输出结果与 *Tigris* 的一个子集十分相似，同时还会添加必要的类型标注、类型 lambda 抽象与类型应用。同样的打印算法也用于输出本文之后所描述的所有 IR 和 SBCL 目标代码。

类型系统、类型检查是 *Tigris* 极为重要的组成部分。在下文我们会有更细节的论述。

3 Type System of the *Tigris* language

命名方式 在本节内，词语“约束”有广泛运用。因为我们实现的是基于约束求解的类型检查器，广义上的“约束”可以指

- 相等约束 (equality constraint)，具有自反性、交换性（虽然我们只用到其一侧方向）和传递性；
- 谓词约束 (predicate constraint)，具有累积性，主要体现在这种类型的约束是可以在类型模式中累积起来的。

虽然一些时候我们狭义地使用“约束”来特指含义二，读者应当有能力根据上下文解读其具体含义，本文更多地使用广义上的“约束”一词。

简单的说，本类型系统实现称为约束变量求解实现，检查器称为约束变量求解器，在类型推断过程产生约束，之后通过合一化进行求解。

Tigris 实现了一个基于约束的 Hindley-Milner 类型系统，并添加了以下扩展功能：

谓词 记作 $C\bar{\alpha}$ ，其中 C 是谓词的头部。编译器在类型推论阶段收集谓词信息，并在约束求解之后进行实例解析，并生成基于字典传递的实现代码。

严格高阶类型 指的是高阶的 TV 和类型构造子必须完全应用。类型表达式 AST 中的 KApp 节点恰好足够实现这一功能，并使得诸如 Functor、Monad 之类的抽象（类型类）能够在 *Tigris* 中被定义，见 [section 2.4](#)。

受限的 Rank-N Polymorphism 指的是模式 (Scheme) 能够直接嵌入类型表达式（通过 TSch AST 结点），但系统本身仍然是 Rank-1 Polymorphic 的。这是因为我们通过 skolemization 的手段，对类型标注的检查做了精细控制。³⁸ 这表示当检查 $e:\sigma$ 时，我们用 skolem 常量³⁹将 σ 实例化，并检查其是否能和系统自身推断的类型合一化。不严谨但直观地说，

- skolem TV 是刚性变量，主要表现在其独一无二，且无法同其他类型合一化；
- skolem TV 在实现中是通过维护一刚性变量集确定的。这一集合用于拒绝错误的替换法则 (substitution)，否则可能导致上述 skolems 逸出作用域，产生重大错误。

约束生成与求解 约束的生成是通过类型推论进行的，且之后会在一次遍历后全部求解。求解器会返回一个替换法则（即解）与残余谓词约束，这些谓词约束最终会在被泛化的类型模式中出现。

Let-Polymorphism 指的是由 Let-绑定的变量会依 TV s.t. $tv \notin \text{ftv } \Gamma$ 被泛化，同时其谓词约束（如果有）则会转变为对应类型模式的上下文。非正式地说，表述

总是可以推断一个程序的类型，即便用户不提供任何签名

是正确的，这一论断由 HM 系统的主类型特性所保证。

³⁸且系统的自动泛化机制永远不会产生 higher-rank 类型。

³⁹这些常量由 TCon 编码，即一个 0-元类型构造子。

3.1 Formal Grammar

类型系统是建构在核心代数上的：Tigris 的核心代数是修改版的 λ -代数。在描述这一类型系统的行为规范（typing discipline）之前，我们首先给出核心代数、类型项的形式文法（formal grammar）。⁴⁰

$expr, e$	$::=$		term
		$const$	constant
		$tname$	variable
		$\lambda x. e$	lambda abstraction
		$\text{let } x_1 = e_1 \text{ and } \dots \text{ and } x_n = e_n \text{ in } e'$	letexp
		$e e'$	function application
		(e_1, e_2)	product
		$\text{if } e_1 \text{ then } e_2 \text{ else } e_3$	conditional
		$\text{fix } e$	fixpoint
		$\text{match } e_1, \dots, e_n \text{ with } Pmatrix$	pattern matching
		$e : sch$	type ascription
$pattern, pat$	$::=$		patterns
		$tname$	variable
		$_$	wildcard
		$const$	constant
		(pat_1, pat_2)	product
		$ctor pat_1 \dots pat_n$	constructor application
$Pmatrix$	$::=$		pattern matrix
		$ pb_1 \dots pb_m$	
$patbranch, pb$	$::=$		pattern branch
		$pat_1, \dots, pat_n \rightarrow expr$	
$tyctor$	$::=$		
		$upperident$	
		Int	
		Bool	
		String	
		Unit	
$tyvar, tv, a, b$	$::=$		
		$lowerident$	
$tvset, \bar{\alpha}$	$::=$		type variable set
		$\emptyset$	
		$\{tv\}$	
		$\bar{\alpha}_1 \cup \bar{\alpha}_2$	
		$\bar{\alpha}_1 \cap \bar{\alpha}_2$	
		$\bar{\alpha}_1 \setminus \bar{\alpha}_2$	
		$\text{ftv}(\Gamma)$	
		$\text{ftv}(ty)$	
		$\text{ftv}(\bar{\pi})$	
		$\text{ftv}(pred)$	

⁴⁰本文大多数文法，规则，都有其对应的 OTT 定义，Latex 源亦通过 OTT 生成。

$tyargs, tys$	$::=$	one or more type arguments
		ty
		$ty\ tys$
$typeexpr, ty, \tau$	$::=$	type expression
		$tyctor$ nullary type constructor
		tv type variable
		$ty \rightarrow ty'$ arrow type
		$ty \times ty'$ product type
		$tyctor\ tys$ nominal type application
		$\kappa\ tys$ higher-kinded TV head application
$predicate, pred, \pi$	$::=$	predicate
		$ctor\ ty_1 \dots ty_n$
$predicates, preds, \bar{\pi}$	$::=$	
		$\emptyset$
		$pred$
		$pred, \bar{\pi}$
		$\bar{\pi}_1, \bar{\pi}_2$
$scheme, sch, \sigma$	$::=$	scheme
		$\forall tv_1 \dots tv_n. \bar{\pi} \Rightarrow ty$
		$(\bar{\alpha}. \bar{\pi} \Rightarrow ty)$ from tvset
		ty Monotype
$context, \Gamma$	$::=$	typing environment
		$\emptyset$
		$\Gamma, x : sch$
		$x : sch$
$constraint, c$	$::=$	constraint
		$ty_1 \sim ty_2$ equality constraint
		$pred_{evidx}$ predicate constraint with evidence
$constraints, C$	$::=$	constraint set
		$\emptyset$
		c
		C_1, C_2 set join
$subst, \theta$	$::=$	substitution
		$\emptyset$
		$tv \mapsto ty$
		$\theta_1 \circ \theta_2$ composition
$evidence, ev$	$::=$	evidence term
		x evidence variable
		$ctor\ ev_1 \dots ev_n$ dictionary constructor
$evctx, \Delta$	$::=$	evidence context
		$\emptyset$
		$\Delta, evidx : pred$

3.2 判断规则与推断规则

现在，我们给出这一系统的判断规则 (judgemental rules) 与推断规则 (inference rules)。每一条判断规则都附带一段关于实现的解说。

合一化 用于计算相等约束的一个最泛因子 (most general unifier, MGU) 合一化引擎在类型检查阶段会

- 检查 TV 出现次数 (occurrence) 来防止无穷递归类型的产生，见 [section 2.4](#);
- 要求类型构造子应当完全匹配;
- 在两者元数相同的情况下将 KApp 与实际类型应用 TApp 合一化;
- 在结构性合一化类型模式前以 α -重命名之。

$$\boxed{\theta = \text{unify}(ty_1, ty_2)} \quad (\text{Unification of types})$$

$$\begin{array}{c} \text{U-REFL} \\ \hline \emptyset = \text{unify}(ty, ty) \end{array} \quad \begin{array}{c} \text{U-VARL} \\ \hline \frac{tv \notin \text{ftv}(ty)}{tv \mapsto ty = \text{unify}(tv, ty)} \end{array} \quad \begin{array}{c} \text{U-VARR} \\ \hline \frac{tv \notin \text{ftv}(ty)}{tv \mapsto ty = \text{unify}(ty, tv)} \end{array}$$

$$\begin{array}{c} \text{U-ARROW} \\ \hline \frac{\begin{array}{l} \theta_1 = \text{unify}(a, a') \\ \theta_2 = \text{unify}([\theta_1]b, [\theta_1]b') \end{array}}{\theta_2 \circ \theta_1 = \text{unify}(a \rightarrow b, a' \rightarrow b')} \end{array} \quad \begin{array}{c} \text{U-PROD} \\ \hline \frac{\begin{array}{l} \theta_1 = \text{unify}(a, a') \\ \theta_2 = \text{unify}([\theta_1]b, [\theta_1]b') \end{array}}{\theta_2 \circ \theta_1 = \text{unify}(a \times b, a' \times b')} \end{array}$$

约束求解 是按从左到右的顺序进行的，这一过程会不断的累积替换法则；谓词约束也在这一阶段被收集并作为残余约束用与实例解析。

$$\boxed{(\theta, \bar{\pi}) = \text{solve}(C)} \quad (\text{Constraint solving})$$

$$\begin{array}{c} \text{SLV-EMPTY} \\ \hline (\emptyset, \emptyset) = \text{solve}(\emptyset) \end{array} \quad \begin{array}{c} \text{SLV-EQ} \\ \hline \frac{\begin{array}{l} \theta = \text{unify}(ty_1, ty_2) \\ (\theta', \bar{\pi}) = \text{solve}([\theta]C) \end{array}}{(\theta' \circ \theta, \bar{\pi}) = \text{solve}(ty_1 \sim ty_2, C)} \end{array} \quad \begin{array}{c} \text{SLV-PRED} \\ \hline \frac{(\theta, \bar{\pi}) = \text{solve}(C)}{(\theta, [\theta]pred, \bar{\pi}) = \text{solve}(pred_{\text{evidx}}, C)} \end{array}$$

实例化 将绑定的 TV 替换为崭新 (fresh) 的 TV，同时返回必须满足的谓词约束。

$$\boxed{(ty, \bar{\pi}) = \text{inst}(sch)} \quad (\text{Scheme instantiation})$$

$$\begin{array}{c} \text{INST-FORALL} \\ \hline \frac{\begin{array}{l} tv'_1 = \text{fresh} \quad \dots \quad tv'_n = \text{fresh} \\ \theta = \{tv_1 \mapsto tv'_1, \dots, tv_n \mapsto tv'_n\} \end{array}}{([\theta]ty, [\theta]\bar{\pi}') = \text{inst}(\forall tv_1 \dots tv_n. \bar{\pi} \Rightarrow ty)} \end{array}$$

泛化 将所有给定环境内的非自由类型变量 (quantified TV. qTV) 提出, 出现这些变量的谓词则会成为类型模式的上下文, 也就是说: 谓词会被保留, 当且仅当该谓词包含出现在结果类型中的 qTV.

$$\boxed{sch = \text{gen}(\Gamma, ty, \bar{\pi})}$$

(Generalization)

$$\frac{\text{GEN-GEN} \quad \bar{\alpha} = (\text{ftv}(ty) \cup \text{ftv}(\bar{\pi})) \setminus \text{ftv}(\Gamma)}{(\bar{\alpha}.\bar{\pi} \Rightarrow ty) = \text{gen}(\Gamma, ty, \bar{\pi})}$$

表达式类型推断 可以生成用于约束求解的约束, 并通过求解这些约束 (应用这些替换法则), 以确定表达式的具体类型。类型判断 $\Gamma \vdash e \Rightarrow ty; C$ 读作:

在 (类型) 环境 Γ 下, 表达式 e 具备类型 ty , 同时生成约束 C .

根据核心代数, 类型推断适用于:

变量 在 Γ 中寻找并结果实例化类型模式,

常量 原本即有具体类型,

Lambda 抽象 其参数可获得崭新的 TV,

函数应用 生成能够让函数的参数类型与参数的类型相等的约束,

元组 生成能够让两个元组相同部分的类型相等的约束,

条件表达式 要求 then/else 分支类型相同, 且要求条件为 Bool,

Let 表达式 推断/求解/泛化右侧之后再推断程序体

Letrec 表达式 首先将与绑定 TV 数量相同的新 TV 推入 Γ 内, 再推断每一个右侧, 并产生与推断的类型相等的约束。

不动点组合子应用 (Fix) 即 $\text{fix } \lambda f. e$ 具有 e 的类型,⁴¹

类型标注 (Ascription) 检查推断类型至少与标注的类型具有相同的泛化程度, 但根据类型的 rank 使用不同的方法:

- 对 rank-1 类型标注而言, 我们直接合一化;
- 对 rank- n 且 $n \geq 2$ 即 TSch 含嵌套全称量化时, 我们在类型推断与检查的四大步骤中做特殊对待:

1. **实例化** 将模式 $\sigma = \forall \bar{\alpha}. \bar{\pi} \Rightarrow \tau$ 用崭新的 TV $\bar{\beta}$ 实例化, 得到 τ_{inst} .

2. **约束求解** 根据合一关系 $\tau_e \sim \tau_{\text{inst}}$ 求解目前所有的约束, 并得到一条替换法则 θ' .⁴²

⁴¹虽然前文并未提到, 但不动点组合子是 *Tigris* 早期加入但不再维护的一项功能。其初衷是用于教学, 但最后考虑实际效用便放弃维护。不动点组合子的语法是 `rec expr`.

⁴²可以把这一步操作看作是一个过早的求解 (premature solving)。我们立即将之求解只是因为需要在之后的刚性检查中使用它们。

3. **刚性检查** 对替换法则 θ' 中的每一个绑定 $\alpha \mapsto \tau'$, 如果 $\alpha \in \mathcal{R}$, 则 τ' 必须同 α 相等 (注意: 根据刚性变量的特征, 即指其一定不能被特殊化)。若这一检测不通过, 则拒绝该类型标注并抛出合一化错误。

4. **Skolemization** 在前文与后文多次强调, 用于保证标注的类型在泛性上与推断类型一致。

模式匹配 对于某模式矩阵中的

- 每一分支 i : 模式行向量 $p_{i,1}, \dots, p_{i,m}$ 会同被匹配对象 (discriminant) 的类型作比较; 而 Γ 则会由模式产生的绑定充盈, 此外, 模式产生的 TV 则会在其自身分支内被标记为刚性。通过这些机制, 我们能够保证 TV 不会逸出到其他分支。⁴³ 每一分支的右手侧则会在上述过程之后推断, 且要求 (约束) 所有分支具备相同类型。
- 每一被匹配对象 e_j : 是各自独立推断的, 产生类型 $\tau_1, \dots, \tau_m$, 且这些类型必须同模式类型相同。

模式完全性、冗余性实际上是在 System F 繁饰阶段进行而并非此处, 因为两者需要类型信息: 不完全的模式产生编译器警告但并非错误; 冗余模式分支则立即从矩阵中移除。

$$\boxed{\Gamma \vdash e \Rightarrow ty; C}$$

(基于约束求解的推断)

I-VAR

$$\frac{x : sch \in \Gamma \quad (ty, \bar{\pi}) = \text{inst}(sch)}{\Gamma \vdash x \Rightarrow ty; C}$$

I-INT

$$\frac{}{\Gamma \vdash \text{intlit} \Rightarrow \text{Int}; \emptyset}$$

I-STRING

$$\frac{}{\Gamma \vdash \text{strlit} \Rightarrow \text{String}; \emptyset}$$

I-TRUE

$$\frac{}{\Gamma \vdash \text{true} \Rightarrow \text{Bool}; \emptyset}$$

I-LAM

$$\frac{tv = \text{fresh} \quad \Gamma, x : tv \vdash e \Rightarrow ty; C}{\Gamma \vdash \lambda x. e \Rightarrow tv \rightarrow ty; C}$$

I-FALSE

$$\frac{}{\Gamma \vdash \text{false} \Rightarrow \text{Bool}; \emptyset}$$

I-UNIT

$$\frac{}{\Gamma \vdash () \Rightarrow \text{Unit}; \emptyset}$$

I-APP

$$\frac{\Gamma \vdash e_1 \Rightarrow ty_1; C_1 \quad \Gamma \vdash e_2 \Rightarrow ty_2; C_2 \quad tv = \text{fresh}}{\Gamma \vdash e_1 e_2 \Rightarrow a; (C_1, C_2), ty_1 \sim ty_2 \rightarrow a}$$

I-PROD

$$\frac{\Gamma \vdash e_1 \Rightarrow ty_1; C_1 \quad \Gamma \vdash e_2 \Rightarrow ty_2; C_2}{\Gamma \vdash (e_1, e_2) \Rightarrow ty_1 \times ty_2; C_1, C_2}$$

I-COND

$$\frac{\Gamma \vdash e_1 \Rightarrow ty_1; C_1 \quad \Gamma \vdash e_2 \Rightarrow ty_2; C_2 \quad \Gamma \vdash e_3 \Rightarrow ty_3; C_3}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow ty_2; (((C_1, C_2), C_3), ty_1 \sim \text{Bool}), ty_2 \sim ty_3)}$$

I-LET

$$\frac{\Gamma \vdash e_1 \Rightarrow ty_1; C_1 \quad (\theta, \bar{\pi}) = \text{solve}(C_1) \quad sch = \text{gen}(\Gamma, [\theta]ty_1, \bar{\pi}) \quad \Gamma, x : sch \vdash e_2 \Rightarrow ty_2; C_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 \Rightarrow ty_2; [\theta]C_2}$$

I-FIX

$$\frac{tv = \text{fresh} \quad \Gamma, f : tv \vdash e \Rightarrow ty; C}{\Gamma \vdash \text{fix } (\lambda f. e) \Rightarrow ty; C, tv \sim ty}$$

I-LETREC

$$\frac{tv = \text{fresh} \quad \Gamma, x : (tv) \vdash e_1 \Rightarrow ty_1; C_1 \quad (\theta, \bar{\pi}) = \text{solve}(C_1) \quad sch = \text{gen}(\Gamma, [\theta]ty_1, \bar{\pi}) \quad \Gamma, x : sch \vdash e \Rightarrow ty; C}{\Gamma \vdash \text{let } x = e_1 \text{ in } e \Rightarrow ty_1; [\theta]C}$$

⁴³这一步操作可以看作是一个预防措施: 此处所谓“逸出”只会发生在模式匹配枚举族类型的值时发生, 而目前 *Tigris* 并不支持。

由于模式匹配语句与类型标注在 OTT 中不易正确表达，我们手动将其表示为⁴⁴

$$\begin{array}{c}
\Gamma \vdash e_j \Rightarrow \tau_j; C_j \quad (\forall j \in 1..m) \\
\alpha = \text{fresh} \\
\text{For each branch } i \in 1..k : \quad |(\bar{p}_i)| = m \quad \Gamma \vdash p_{i,j} \Leftarrow \tau_j \Rightarrow \Gamma_{i,j}; C_{p_{i,j}} \quad (\forall j \in 1..m) \\
\Gamma_i = \Gamma_{i,m} \quad (\text{final extended environment for branch } i) \\
\bar{\beta}_i = \text{ftv}(\text{bound variables in } \bar{p}_i) \quad \text{pushRigid}(\bar{\beta}_i) \quad \Gamma_i \vdash r_i \Rightarrow \tau_{r_i}; C_{r_i} \quad \text{popRigid}() \\
C_{\text{unify}} = \{\tau_{r_i} \sim \alpha \mid i \in 1..k\} \quad C = \bigcup_{j=1}^m C_j \cup \bigcup_{i=1}^k \left(\bigcup_{j=1}^m C_{p_{i,j}} \cup C_{r_i} \right) \cup C_{\text{unify}} \\
\hline
\Gamma \vdash \text{match } e_1, \dots, e_m \text{ with } \overline{p_1, 1, \dots, p_1, m \rightarrow r_1 \Rightarrow \alpha; C} \quad \text{I-MATCH}
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash e \Rightarrow \tau_e; C_e \\
\hline
\Gamma \vdash (e : \tau) \Rightarrow \tau; C_e, \tau_e \sim \tau \quad \text{I-ASCRIBE-MONO}
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash e \Rightarrow \tau_e; C_e \\
\sigma = \forall \bar{\alpha}. \bar{\pi} \Rightarrow \tau \quad \bar{\beta} = \text{fresh}^{|\bar{\alpha}|} \quad \theta = [\bar{\alpha} \mapsto \bar{\beta}] \quad \tau_{\text{inst}} = \theta(\tau) \\
C_0 = \text{current constraints} \quad (\theta', _) = \text{solve}(C_0, \tau_e \sim \tau_{\text{inst}}) \\
\mathcal{R} = \text{current rigid set} \quad \forall (\alpha \mapsto \tau') \in \theta'. \alpha \in \mathcal{R} \implies \tau' = \alpha \\
\tau_{\text{sk}} = \text{skolem}(\sigma) \quad \text{add constraints for } \bar{\pi} \\
\hline
\Gamma \vdash (e : \sigma) \Rightarrow \tau_{\text{inst}}; C_e, \tau_e \sim \tau_{\text{inst}} \quad \text{I-ASCRIBE-POLY}
\end{array}$$

其中 rules **I-ASCRIBE-POLY** and **I-ASCRIBE-MONO** 正对应前文类型标注推断中表述的（非）泛情况。

模式 可以绑定变量，亦可以从被匹配对象中产生新约束。判断规则 $\Gamma \vdash \text{pat} \Rightarrow \Gamma'; C; ty$ 读作：

模式 $\text{pat} : ty$ 将 Γ 充盈（extend）为 Γ' ，产生约束 C 。

$\Gamma \vdash \text{pat} \Rightarrow \Gamma'; C; ty$ (模式推断)	P-WILD $\frac{}{\Gamma \vdash _ \Rightarrow \Gamma; \emptyset; ty}$ P-CONSTSTR $\frac{}{\Gamma \vdash \text{strlit} \Rightarrow \Gamma; ty \sim \text{String}; ty}$	P-VAR $\frac{x \notin \text{dom}(\Gamma)}{\Gamma \vdash x \Rightarrow \Gamma, x : ty; \emptyset; ty}$ P-CONSTTRUE $\frac{}{\Gamma \vdash \text{true} \Rightarrow \Gamma; ty \sim \text{Bool}; ty}$ P-CONSTFALSE $\frac{}{\Gamma \vdash \text{false} \Rightarrow \Gamma; ty \sim \text{Bool}; ty}$
P-CONSTUNIT $\frac{}{\Gamma \vdash () \Rightarrow \Gamma; ty \sim \text{Unit}; ty}$	P-PROD $\frac{a = \text{fresh} \quad b = \text{fresh} \quad \Gamma \vdash \text{pat}_1 \Rightarrow \Gamma_1; C_1; a \quad \Gamma_1 \vdash \text{pat}_2 \Rightarrow \Gamma_2; C_2; b}{\Gamma \vdash (\text{pat}_1, \text{pat}_2) \Rightarrow \Gamma_2; (C_1, C_2), ty \sim a \times b; ty}$	

因为构造子应用在 OTT 中不易表述，且可能存在可读性问题，仿照前文的形式，我们手动表述：

$$\begin{array}{c}
\text{ctor} : \sigma \in \Gamma \quad (\tau_{\text{ctor}}, \bar{\pi}) = \text{inst}(\sigma) \quad \text{peel}(\tau_{\text{ctor}}) = (\tau_1, \dots, \tau_n, \tau_{\text{res}}) \quad n = |\bar{p}| \\
\Gamma_0 = \Gamma \quad \Gamma_{i-1} \vdash p_i \Leftarrow \tau_i \Rightarrow \Gamma_i; C_i \quad (\forall i \in 1..n) \quad C_{\pi} = \bar{\pi} \\
\hline
\Gamma \vdash \text{ctor } p_1 \cdots p_n \Leftarrow \tau \Rightarrow \Gamma_n; C_1, \dots, C_n, C_{\pi}, \tau \sim \tau_{\text{res}} \quad \text{P-CTOR}
\end{array}$$

⁴⁴读者需要注意：这些规则是手动打出的，一些符号并未在上述形式化文法中定义为生成规则，或者 OTT 源中的公式（formula）。在这种情况下，我们使用恰当的自然语言直接在行内对之作出描述、定义。笔者只能怪罪其自身，因为他运用 OTT 并不很熟练。

其中 `peel` 用于解构由箭头类型构成的树状数据结构（类型 AST），将之划分为参数类型与结果类型：

$$\text{peel}(\tau_1 \rightarrow \tau_2 \rightarrow \cdots \rightarrow \tau_n \rightarrow \tau_r) = (\tau_1, \tau_2, \dots, \tau_n, \tau_r)$$

构造子的类型模式 σ 是从 Γ 中查找的，同时用崭新 TV 实例化之。实例化产生的谓词约束亦会被添加到类型环境中，保证类型类要求的正确传播。

下面，我们列出上文没有充分解说的一些概念。

Remark (Skolemization). 将 TV 替换为 skolems 的过程，skolems 不能和除自身以外的任何类型合一化。

$$\text{skolem}(\forall \bar{\alpha}. \bar{\pi} \Rightarrow \tau) = [\bar{\alpha} \mapsto \overline{\text{sk}_{\bar{\alpha}}}] (\tau)$$

其中每一 sk_{α} 都是崭新的 skolem 常量（通过 0-元类型构造子实现，在实现中有形式 $\text{?sk}.\alpha$ ）。

“刚性”一词很好地表达了 skolem 的关键性质： $\text{sk}_{\alpha} \sim \tau$ 正确，当且仅当 $\tau = \text{sk}_{\alpha}$ 。

Remark (刚性变量集合/栈 (rigid set/stack)). *Tigris* 的类型系统维护一个栈，其元素为一个个刚性变量集合（注意：这些集合是 $\mathcal{R}$ 的组成部分）。我们通过栈作为数据结构的特性来很自然的实现对嵌套作用域中的刚性变量的记录。这套系统有如下两个重要方法：

$$\begin{aligned} \text{pushRigid}(\bar{\alpha}) : \quad \mathcal{R} &:= \mathcal{R} \cup \bar{\alpha}; \quad \text{stack} := \bar{\alpha} :: \text{stack} \\ \text{popRigid}() : \quad \text{let } (S :: \text{rest}) &= \text{stack in } \mathcal{R} := \mathcal{R} \setminus S; \quad \text{stack} := \text{rest} \end{aligned}$$

不变量 (Invariant.) 任一时刻，刚性变量集合 $\mathcal{R}$ 都具有以下性质：

$$\mathcal{R} \equiv \bigcup \text{stack}$$

即 $\mathcal{R}$ 是栈上的集合的并。在约束求解时，任一替换规则 $\alpha \mapsto \tau$ 满足 $\alpha \in \mathcal{R}$ 和 $\tau \neq \alpha$ 都将被拒绝。注意赋值操作 ($:=$) 是由 ST 单子附带的，通过 `MonadStateOf` 接口实现。

Remark (求解刚性变量). 约束变量求解器（即合一化求解过程）在遇到刚性变量时按以下规则工作：

$$\frac{\alpha \in \mathcal{R} \quad \tau \neq \alpha}{\text{solve}(\alpha \sim \tau) = \text{error}} \text{ SOLVE-RIGID} \qquad \frac{\alpha \in \mathcal{R}}{\text{solve}(\alpha \sim \alpha) = \emptyset} \text{ SOLVE-RIGID-OK}$$

尽管由于原生不动点组合子的存在，本系统已经存在矛盾，但只要不再使用这一已经不再被维护的功能，系统仍然是合理的；此外，对 HM 系统添加的扩展也日渐影响着我们的类型系统的合理性。本系统并不承担严谨的数学证明功能，因而“破坏合理性”从编程角度考虑，有程度之分。例如一个存在矛盾的系统，如果不存在爆炸原理（ex falso quodlibet, EFQ）⁴⁵，则不一定能造成工程上的问题（例如 Java 与 Scala 的类型系统都已证明存在矛盾，但并不作为能攻击它们安全性不好的理由）。

下面，我们论述本系统是通过何种机制保证合理性的。

Remark (合理性 Soundness). *Tigris* 通过以下若干机制保证其类型系统的合理性：

Skolems 类型检查器在遇到模式匹配和类型标注时，一些 TV 会被标注刚性并由 $\mathcal{R}$ （在实现中是 `rTV`）记录。检查器通过拒绝将这些类型实例化到不相容（incompatible）类型的替换规则以避免不合理的泛化。

⁴⁵即从矛盾能推任意命题。这一原理是类型论实现假命题（和空类型，不严谨的说，两者同构）的最简编码，其消去子：

$$\frac{\Gamma \vdash p : \text{False} \quad \Gamma, x : \text{False} \vdash C(x) : \text{Sort } u}{\text{False.rec}(p; C) : C(x)} \text{ FALSE-ELIM} \quad (\text{无计算法则})$$

空类型的消去子是空完备（vacuously total）的：不需要分情况讨论，因为此处不存在任何情况，因而，它不具备计算意义，也不能构造任何值（数据）。

Rank-N skolemization 当检查 $e : \forall tv.\pi \Rightarrow ty$ 时, 我们用崭新的 skolem 常量替换 tv , 以此来确保程序确实能对所有类型都工作——这一想法利用了全称量化的性质: 如果表达式对任意类型都成立, 则将之用任意独立不可合一的 skolem 实例化时也应成立, 因为该 skolem 是随机挑选的 (崭新的), 所以这是表达式成立的充要条件。换句话说, 如果这一检查失败, 则说明表达式泛性不够强 (not as generalized), 与标注的类型不符合。

存在次数检查 是用来禁止用户构造等式递归 (equirecursive) 类型, 比如 $\alpha \sim \alpha \rightarrow \beta$ 和 $\alpha \sim \alpha \times \beta$, 对应表达式 `fun x  $\Rightarrow$  x x` 或 `fun | (x, y)  $\Rightarrow$  x | z  $\Rightarrow$  z`。

洁净 (Hygienic) 泛化 只有 Γ 中绑定的 TV 才会被泛化。谓词则根据前文描述的条件进行保留, 以避免空约束。

语法限制 细心的读者如果详细地阅读了第一部分的类型表达式的 BNF 文法就会发现, 目前的文法并不允许诸如 $(\forall a.a \rightarrow a) \rightarrow \text{Int} \times \text{Bool}$ 这样的 rank-2 type, 即便类型表达式的 AST 定义中允许内嵌模式。的确, 唯一能够引入内嵌模式的方法就是通过数据类型定义的多态栏位, 而这一方式并不有害 (在 Haskell98 中得到验证)。我们认为, 这一条机制/限制是最强的, 能够有效保证类型系统的合理性。

Remark (反例: 为什么我们需要刚性变量). 到目前为止, “刚性”一词出现过许多次, 我们亦提到过为什么需要这样的机制, 但仅局限于抽象的理论表述。下面提供若干反例, 用具体例子展示刚性变量的必要性。

- (不确实例化 (Specialization)) 考虑

```
let id = (fun x  $\Rightarrow$  x :  $\forall a, a \rightarrow a$ ) 1
let id' :  $\forall a, a \rightarrow a$  = fun x  $\Rightarrow$  (x : Int) -- [ERROR] Can't unify type Int with ?sk.a.
let id'' x :  $\forall a, a \rightarrow a$  = x + 1
```

例子一是合法的: 类型标注要求 `fun x  $\Rightarrow$  x` 是多态函数, 但根据整个定义, 该函数实际上被单态化 (monomorphized) 成 `Int`, 相当于用户手动指导编译器将之实例化;

而例子二则不是: 如果在函数体内 `fun x  $\Rightarrow$  ...` 内将 a (此时其为刚性变量) 实例化为 `Int` (通过类型标注 $(x : \text{Int})$) 则不合理。这是因为, `id'` 由用户声明为多态函数, 但其右侧的定义则是单态函数, 泛性不匹配; 例子三同理。

- (泛性恢复 (Polymorphism recovery)) 一般来说, 类型类的方法 (其本身就很有可能是多态函数) 在程序中是直接调用的, 因而, 我们参照 Haskell98 的特殊对待实现, 在所有的方法调用处重新将之泛化。但考虑边界情况, 如果用户希望直接使用 (但不推荐) 某个字典 (实例) 内的方法实现 (`l` 是整数列表; `l'` 是字符串列表; `const` 是常函数, 列表作为自函子的实现方法, 即此处的 `f`, 是列表映射 `List.map`):

```
let prog (Functor f) =
  let x = f (_ + 1) l and y = f (const ()) l'
  in (x, y)
```

这同样也是被 Tigris 所支持的, 但允许这种写法需要归功于刚性变量机制: a 与 b 在每次方法调用时都为刚性。这允许我们独立地, 在不同位置实例化之, 比如上述例子中 f 在 x 处有实例化 ($a \mapsto \text{Int}, b \mapsto \text{Int}$); 在 y 有实例化 ($a \mapsto \text{String}, b \mapsto \text{Unit}$): 两个用例互不干扰。

3.3 实例解析

实例解析不仅仅是将实例与对应类型类配对。准确的说，编译器需要一套标准流程（算法）来为某个类的类型寻找其对应实例。⁴⁶ 这一标准流程称为实例解析。下文叙述的算法从实例环境中搜索匹配的实例，并在链上递归地解析谓词约束（例如前文出现过依赖其他实例的实例，这就产生了链）。

实例元数据 每个记录的实例都是一个 4-元组结构：

$$I = \langle \text{iname}, \text{cls}, \bar{\alpha}, \bar{\pi}_{\text{ctx}} \rangle$$

其中

- iname 是该实例的独特标识符（命名例：i_Eq_0）；
- cls 是类型类的头部；
- $\bar{\alpha}$ 是实例声明中的类型参数；
- $\bar{\pi}_{\text{ctx}}$ 是实例的上下文（即依赖的其他实例）。

例如，实例声明 **instance** : forall α [Eq α], Eq (List α) = ... 产生元数据

$$I = \langle \text{i_Eq_1}, \text{Eq}, [\text{List } \alpha], [\text{Eq } \alpha] \rangle$$

实例头部合一化 (Head Unification) 指的是检查某一目标 (goal) $C \bar{\tau}$ 是否可以被一个实例头部匹配 $C \bar{\alpha}$ 。这一工作在进一步的实例解析前开展。

Algorithm 1 Head Unification

```

1: function UNIFYHEAD( $\bar{\tau}_{\text{goal}}, \bar{\alpha}_{\text{inst}}$ )
2:   if  $|\bar{\tau}_{\text{goal}}| \neq |\bar{\alpha}_{\text{inst}}|$  then
3:     return error
4:   end if
5:    $\theta \leftarrow \emptyset$ 
6:   for  $i \leftarrow 1$  to  $|\bar{\tau}_{\text{goal}}|$  do
7:      $\theta' \leftarrow \text{unify}(\text{apply}(\theta, \tau_i), \text{apply}(\theta, \alpha_i))$ 
8:      $\theta \leftarrow \theta' \circ \theta$ 
9:   end for
10:  return  $\theta$ 
11: end function

```

随着类型类定义、对应实例定义的增多，需要在环境中配对实例所需的时间就越久。因而，上面的合一化检查是一种剪枝：只要头部不匹配，则两者一定不能配对，则以这两者配对为目标的实例解析提前终止，编译器则继续往下寻找其他可以配对的实例。显然，在找到正确实例之前的所有实例解析都不会返回结果，因而我们认为在这里做剪枝操作能够产生较大的优化成效。

实例解析算法 主实例解析算法维护一个访问集 (visited set, $\mathcal{V}$) 来检测链上的自环，表现为解析结果歧义或环导致的解析不停机现象。

⁴⁶PL 正式表述：找到一个见证 (witness, 即实现) 谓词约束 $\pi = C \bar{\tau}$ 的证据项 (evidence term, 即字典)。

Algorithm 2 Instance Resolution

```

1: function RESOLVE( $\Gamma, \pi, \mathcal{V}$ )
2:   Input: Environment  $\Gamma$ , predicate  $\pi = C \bar{\tau}$ , visited set  $\mathcal{V}$ 
3:   Output: Evidence expression  $e$  s.t.  $e : C \bar{\tau}$ , or error
4:
5:   if  $\pi \in \mathcal{V}$  then
6:     return error: “cyclic instance resolution”
7:   end if
8:    $\mathcal{V} \leftarrow \mathcal{V} \cup \{\pi\}$ 
9:
10:   $insts \leftarrow \Gamma.instInfo[C]$  ▷ 类  $C$  的所有实例
11:  if  $insts = \emptyset$  then
12:    return error: “no instance for  $\pi$ ”
13:  end if
14:
15:   $found \leftarrow \text{none}$ 
16:  for each  $I = \langle iname, C, \bar{\alpha}, \bar{\pi}_{ctx} \rangle$  in  $insts$  do
17:    try:
18:       $\theta \leftarrow \text{UNIFYHEAD}(\bar{\tau}, \bar{\alpha})$ 
19:       $\bar{\pi}'_{ctx} \leftarrow \text{apply}(\theta, \bar{\pi}_{ctx})$ 
20:      for each  $\pi' \in \bar{\pi}'_{ctx}$  do
21:         $\_ \leftarrow \text{RESOLVE}(\Gamma, \pi', \mathcal{V})$  ▷ 保证可解
22:      end for
23:       $e \leftarrow (iname : C \bar{\tau})$  ▷ 引用实例的类型标注
24:      if  $found \neq \text{none}$  then
25:        return error: “ambiguous instance for  $\pi$ ”
26:      end if
27:       $found \leftarrow \text{some}(e)$ 
28:    catch: continue ▷ 捕捉错误，即解析失败，尝试下个实例
29:  end for
30:
31:  if  $found = \text{none}$  then
32:    return error: “no matching instance for  $\pi$ ”
33:  end if
34:  return  $found$ 
35: end function

```

Remark (ResolveM). 上述算法运行在 ResolveM 单子中。正如之前我们提到过的类型检查器的 InferC 单子一样，这里的 ResolveM 定义不过是把状态变成了 ResolveState，即上述 $\mathcal{V}$ ，但在实现上我们使用哈希表实现的集合 HashSet 作为容器储存谓词。

```

1  abbrev ResolveState := Std.HashSet Pred
2  instance : MonadLift (Except TypingError) (EST TypingError  $\sigma$ ) where
3    monadLift
4    | .error e  $\Rightarrow$  throw e
5    | .ok e  $\Rightarrow$  return e

```

同时，我们为这一单子实现 MonadLift 接口，以定义从错误处理单子 Except 到 EST 的自然变换⁴⁷

⁴⁷自然变换是范畴论的概念。函数式编程的定义与其在数学中类似：将一个在单子 m 中运行的操作提升进 n 中，有方法 $\text{liftM} : m \alpha \rightarrow n \alpha$ 。在 Lean 中，这一变换不需用户显式写出，是自动插入的；但实例本身需要用户自行定义。

(natural transformation), 将任何可能报错的纯函数 (操作) 提升 (lift) 到后者中以运行。

Remark (实例解析的性质). 下面, 我们给出实例解析算法的一些性质。

停机性 还没有被证明, 从源代码定义中的 `partial` 关键字亦可看出。直观的讲, 实例解析停机当且仅当某一实例链是良基的, 即不存在环。集合 $\mathcal{V}$ (第5行) 虽然可以检测直接的环 (自环), 但并不一定能够保证停机, 尤其是针对一些边界情况。

语义一致性 是通过单一匹配原则保证的: 如果有多个实例匹配一个目标 (第24行), 解析算法将抛出歧义错误。这告诉我们一个重要性质: 程序的意义不会受到随机选择的实例影响, 因为解析过程不随机选择实例。然而与常规实现不同, 这一原则适用所有实例, 与实例本身的泛性无关: 也就是说, 若存在多个匹配的实例, 且存在其中之一最“具体”的实例也不会导致其被优先选择, 而是出现歧义错误, 尽管优先选择它在工程意义上看起来更合理。若允许多个匹配实例的存在, 其中一个做法是加入优先级和默认实例, 但目前我们还不支持。

证据生成 (Evidence Generation) 指以下类型标注的实例标识符:

$$e = (\text{iname} : C \bar{\tau})$$

其之后在 System F IR 翻译阶段将被繁饰到字典传递风格的实现, 并用字典投影结点 `Proj src mname i ty` 编码。

上下文传播 指对具有上下文谓词 (依赖其他实例的实例) 的实例 π_{ctx} 使用合一化产生的替换法则实例化所有的上下文谓词 (第19行), 并递归地解析它们 (第20行)。例如 `instance :  $\forall \alpha [\text{Eq } \alpha], \text{Eq } (\text{List } \alpha) = \dots$` 中的 `Eq a` 就是一个上下文谓词 (显然, 很多时候上下文谓词不止一个)。

为了加深对算法的理解, 我们用一个简单的例子模拟实例解析算法。考虑解析 `Eq (List Int)`, 给定实例环境如下:

$$I_0 = \langle i_{\text{Eq}_0}, \text{Eq}, [\text{Int}], \emptyset \rangle, \quad I_1 = \langle i_{\text{Eq}_1}, \text{Eq}, [\text{List } \alpha], [\text{Eq } \alpha] \rangle.$$

1. 目标: `Eq (List Int)`
2. 尝试 I_0 : `unify(List Int, Int)`, 失败
3. 尝试 I_1 : `unify(List Int, List  $\alpha$ )`, 由 $\theta = [\alpha \mapsto \text{Int}]$ 成功
4. 上下文: `apply( $\theta$ , [Eq  $\alpha$ ]) = [Eq Int]`
5. 递归地解析 `Eq Int`:
 - (a) 尝试 I_0 : `unify(Int, Int)`, 由 $\theta = \emptyset$ 成功
 - (b) `Return (i_Eq_0 : Eq Int)`
 - (c) 上下文: $\emptyset$, 为空则不再需要递归
6. 返回 `(i_Eq_1 : Eq (List Int))`

Remark (与 GHC 的比较). 类型类由 P. Wadler 提出, 并最先应用于 Haskell 语言。作为其最流行的实现, GHC 实现了一个健壮性 (robust) 更强, 更加复杂, 且具备语言标准中未提及的许多扩展的类型类机制。虽然 *Tigris* 类型类的实现从 GHC 学习、汲取了诸多灵感, 但必须承认相比于后者, 前者的实例解析策略要简单得多:

- *Tigris* 不允许重叠实例, 无论这些实例在源代码中出现的先后顺序; GHC 则提供一套实例优先级机制, 允许用户针对实例解析做高细粒度的控制。

- 此外, *Tigris* 不允许针对实例设置后备 (fallback) 实例。

但是, 若和 Haskell '98 或 2010 做比较, *Tigris* 的实例声明在一些地方反而更具表达性: 譬如我们部分实现了 GHC 语言扩展 FlexibleContext⁴⁸, 因而可以定义诸如 `instance ∀a [Eq a], Eq (a * a) = ..` 的实例。此外, 多类型参数的类型类也是支持的, 比如前文曾出现过的 HAdd 和 HEq 的例子。

3.4 System F 繁饰

带类型的语法树 (typed syntax) TExpr, 在经历实例解析的同时, 也正在被繁饰到 FExpr. 下面就这一过程进行直观的探讨。在这一部分我们侧重谈实现, 更多的使用“字典”一词表示某接口 (类) 的实现。首先给出繁饰的具体行为:

给定一模式

$$\forall \alpha_1 \cdots \alpha_n [P_1, \dots, P_k], \tau$$

我们将一个绑定的右手侧翻译为

$$\Lambda \alpha_1, \dots, \alpha_n. \lambda d_1, \dots, d_k. \text{body}$$

其中 $d_1, \dots, d_k$ 是谓词 $P_1, \dots, P_k$ 对应的实例。若右手侧已经由正好 k 个字典 lambda (即绑定字典的 lambda 抽象) 起头, 且这些字典 lambda 是编译器自动为类型类方法生成的封装, 则这种情况下只在头部添加 TyLams. 每当繁饰过程遇到一个 Var 结点时,

$$\text{Var } x : \sigma \text{ where } \sigma = \forall \bar{\alpha} \bar{P}, \text{tbody}$$

会繁饰为

$$\begin{aligned} & \text{TyApp } \cdots (\text{TyApp } (\text{Var } x \text{ ty}_{\text{mono}}) \alpha_1) \cdots \alpha_n \\ & \text{App}(\cdots) d_1 \\ & \vdots \\ & \text{App}(\cdots) d_k \end{aligned}$$

字典参数⁴⁹是以下两种情况之一

- 存在于作用域内, 也就是说, 与某一由上述字典 lambda 产生的谓词模板相匹配; 或,
- 由实例解析过程合成 (synthesized), 产生一个不带类型的表达式 (Expr)。我们在该表达式上重新做类型推断与字典繁饰, 以转变为 FExpr. 这一过程显然是递归的。

若某一方法具有多态类型, 例如 Functor 的 map, 或者从实现上说, TSch (Forall as [] τ), 则其将被实化 (reified) 到 System F 的类型抽象 (type abstraction, 即 type lambda), 这一转变是通过将其所有的 $\forall$ binder 一一包裹在 TyLam 结点中而达成的。对于以后计划实现 Higer-Ranked 类型时同理。但需要注意, 各方法独自的谓词上下文 (例如, map 方法依赖某一接口) 目前还不支持, 且若出现则会抛出错误。

⁴⁸这一扩展解除了 Haskell 98 的限制, 不再要求谓词约束必须具有 class type-variable 或 class (type-variable type1 type2 .. typen) 的形式。

⁴⁹标识符为 $d_P.\text{cls_}i$, 其中 P 是谓词, i 是该类型类的实例计数器; 若这一字典参数是持久化的, 则在其之前加字符 r .

字典持久化 指的是将合成的字典持久化的优化。实践证明，绝大多数情况下同一字典都在代码中使用多次，如果每次使用都需要重新合成相同的字典，或生成字典间的应用（如果实例具备上下文），则编译效率就比较低下。持久化通过在函数层级记录每个字典的第一次使用，在遇到同一字典多次使用时，重复使用第一次合成的字典，来提高性能。而语言自带的 Let-绑定正为这一优化铺平了道路：我们在函数头部插入一个 Let-绑定，将所有行内使用到的字典提出（hoist）并用 Let 绑定之。从语义上讲，字典持久化类似公共子表达式消除（common subexpression elimination），只不过字典是很特定的语言对象，因而使得优化实现起来简单。我们用一个稍复杂的例子（简单例子不需优化）Functor、Applicative、Monad 作为代数结构的继承关系来展示这个优化：

```
class Applicative (f : 1) = {pure : ∀ a, a -> f a, seq : ∀ a b, f (a -> b) -> f a -> f b}
class Monad (m : 1) = {bind : ∀ a b, m a -> (a -> m b) -> m b, return : ∀ a, a -> m a}
infixl 55 ">>=" ⇒ bind ;; infixr 60 "<*>" ⇒ seq
instance ∀(m : 1) [Monad m], Applicative m =
  {pure = return, seq f x = f >>= fun f ⇒ x >>= fun x ⇒ return $ f x}
instance ∀(f : 1) [Applicative f], Functor f = {map f x = pure f <*> x}
instance Monad Option = {return = Some, bind | None, _ ⇒ None | Some x, f ⇒ f x}
let seqOp = (Some id <*> _)
```

已知 i_Monad_0, i_Applicative_0 分别是 Option 作为 Monad、Monad 作为 Applicative 的实例，则有 IR 结果

```
let seqOp : ∀ α, (Unit → Option α) → Option α =
  let rd_Applicative_1 : Applicative Option =
    let rd_Monad_0 : Monad Option = i_Monad_0
    in i_Applicative_0@Option rd_Monad_0
  in (λ α. rd_Applicative_1[1, seq]@Option (Some@(α → α) id@α))
```

如上所示，持久化优化被触发，两个字典，以及他们的应用，都通过 Let 缓存（cached）于函数头部，且绑定的标识符为 rd_Applicative_1。注意到其计数器为 1，这说明持久化优化在机制上会合成新的字典（但效率远高于正常合成步骤）。所以我们认为，在繁饰过程中加入这个优化，实现简单且十分实用，能够切实提高编译性能。

繁饰过程运行在 F 单子中。这一单子与前面提到的单子相似，只不过包含繁饰需要的特定状态类型。但不同于以往，F 基于 EStateM，一个自带错误处理与（用纯函数模拟的）的可变状态的单子，而非之前真正具备（线程独立的）可变状态的 ST。我们这样设计，是因为繁饰过程涉及到许多本地状态，搭配递归调用，我们不必次次修改可变状态，而是在调用处传入这一调用所需的本地状态即可，这在底层是通过栈分配的函数参数实现的。

如同之前的抽象语法树，System F IR 也使用类 Wadler 的 pretty printing 算法，方便 debug 和教学使用。

我们以笔者对这一模块设计上的考量与反思总结本部分。

Remark (System F IR 的实用性与设计上的反思). 自本课题开题伊始，笔者本人对 *Tigris* 的目标便十分明确：编译到 Common Lisp. LISP 是动态类型语言，而最流行的 SBCL 实现则自带一些类型推断（但大概率不是 gradual typing）、传播的扩展功能，因而我们做泛型特殊化、或将类型传播到下一个降级阶段（即下文的 Lambda IR 降级）的理由不是很充分。⁵⁰

因为这些考量，我们其实并不需要上文的 System F IR，因为没有它类型类繁饰也能实现，但最终仍然保留了这一 IR。笔者需要承认，这里既有一些设计上的前瞻性，也有设计失误、考虑不够充分的问题：

- 前者表现在若我们之后需要实现 arbitrary rank polymorphism 和 inductive families 时很自然：可以由当前的 IR 编码。

⁵⁰虽然之后的编程实践证明，带着类型走完全部编译流程是极方便的，且更加可靠。

- 后者表现在下文的 Lambda IR 中。在设计 Lambda IR 时我们没有考虑将 System F IR 的类型信息携带到下一层级，因而 Lambda IR 不带类型信息，直接导致后端代码生成时的附加困难，更使得后端实现的健壮性成疑。目前的后端会通过收集变量的形状来产生代码，但这并不可靠，且存在一些笔者临时修补的错误。不仅如此，若后期我们需要添加静态类型语言后端，比如 C，这一 IR 就几乎不可用。

虽然此处的问题和 System F IR 联系不深，但优点抑或缺陷，都向我们揭示了一个宝贵的教训：

在任意编译阶段，都应当具备携带类型的 IR.

3.5 本章小结

(略)

4 The *Lambda* IR, CPS, Codegen & FFI

System F IR 的繁饰标志着编译“上升期”的结束；Lambda IR⁵¹则作为一个真正的中间表示 (intermediate representation, IR)，标志着编译“下降期”（降级）的开始。*Lambda* IR 应当算做 ANF，后者是函数式语言编译器常用的一类 IR。

ANF 的精髓在于变量。在 ANF 中，某一语言对象，或（非基础的，nontrivial）表达式，都通过 Let-绑定捕捉为变量，并由后者指代。这一特性在某种程度上模仿相同程序的汇编代码。虽然 *Tigris* 编译到一个高级语言，但其产生的后端代码风格与一般程序员不同，因而我们认为 SBCL 可以从中获得一定优势，相当于 *Tigris* 编译器已经帮其做了部分工作。

4.1 *Lambda* IR

***Lambda* IR 的定义** *Lambda* IR 的设计从六大角度开展：值、右手侧、陈述语句、尾调用位置、表达式、函数。对应到实际实现，就是 Value、Rhs、Stmt、Tail、LExp、LFun，我们将逐一介绍各个部分。读者应当注意上述六个类型是互递归的。

Value 也叫立即数/值，不能再化简。立即数包括变量、常量（整数、字符串、布尔值、幺元）、构造值和 Lambda 抽象。应当注意，此时的 Lambda 抽象还并没有经过闭包转换，因此可以捕捉自由变量。

Rhs 指 ANF 的右手侧，且只能以变量作为参数。右手侧包含原生函数（整数四则运算、常量的等号）、字典投影、元组、构造子应用（在定义上与立即值的构造值一致，但意义不同）、构造子测试（即测试两构造子是否相同）与函数调用。

Stmt 指陈述语句，目前只有 Let1。

Tail 表示尾调用位置，即某一函数的最后一条语句（一个操作）但不仅包括函数调用，还有函数返回、条件表达式、多分支条件匹配表达式（switch）和一特殊 token MATCHFAIL。Tail 具有标注尾调用位置的重要功能，方便后期做尾调用优化。在具体实现中，switch 语句又依匹配对象不同分为常量、构造值两类——这样的区分有利于后期更方便地做常量折叠优化。

此外，MATCHFAIL 是一特殊常量，用于表示不完全模式匹配的后备分支。如前文所述，若进入这个分支，则程序立即抛出错误并终止。在具体实现中，MATCHFAIL 能够接收一个被匹配对象变量名的参数，后者会传递到并用于后端代码生成，以产生更好定位的错误信息。

⁵¹ *Lambda* IR 是致敬 Appel 在其工作 *Compiling with Continuations*（可能也包括 SML/NJ 编译器）使用的同名 IR；*Tigris* 本身则是致敬 Appel 的 *Modern Compiler Implementation in ML*，昵称虎书，因为其封面有一只老虎。*Tigris* 有拉丁语的老虎之意。

LExpr 与 LFun 前者是 *Lambda IR* 程序的主体, 包含序列操作、立即数 Let-绑定 (记 let_ι)、右手侧 Let-绑定 (记 let)、LetRec-绑定 (记 let_ω): 三种 Let 因其右手侧不同而不同, 分别是 Value、Rhs 和递归定义。

LFun 则表示 IR 中的一个顶层函数, 是一个 3-元组 fid, param, body, 分别代表函数名、参数名和函数体 (LExpr)。在具体实现中, LetRec-绑定携带一系列顶层函数 (LFun), 而非 $\text{Name} \times \text{Name} \times \text{LExpr}$, 即便 LetRec 绑定的是本地函数。原因完全是性能上的, 因为两者具有相容的表示, 直接使用前者替代后者能够省去两种类型之间多余的转换。

尽管 *Tigris* 是纯函数式语言, 我们仍用序列操作 seq 来捕捉尾调用 Tail。这是因为具体实现中, 只有 seq 结点能够携带尾调用信息。而且, 考虑到以后可能为 *Tigris* 加入不纯模式, 将其转变为类似 OCaml 的语言, 具备 seq 结点可以减轻工作量。

Lambda IR 沿用前述的 pretty-printing 算法, 但因为篇幅限制, 我们不再给出范例。读者只需知道其语法类似 Haskell 和 SML 的混合体即可。

IR 降级过程中有一些很可注意的细节值得探讨。下面我们针对降级期间一些特殊对待的情形作探讨。

捷径: 构造子与原生操作 *Tigris* 的七种原生操作与 0-元构造子不存在函数调用产生的性能损失, 而原生调用的检测则是通过语法树上的模式匹配实现的。因而有时需要做 η -expand, 否则可能出现漏检情况。具体实现中, 无论是原生操作还是普通函数, 对降级过程而言都是相同的, 经由 App 分支, 但很明显特殊对待前者能够获得更好的性能——目前两者唯一的区别就是其标识符——通过在降级最开始的位置匹配这些标识符, 我们即可优先处理原生调用, 避免函数调用带来的性能损失 (因为目前编译器未做 inline optimization)。

```

1  partial def lowerF (e : FExpr) (p : Env) (ctors : Std.HashMap String Nat)
2    (k : Name → M σ LExpr) : M σ LExpr := do
3    match matchBinaryPrim e with
4    | some (op, x, y) =>
5      lowerF x p ctors fun vx =>
6      lowerF y p ctors fun vy => do
7      let r ← fresh "p"
8      let cont ← k r
9      return .letRhs r (.prim op #[vx, vy]) cont
10   | none =>
11     match matchCtorApp ctors e with (...) -- 此处处理构造子 (与上同) 和普通降级

```

字典投影 如前文所述, 是专门为类型类方法设计的。根据最原始的字典传递实现, 方法是通过模式匹配投影出来的。这不仅增加代码生成体积, 亦容易复杂化降级过程。因而在前文引入这一结点, 并在 *Lambda IR* 的降级过程中传递, 最后在后端直接生成投影代码, 避免经过模式匹配。

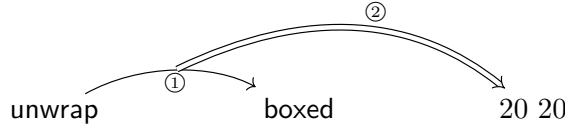
函数, 函数应用与逆柯里化 函数式语言通常保证柯里化。但当我们考虑生成代码时, 就不得不注意其带来的性能问题——为 n -元函数产生 n 个函数显然是不现实且臃肿、低效的, 同时还增加目标语言做尾递归优化的难度。因而绝大多数以函数作为头等对象的语言 (广义上的函数式语言) 都做逆柯里化 (uncurrying) 优化。该优化通过将连续的函数头 (柯里化满足这一特征) 压缩成一个 n -元组作为单参数 (这是我们的具体实现) 以便被编码为立即值 Value 并简化柯里化函数的 IR 降级。

函数降级有以下若干值得注意的概念:

1. **解构连续函数头** 是用于压缩 (squash) 具有连续函数头 (无论是否由柯里化造成) 的函数, 例如 $\text{fun } x \Rightarrow \text{fun } y \Rightarrow \dots$, 并返回 3-元组 (p0, rest, core), 分别指代函数首参数、剩余参数与函数体。
2. η -expansion 不仅仅指 λ -代数中的计算法则, 在此处更指用于解决降级的函数应用 (App) 分支没有正确包含的边界情况。考虑应用通过模式匹配投影产生的函数, 如 (Boxed 是一构造子)

```
let boxed = Boxed ( _ + _ ) and unwrap (Boxed f) = f
in unwrap boxed 20 20
```

其中 `unwrap boxed 20 20` 称为高阶应用 (higher-order application): `boxed` 的应用在 `Let` 程序体内发生, 但并不能被降级过程捕捉, 因为只有应用头 `unwrap` 对其可见, 且 `unwrap` 是一高阶函数 (即它返回函数, 但其函数体内没有应用), 因而这条语句其实发生两次函数应用, 但降级过程只可见第一次; 形象地说:



应用①对降级过程可见, 但是, 由其返回的函数产生的应用②对降级不可见。因而, `unwrap` 在这一过程中会隐式 η -expand 为 `let unwrap (Boxed f) x y = f x y`, 显式化高阶应用。 η -expansion 过程产生的参数有 $.. \eta N$ 的形式, 例如 `_pR#η0`。

熟悉 λ -代数的读者或许知道 η -expansion 可能在 CBV 下带来意想不到的求值语义——譬如上文提到的 Y 组合子的 Church 实现若不加更改, 在 CBV 下不会停机, 但 η -expansion 推翻了这一结论, 产生语义不一致问题——且我们无法避免其发生。因此, 与其无条件地做 η -expansion, 在具体实现中我们只对上面这种情况进行特殊处理, 其他情况不变, 以期尽可能减少对于语义的影响。

此外 Lean 亦有 η -expansion 导致语义不一致的问题, 但由于危害不大, 其开发者目前也不准备对此做任何处理。见脚注笔者发表的 Proofexchange 帖子⁵², 以及帖子中 Lean 类型论设计者的回答和 GitHub issue #5908⁵³。

3. **语法元数 (syntactic arity)** 指的是用户在函数定义时给出的元数。问题在于将不同种类的元数同上述的高阶应用/函数混搭时易出现问题, 而函数式语言恰巧存在大量此种风格的程序。一般来说, 降级工作通过检查函数对应的类型来决定函数的元数——实践中发现这一检查并不足够。考虑上文的 `unwrap : BoxedAdd → Int → Int → Int`, 从类型上看应为 3-元函数 (由于柯里化), 但根据用户给出的定义, 实际上只接受一个参数——即便传给它另外两个参数, 它也不知道该怎么办——因为其函数体根本没有出现另外的参数。因而, 只有在经过 η -expansion 后它才能够是真正的 3-元函数。相比于类型决定的元数, 在逆柯里化工作中, 我们更多依靠语法元数。
4. **逆柯里化投影生成** 指的是为逆柯里化的函数生成投影代码的过程。因为我们使用元组来实现逆柯里化, 因而在函数实际要使用传入的参数时需要首先将其投影出来。这一变换通常与闭包转换一起进行, 我们在下文介绍。
5. **栈底层降级工作** CPS 风格的递归代码在降级过程中是经常使用的, 而递归的首要工作就是明确基准情况 (base case) ——续点栈 (continuation stack) 底层的降级工作 `lowerFCore` 所表示的就是这一情况。`lowerFCore` 可以看作和 IR 中的 `Tail` 存在同像性 (homoiconicity), 因为从降级工作的角度, 或是 IR (即 object language, 数理逻辑上的对象语言) 的角度而言, 两者都代表尾调用——`lowerFCore` 本身也用于 `Tail` 的降级。前者定义为:

```
1 @[inline] partial def lowerFCore
2   (e : FExpr) (p : Env) (ctors : Std.HashMap String Nat) : M σ LExpr :=
3   lowerF e p ctors $ pure ○ .seq #[] ○ .ret
```

其中 `○` 表示函数复合运算符, 在函数式语言中十分常见。而对比上方的 `lowerF` 也可以发现, 栈底层的降级不需要传入续点 $k : \text{Name} \rightarrow M \sigma LExpr$, 因而不存在由续点造成的控制转移 (control transfer)。关于续点、续点传递 (化) 的概念我们将在 CPS 化一节中谈到。

⁵²<https://proofassistants.stackexchange.com/a/4424>

⁵³<https://github.com/leanprover/lean4/issues/5908>

实现逆柯里化是在函数定义和函数应用上共同努力的结果——不妨先来定义逆柯里化函数的参数安排 (layout)。对于某 n -元函数的参数 $a_1, \dots, a_n$ ，我们构造一元组 pl (payload) 其中

$$pl = \begin{cases} ((), ()) & n = 1, a_1 = () \\ (a_1, ()) & n = 1 \\ (a_1, \dots, a_n) & n \geq 2 \end{cases}$$

作者必须承认，事后考虑起来，这一 layout 存在一些设计问题。虽然它是元组，但编码上其与 List 相仿，比如单元素时在元组尾端加入了一个哨兵元 (sentinel)，但在 $n \geq 2$ 时又将之去掉。如果——且恰巧——没有类型信息的辅助，要判断这一元组的形状较难，为后端代码生成徒增了不必要的麻烦。此外，我们还应当在编译器前端就将语法元数收集起来，而不是在这一阶段按需收集，导致代码的可维护性下降。

从函数应用的角度考虑，逆柯里化的函数在应用时必须区分完全应用与部分应用。我们的做法是分情况讨论：对于前者，分为单参数、类型/语法元数不匹配两种情况进行处理；对于后者，根据缺少的部分计算需要补充的元数，并根据数目封装一层新 Lambda 抽象。同理，对于函数形式的值——构造子，我们也采取相同的操作。但由于构造子毕竟不是函数，不具备函数体，仅仅是能够应用，则处理起来相对简单许多。

模式匹配 是通过决策树算法⁵⁴ 开展编译工作的。我们定义三种类型 Sel、RowState、DTree，分别表示模式投影、模式行向量以及决策树的数据结构本身。其中，决策树有五个分支，分别编码匹配失败、叶子（即一个行向量）、常量匹配、构造子匹配与元组模式分割。在 PL 术语中，模式匹配编译工作的目标叫做匹配自动机 (matching automata)，主要分为决策树 (decision tree) 以及回溯自动机 (backtracking automata)。后者较为简单，且与人的思考方式类似：从模式矩阵的左上角开始，按先行后列顺序匹配矩阵中的每一个模式，直到其到达矩阵右下角（即遍历）。这类自动机的好处在于能够保证本部分代码生成的线性大小。但是正如其名，它可以回溯、重复匹配某些分支或模式，导致运行时的性能问题。与之不同，我们实现了一个能够保证不存在重复匹配同一值、模式对的决策树模式匹配编译器。为了展现这一编译器的优势，我们沿用 Maranget 工作的一个范例，考虑（为可读性，用 F、T、($\cdot$) 表示 false、true 和通配符）模式矩阵

$$\begin{pmatrix} \cdot & F & T \\ F & T & \cdot \\ \cdot & \cdot & F \\ \cdot & \cdot & T \end{pmatrix} \Rightarrow \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$$

在我们的实现下被翻译为 ($\emptyset$ 表示后备分支或通配符分支) 以下 IR：

```

1  case x of
2    false → case y of
3      true  → case z of true → RET 2 | false → RET 2
4      false → case z of true → RET 1 | false → RET 3
5     $\emptyset$    → case y of
6      false → case z of true → RET 1 | false → RET 3
7       $\emptyset$   → case z of true → RET 4 | false → RET 3

```

可以看出在最坏情况下，仅需 3 次即可得出结果，相比回溯自动机快上许多。当然，我们发现其实可以提出第 3 行的公共表达式，消去无用分支并直接返回 2。但这已超出决策树的优化范围且我们并没有做相关优化。不仅如此，由于已经在类型检查部分做过完全性检查，我们分担了一部分决策树算法的工作，因而此处实际上实现了一个轻量版的决策树算法，因为可以沿用前面步骤得到的信息。

⁵⁴从实现上看，完全/冗余性检查可以看作是这一算法的超轻量版本。

4.2 Lambda IR 的闭包转换

闭包转换 指的是将本地绑定的函数提 (hoist) 到全局, 并将之变换为全局定义的函数的操作。实际的工作并不像听起来一般简单, 因为本地函数——其他语言常有的 lambda——会捕捉当前词法作用域内的自由变量, 因而我们的主要挑战就是消去这些捕捉的自由变量。

虽然一些文章认为闭包转换与 lambda 提升 (lambda lifting) 有差异, 因为前者只是将自由变量加入参数列表而不调整调用处——此处仍用“闭包转换”一词表示 *Tigris* 取两者其中的实现思路。我们实现了前者, 但同时亦做了一些独属于后者的变换, 故而称之“取两者其中”。

此外, 有一些读者可能要问, 既然我们以本就是函数式语言的 Common Lisp 作为编译目标, 为何还要花费心思去做这一件事——原因主要是工作量上的, 我们希望重新发明轮子以展示本项目的工作量, 同时亦作为一种对函数式语言设计实现的实践训练, 还为本课题的目标对象——开展编程语言实践训练的本科生们作为学习样例。

“中间派”实现思路 指我们将 *Tigris* 程序中出现的的所有 Lambda 都进行闭包转换, 并为之添加一个显式的环境参数用于存储捕捉的自由变量的步骤。这一环境参数同函数指针⁵⁵一起包装, 称为函数对象 (function object)。当我们应用函数对象时, 环境参数 Γ (后同) 会同逆柯里化的参数元组, 即前面的 pl ⁵⁶ 组装起来, 并一同传给函数指针。“取两者其中”体现在我们虽然消去了所有的自由变量, 但同时也创建了一个函数对象, 虽然函数本身在这一过程中已经提到顶层。而当提到 (referencing, 指调用或传递) 该函数时, 我们实际上是在指代函数对象而非孤零零的函数指针。因而, 我们不仅消灭了 lambda, 也修改了函数调用位置, 既不是单纯的闭包转换, 也没有完全达到 lambda lifting, 因为后者不允许出现上面的 Γ 。

与逆柯里化一样, 闭包转换也在影响着函数定义与应用的方方面面。既然我们已经介绍了这两个概念, 现在就可给出 *Tigris* 的函数调用规范 (function calling convention) 了。

函数调用规范 从本质上看就是函数对象的编码。综合以上内容, *Tigris* 的函数调用规范由以下五点给出:

1. pl 即 payload, 指经过闭包转换的函数的参数, 有 $pl = \langle arg, \Gamma \rangle$ 其中 arg 指逆柯里化产生的参数元组。
2. Γ 又称闭包环境, 指对捕捉的自由变量的显式存储。闭包环境是一构造值 $\mathbf{E}[fv_1, \dots, fv_n]$, 在实践中, 为了避免构造子标识符的冲突, 我们使用不能通过词法分析的特殊符号 U+1D404 MATHEMATICAL BOLD CAPITAL E 指代闭包环境构造子。 Γ 与其余构造值并无不同, 仍然使用现有 IR 的语言组件编码。
3. 闭包又称函数对象, 是 2-元构造值 $\mathbf{C}[ptr, \Gamma]$ 其中 ptr 指代一函数指针 (又称代码指针, code pointer), 指针本质上是字符串。具体实现中, 为了避免构造子标识符的冲突, 我们使用不能通过词法分析的特殊符号 U+1D402 MATHEMATICAL BOLD CAPITAL C 指代函数对象构造子。同上, 闭包仍然由现有 IR 的语言组件编码。
4. 闭包应用是通过投影 $pl = \langle arg, \Gamma \rangle$ 并将 (arg, Γ) 传给 ptr 实现的。
5. 互递归也包括广义的单递归, 是通过将某一 Letrec-绑定组做变换使得其中的每一个函数的函数体都从 payload 中投影 (arg, Γ) . 函数体内的递归调用直接使用同一个闭包环境⁵⁷, 一组互递归函数共享同一 Γ 。

前文说过, *Tigris* 不区分函数绑定与传统意义上的值绑定, 这就是说任一函数定义都可视作 (也实际是) 一个 lambda 抽象——所以无论一个函数全局与否, 其总是要经过闭包转换。而对所谓的全局

⁵⁵在 SBCL 后端中, “指针” 无非是用来指代函数的标识符, 从语义上可称指针, 非 C 语言的同名概念。

⁵⁶读者注意, 后文用 pl 指代逆柯里化的参数元组与环境参数的综合体, 而非单独的前者。

⁵⁷其好处在于能够避免每次递归都要组装一遍 pl 带来的多余性能损失。通过这点我们能够获得逼近在目标语言中原生递归函数的递归性能 (因为生成的后端代码的递归调用与原生无异)。

函数而言，闭包转换就在全局收集自由变量并显式地存在对应的闭包环境中。这样做，保证了理论的统一性（ λ -代数本来也不存在全局/本地的差异），也简化了代码实现，使得我们不必特殊处理全局函数。

Remark (性能考量). 有读者可能认为对于全局定义、不包含自由变量 ($\Gamma = \emptyset$) 的函数来说，将其不断的装入/脱去某函数对象是一件很没必要且浪费性能的事情，为什么不直接应用这些函数——的确，本文作者极其认同这一点。在旧版的 *Tigris* 中我们启用了这一试验性优化，但也因此产生了一些问题，所以目前为禁用状态。虽然作者同样认为占据程序性能大头的（互）递归函数才是真正的问题所在，而这些函数依据上面的调用规范不存在这一性能问题，因此我们认为实际产生的性能影响并不很严重。

Remark (优化：融合立即出现的尾调用). 我们用一个简单的模式匹配来处理极常见的一种可以优化的情况：如果表达式具有模式 $f^T(a)$ ，其中 f 是上一行（层）代码刚刚由 fid, Γ 构造的函数对象，我们将之改写为 $fid^T((a, \Gamma))$ ，并通过下文将要介绍的 DCE 优化将不再用到的旧函数对象删除。

```

1 def fuseTail (envName x fid : Name) : LExpr → LExpr
2   | .seq binds (.app f a) ⇒
3     if f == x then
4       let pl := "p"; .seq (binds.push (.let1 pl (.mkPair a envName))) (.app fid pl)
5     else .seq binds (.app f a)
6   | .letVal y v b ⇒ .letVal y v (fuseTail envName x fid b) | .letRhs y r b ⇒ -- (.. 同理, 略..)

```

4.3 Lambda IR 的优化

代码优化 是在闭包转换前后所作的。闭包转换之后可能暴露出更多优化机会、产生更多语法噪音，因此需要重新过一遍优化。*Tigris* 编译器目前主要有三种优化：常量折叠/传播、复制传播及死代码消除（dead code elimination, DCE）和最为重要的尾调用优化（tailcall-ify）。

常量折叠与传播 指通过将绑定的常量传播到所有与之相关的代码，并在编译期直接计算出结果的优化。目前而言，*Tigris* 编译器可以做整数的四则运算与所有常量类型的相等比较。虽然这里涉及的许多操作可能存在结合律、交换律与分配律，我们的实现目前不支持这些变换。例如 $10 + 10 + x + 10$ 可以化简为 $20 + x + 10$ ，但并非 $30 + x$ 。当然，编译器还支持控制流语句的折叠，譬如模式匹配搭配条件语句

```

let xy = {x = 1, y = 2} and b = true
in let {x = x', y = y'} = xy in if b then x' + y' else 0

```

会被化简到 3。除此之外，常量折叠搭配类型类能产生意想不到的效果。考虑某一通过变量名实现的实例，如用整数的相等关系 `eqInt` 实现 `Eq` 的 `(=)` 方法。当我们在代码中调用后者如 `(1 = 0)` 时，因常量折叠能够处理字典投影，表达式直接被重写为 `eqInt 1 0`。这一优势在于传统的字典传递实现的类型类，虽较面向对象的接口更快，但仍然存在字典投影的 overhead。通过搭配常量折叠，我们能真正地实现纯静态编译期的类型类系统，达成零成本抽象。

复制传播与死代码消除 前者是与其他优化，比如上述常量折叠一同使用，通过 `Let`-绑定中等号的传递性，编译器能够追逐（chase）某一标识符，直到其表示某一个常量为止。沿用上面的例子：编译器之所以知道 `xy` 指代前面的结构体，就是得益于复制传播。

另一方面，DCE 就如同其字面义一般简单：消除所有无用的代码，比如某一函数中从来未使用过的绑定。*Tigris* 的问题在于其虽然是纯函数式语言，但我们也提供了直接通过 FFI 引入副作用的 hack。因而在新版本的 *Tigris* 中 DCE 不再过于激进，如果 `Let`-绑定的左手侧是一通配符模式，则我们不移除这条代码，因为这种写法常见于不纯函数式语言中用于执行某些具有副作用的代码，例如 OCaml。

```

let rec countTree
| Empty  $\Rightarrow$  0
| Node _ f  $\Rightarrow$  1 + countForest f
and countForest
| Nil  $\Rightarrow$  0
| Cons t f  $\Rightarrow$  countTree t + countForest f

let rec even
| 0  $\Rightarrow$  true
| n  $\Rightarrow$  odd $ n - 1
and odd
| 0  $\Rightarrow$  false
| n  $\Rightarrow$  even $ n - 1

```

(a) non-tail recursive

(b) tail-recursive

Figure 5: Non-tailcall vs. tailcall

尾调用优化 是函数式语言中几乎必备的一个优化。使用任一函数式语言，或阅读过 *SICP*⁵⁸ 的读者可能已经对此比较熟悉，但我们仍不厌其烦地在下文作一简单介绍。

函数式语言最明显的特征之一就是其在程序中广泛地使用函数 (subroutines)。函数与调用栈 (call stack) 不可分割：当程序通过某一调用进入 subroutine 时需要将对应调用的上下文 (返回点、栈上分配的值) 压入调用栈中。当 subroutine 重新将控制权移交回上一级，则出栈。若某一程序的栈空间用完，则其将被操作系统杀死并产生 *stack overflow* 异常。然而我们发现，在程序 (函数) 的中间调用其他函数，与在尾部调用是两种不同的情况——后者可看作控制权的移交 (transfer of control)⁵⁹，因而可用指令 *j* 而非 *call* 替代 (假设 RISC-V ISA)。相比后者，前者在控制转移后可以立即弹出栈帧，释放栈空间。

显然，最需要调用栈资源的语言对象就是递归调用了。我们称一个函数是尾递归的，当其以调用自身 (或同一互递归函数组的函数) 结尾。在这种情况下，一个经过尾递归优化的函数会不停的在尾递归调用之间压栈、出栈调用栈。尾调用优化要求像这样的函数 (包括非递归函数) 的空间复杂度为 $O(1)$ 而非 $O(n)$ 。Figure 5 展示了两组互递归函数，一组具备尾递归优化的先决条件，一组则不具备。

将一个函数转变为尾调用风格，既可以通过手写，也可以通过机械化的 CPS 变换。直观的说，尾递归函数在语义和思考模式上和迭代相似，而后的栈空间复杂度显然是 $O(1)$ 。根据上面的叙述，有读者会问：尾调用优化看似是在底层做的，那么 *Lambda IR* 层面的尾递归优化到底代表什么呢？

答案是在这一阶段标记尾调用位置 (由 Tail 的 app 构造子编码) 并将这一信息传递给后端，由其保证在生成的 LISP 代码中标记的尾调用位置不变——如果缺少这一信息，后端的压力会大上许多。上面说过，尾调用位置形如 $f(arg)^T$ ，其中 *f* 根据编译阶段是 (CC 前后) 函数名或函数指针。简单总结几个优化的顺序，就是

- ① 常量折叠 ② DCE ③ 尾调用标记 ④ DCE.

最后，用于总结这一部分的内容，我们来探讨上文反复提到的 *CPS*，亦为下一部分续点传递化的叙述作理论基础。*CPS* 作为一种编程风格，在我们的降级实现中十分常见。续点函数 (continuation function) *k*，由同像性，在对象语言中表示一个挖空^{kōng} (pluggable) 的表达式，即从这一点往后的程序。

$k :$	$\text{Name} \longrightarrow \mu \sigma \text{LExp}$	(元语言; 仅含类型)
	$\downarrow \text{plugging}$ $\downarrow \text{lowering}$	
let	$\bullet \quad = \dots \text{in} \quad e$	(对象语言: <i>Lambda IR</i>)

挖出的“空” ($\bullet$) 可能 (但应该, 如果不是没有意义的话) 在 *e* 中有绑定。我们称之为“挖空”, 是因为如同“填空题”一般, 将这一续点应用 (把空填上) 到某个语言对象 (这里只能是变量, 即字符串) 上, 比如变量 *m*, 则会使得 *m* 在 *e* 中有绑定, 并返回剩下的表达式, 即这一点往后的程序。从这个意义上讲, 续点可以看作是一个程序的模板。从实现的角度考虑, 这一操作不过是在显式的 *CPS* 栈

⁵⁸ *Structure and Interpretation of Computer Programs*

⁵⁹ 显然控制权的移交还包括各种控制流语句或非本地出口。后者指那些不是通过函数返回跳出某一函数的情况, 例如传统语言中具备的 *goto*、*throw*, 或 *early return*。

中运行单子 μ 下的操作而已——CPS 栈是我们显式维护的，本质上是由许多用于降级工作的小续点（续点在编程语言中基本使用函数编码，除非原生支持）复合而成的大闭包； μ 则是 M 单子，专门用于降级过程，有定义

```
1 abbrev M  $\sigma$  := StateRefT (Nat  $\times$  AMap) (ST  $\sigma$ )
```

其中 AMap 正是我们用于记录语法元数，从 Name（表示 ANF 的名字，即字符串）映射到 Nat（自然数）的哈希表。之所以使用 CPS 风格实现递归，是根据上述的同像性我们能够轻易的将变量名从元语言（即 Lean）传播到对象语言（即 IR）中。

增量式 API 与 Tigrisi IR 解释器 是本项目编程子系统的一个试验性扩展。While the *Tigris* compiler compiles programs in one go, it is rather inefficient for REPLs as they have different use logic: users of a REPL session usually enter a program *piece by piece*, while expecting immediate result after entering them. A naive REPL might track everything they entered since the start of REPL then *re-compiles* it per user entrance, which, for multi-stages language implementations with several passes is horribly inefficient. Thus, the incremental APIs are born, which lives in the Incremental namespace.

When implementing the incremental APIs, we not only have to track every state⁶⁰, but also the use of it inside `{ftransform, lift, opt}.lean` (is it intertwined, etc):

```
structure LoweringState where
  gensym : Nat
  env     : Env
  ctors   : Std.HashMap String Nat
  arity   : Std.HashMap String Nat
  deriving Inhabited
```

Interested users shall refer to them in the bottoms of `ftransform.lean` and `lift.lean`. This is considered highly experimental and unstable since it is not thoroughly tested.

the Tigrisi IR interpreter targets the *Lambda* IR and can run independently regardless of with incremental APIs or not. Though in the other case it runs much slower due to worse compilation performance. The interpreter itself is stable, but due to its partial dependency on the incremental APIs, we still consider it experimental.

Interpreters, just like virtual machines, have states, such as the OCaml VM that runs the bytecode produced by `ocamlc`. Same is applied to *Tigris*. Largely, these states are introduced by bindings, whether local or global, or functional. In this case, the state is a *mapping*, or *table* (e.g. a Hashmap or Treemap). An important design aspect to consider is to make the state as lightweight as possible, since allocating them, projecting/modifying them consumes time and space. In our implementation, we define immediate values and environments as:

```
inductive Val where
  | unit
  | int (i : Int)
  | bool (b : Bool)
  | str (s : String)
  | pair (p q : Val)
  | ctor (tag : Name) (fields : Array Val)
  | code (pointer : Name)
  deriving Repr, BEq, Inhabited
```

⁶⁰and temporary states that is dropped after the compilation of a program, but should be saved in this case.

```

abbrev Env      := Std.HashMap Name Val      -- local binding table
abbrev FunTab   := Std.HashMap Name LFun     -- Function table
abbrev GlobalEnv := Std.HashMap Name Val     -- global binding table

```

On encountering a `let*`⁶¹, relevant bindings are inserted to one of the three tables defined above.

Tigrisi runs in `EIO`⁶² and is *multi-threaded*: evaluation runs in a separate thread. This way it can respond to the keystroke `<C-c>` from user, on encountering which the REPL thread is terminated. Termination of threads is *cooperative*: A joint effort of the interpreter, through `checkInterrupt` up front at crucial branch (`eval*` functions):

```

1  /-! leval.lean -/
2  private def checkInterrupt : EIO TypingError Unit :=
3    IO.checkCanceled >=> fun
4      | true  => throw .Interrupted
5      | false => return ()
6
7  /-! ffidecl.lean -/
8  @[extern "lean_sigint_pipe"]
9  opaque installSigintPipe : IO Int32
10 @[extern "lean_read_fd_byte"]
11 opaque readFdByte : @&Int32 → IO Int32
12
13 /-! Tigrisi.lean -/
14 -- (.. within main ..)
15 let stdin ← getStdin
16 let replST ← mkRef LApp.initREPL
17 let EVS : IO.Ref EvalState ← mkRef none
18
19 let fd ← installSigintPipe
20
21 if fd ≥ 0 then
22   discard $ asTask (prio := .dedicated) do
23     repeat do
24       let r ← readFdByte fd
25       if r ≤ 0 then pure ()
26       else
27         if let some t ← EVS.get then
28           IO.cancel t
29         else pure ()
30 -- (..)

```

with the help of CLI frontend (above) and Lean's C FFI interface (we omit the `win32` implementation).⁶³

```

1  typedef lean_obj_res OBJRES;
2  /* (.. win32 impl ..) */
3  static int32_t sig_pipe[2] = {-1, -1};

```

⁶¹* is wildcard, matching empty string, ι or ω , as in the pretty-printed syntax of *Lambda IR*. It has nothing to do with sequential application from LISP or OCaml.

⁶²that is, IO with exception handling.

⁶³this approach is adopted from the deprecated REPL *tigris* (which is a subset of the current *Tigris*).

```

4
5 static void sigint_handler() {
6     // write is async-signal-safe
7     char b = 1;
8     if (sig_pipe[1] >= 0) {
9         (void)!write(sig_pipe[1], &b, 1);
10    }
11 }
12
13 LEAN_EXPORT OBJRES lean_sigint_pipe() {
14     if (sig_pipe[0] != -1)
15         return lean_io_result_mk_ok(lean_box(sig_pipe[0])); // already installed
16     if (pipe(sig_pipe) != 0)
17         return lean_mk_io_user_error(
18             lean_mk_string("sigint pipe installation failed"));
19     struct sigaction sa;
20     memset(&sa, 0, sizeof(sa));
21     sa.sa_handler = sigint_handler;
22     sigemptyset(&sa.sa_mask);
23     sa.sa_flags = SA_RESTART; // restart interrupted syscalls
24     if (sigaction(SIGINT, &sa, NULL) != 0) {
25         close(sig_pipe[0]);
26         close(sig_pipe[1]);
27         sig_pipe[0] = sig_pipe[1] = -1;
28         return lean_mk_io_user_error(lean_mk_string("call to sigaction() failed"));
29     }
30     return lean_io_result_mk_ok(lean_box(sig_pipe[0]));
31 }
32
33 LEAN_EXPORT lean_obj_res lean_read_fd_byte(int32_t fd) {
34     char b;
35     for (;;) {
36         ssize_t r = read(fd, &b, 1);
37         if (r == 1)
38             return lean_io_result_mk_ok(lean_box(1));
39         if (r == 0)
40             return lean_io_result_mk_ok(lean_box(0)); // EOF
41         if (errno == EINTR)
42             continue; // retry
43         return lean_mk_io_user_error(lean_mk_string("read() failed"));
44     }
45 }

```

The CLI frontend is responsible for launching evaluation threads EVS as well as a *sentinel thread* that keeps reading the sigint pipe, on detecting cancel signals from the user, signals EVS to exit, which is then responded by `checkInterrupted` within the thread, who exits *gracefully* i.e. through normal returns. EVS is a threadlocal mutable state `IO.Ref EvalState` where `EvalState` is defined as

```

abbrev EvalState := Option
    $ Task
    $ Except Error

```



```

notchip in ● pop-os122 in tigris on / cwc [!+] via λ 9.6.7 via ↗ v5.3.0 (default)
[7s] $ .lake/build/bin/tigris
A basic language using Hindley-Milner type system
with a naive (term-rewriting) interpreted implementation.
For language specifications see source.
USAGE
• Type #help;; to check available commands.
• Exit with <C-d> or <C-z>-Ret> (Windows).
• Interrupt with <C-c> (doesn't work if
  running through 'lake exe')
> let rec id x = id x in id 20;;
^C[ERROR] Interrupted.
>

```

Figure 6: A user interrupting a computation

\$ LApp.REPLState × LInterpreter.Val × Scheme

where, Task is a Lean primitive representing asynchronous computation, similar to *coroutines*, such as Scala’s Future, Javascript’s Promise and Rust’s JoinHandle. The POSIX sigint pipe can only be read if it is installed. Implementation of this pipe, its handler and reader is in C, through the FFI to be called in Lean, as shown above.

Now, if some computation diverges, e.g., `let rec loop x = loop x in loop ()`, users can press `<C-c>` to interrupt a thread: Figure 6. Though, users should note that there is a caveat: an tigris REPL launched through Lake⁶⁴ has broken sigint pipe where `<C-c>` just terminates the program, instead, users should launch the program directly, which, should be the default behavior if not building from source.

The *Tigris* interpreter can be invoked with `interpretL` (w/o incr. APIs) or `interpretI`, of which the former is stable and is used to quickly test the lowering stage to see if there is any issue.

4.4 CPS transformation

CPS transformation refers to the transformation of *Lambda* IR into its CPS counterparts where

- every function takes an explicit continuation parameter k ,
- function calls pass results to continuations rather than returning,
- continuations are *not* first-class: they are either locally-bound labels or the special parameter k ,
- ANF is maintained.

The use of CPS, as explained in the section about tailcall, is to transform non-tailrec functions. This is rather specific as in general, mechanized CPS is used to encode complex control flow, similar to a longjmp and CPS transformation is to make control flow explicit, at the expense of increased heap usage, as continuations are usually lambda abstraction⁶⁵ them selves, being passed around together with function parameters. The intuition is that, CPS trades heap space for stack space, by allocating and *compositing* continuations, which, can be thought as

an explicit call stack (that replaces the actual call stack) which is unwinded when applied.

The CPS IR is similar to *Lambda* IR, defined as (CName is Name)

⁶⁴Lean’s build system

⁶⁵which usually is allocated in heap. Though most compilers may force stack allocation if there is an optimization window, that defeats the purpose of using CPS to save stack space if it is too naive.

```

1  inductive CRhs where
2    | prim      (op : PrimOp) (args : Array CName)
3    | proj      (src : CName) (idx : Nat)
4    | mkPair     (a b : CName)
5    | mkConstr  (tag : Name) (fields : Array CName)
6    | isConstr  (src : CName) (tag : Name) (arity : Nat)
7    | const     (k : Const)
8    | alias     (y : CName) -- x = y
9  deriving Repr, Inhabited, BEq
10
11  mutual
12  inductive CTail where
13    | appFun      (f : CName) (payload : CName) (k : CName)
14    | appKont     (k : CName) (value : CName)
15    | ite         (condVar : CName) (tBranch eBranch : CExpr)
16    | switchConst (scrut : CName)
17                  (cases : Array (Const × CExpr))
18                  (default? : Option CExpr)
19    | switchCtor  (scrut : CName)
20                  (cases : Array (Name × Nat × CExpr))
21                  (default? : Option CExpr)
22    | halt        (value : CName)
23    | matchFail   (pat : Array Name)
24  deriving Repr, Inhabited, BEq
25
26  /-- CPS expression in ANF: pure let1 / letKont / local fun groups, ending with a tail. -/
27  inductive CExpr where
28    | let1      (x : CName) (rhs : CRhs) (body : CExpr)
29    | letKont   (kid : CName) (param : CName) (kBody : CExpr) (body : CExpr)
30    | letRec    (funs : Array CFun) (body : CExpr)
31    | tail      (t : CTail)
32  deriving Repr, Inhabited, BEq
33
34  structure CFun where
35    fid          : CName
36    payloadParam : CName
37    kontParam    : CName
38    body         : CExpr
39  deriving Repr, Inhabited, BEq
40  end

```

The CPS transformation is the last step of the IR lowering process, which, is further lowered by the SBCL backend to Common Lisp. Semantics of the actual program should be similar to that of CPS, provided if there is not any backend inconsistency⁶⁶

First, we present the formal grammar of *Lambda* IR, and its CPS target:

$$v \in \text{Val} ::= x \mid k \mid \text{ctor}(t, \bar{x}) \mid \lambda p. e$$

⁶⁶which, unfortunately, remains an assumption because *Tigris* is not formally verified. Note that, the M monad is continued to use for this transformation.

$$\begin{aligned}
r \in \text{Rhs} &::= \text{prim}(op, \bar{x}) \mid \text{proj}(x, i) \mid \text{mkPair}(x, y) \\
&\mid \text{mkCtor}(t, \bar{x}) \mid \text{isCtor}(x, t, n) \mid f(a) \\
s \in \text{Stmt} &::= \text{let}_1 x = r \\
\tau \in \text{Tail} &::= \text{ret } x \mid f(a)^\top \\
&\mid \text{if } c \text{ then } e_1 \text{ else } e_2 \\
&\mid \text{switch}^{\text{Const}}(x, \overline{k_i \rightarrow e_i, e_i}) \\
&\mid \text{switch}^{\text{ctor}}(x, \overline{(t_i, n_i) \rightarrow e_i, e_i}) \\
&\mid \text{matchFail} \\
e \in \text{LEExpr} &::= \bar{s}; \tau \\
&\mid \text{let}_l x = v \text{ in } e \\
&\mid \text{let}_1 x = r \text{ in } e \\
&\mid \text{let rec } \overline{f_i(p_i) = e_i} \text{ in } e \\
r_c \in \text{CRhs} &::= \text{prim}(op, \bar{x}) \mid \text{proj}(x, i) \mid \text{mkPair}(x, y) \\
&\mid \text{mkCtor}(t, \bar{x}) \mid \text{isCtor}(x, t, n) \mid k \mid x \\
\tau_c \in \text{CTail} &::= f(p, k) \mid k(v) \mid \text{halt}(v) \\
&\mid \text{ite}(c, e_1, e_2) \\
&\mid \text{switch}^{\text{Const}}(x, \overline{k_i \rightarrow e_i, e_i}) \\
&\mid \text{switch}^{\text{ctor}}(x, \overline{(t_i, n_i) \rightarrow e_i, e_i}) \\
&\mid \text{matchFail} \\
e_c \in \text{CEExpr} &::= \text{let}_1 x = r_c \text{ in } e_c \\
&\mid \text{letK } \kappa(p) = e_c \text{ in } e_c \\
&\mid \text{let rec } \overline{f_i(p_i, k_i) = e_i} \text{ in } e_c \\
&\mid \text{tail } \tau_c
\end{aligned}$$

We define the *domain of semantics* (i.e. datatypes) as:⁶⁷

$$\begin{aligned}
\text{Val} &= () + \mathbb{Z} + \mathbb{B} + \text{String} + (\text{Val} \times \text{Val}) + (\text{Name} \times \text{Val}^*) + \text{Name} \\
\rho &= \text{Name} \rightarrow \text{Val} \\
\Phi &= \text{Name} \rightarrow (\text{Name} \times \text{Name} \times \text{CEExpr}) \\
\Phi_\kappa &= \text{Name} \rightarrow (\text{Name} \times \text{CEExpr}) \\
\text{Ans} &= \text{Val} + \text{Error}
\end{aligned}$$

The transformation $\mathcal{C} : \text{LEExpr} \rightarrow \text{CEExpr}$ is defined with respect to a *current continuation* k , which is either a continuation variable or should be the identity continuation id_{Ans} ⁶⁸ for top-level expressions.

Below, we present the denotational semantics for this transformation. Note that $C_{[\bullet]}^k$ represents a transformation with bound continuation k , with a hole denoting an element of the input domain $[\bullet]$, which is one of v, r, s, τ, e ; And $C_s^k[\![\bullet]\!](e_c)$ threads e_c . We simplify let^* and switch^* to just one binding/one discriminant.

⁶⁷think of $(+)$ and $(\times)$ as coproduct and product, respectively.

⁶⁸The standard *driver* in SBCL backend is this.

$$\begin{aligned}
\mathcal{C}_v \llbracket x \rrbracket &= x && \text{(Values)} \\
\mathcal{C}_v \llbracket k \rrbracket &= \text{const}(k) \\
\mathcal{C}_v \llbracket \text{ctor}(t, \bar{x}) \rrbracket &= \text{mkCtor}(t, \bar{x}) \\
\mathcal{C}_r \llbracket \text{prim}(op, \bar{x}) \rrbracket &= \text{prim}(op, \bar{x}) && \text{(Rhs)} \\
\mathcal{C}_r \llbracket \text{proj}(x, i) \rrbracket &= \text{proj}(x, i) \\
\mathcal{C}_r \llbracket \text{mkPair}(x, y) \rrbracket &= \text{mkPair}(x, y) \\
\mathcal{C}_r \llbracket \text{mkCtor}(t, \bar{x}) \rrbracket &= \text{mkCtor}(t, \bar{x}) \\
\mathcal{C}_r \llbracket \text{isCtor}(x, t, n) \rrbracket &= \text{isCtor}(x, t, n) \\
\mathcal{C}_r \llbracket f(a) \rrbracket &= (\text{see call transformation below}) \\
\mathcal{C}_\tau^k \llbracket \text{ret } x \rrbracket &= \text{tail } k(x) && \text{(Tails)} \\
\mathcal{C}_\tau^k \llbracket f(a)^\top \rrbracket &= \text{tail } f(a, k) \\
\mathcal{C}_\tau^k \llbracket \text{if } c \text{ then } e_1 \text{ else } e_2 \rrbracket &= \text{tail ite}(c, \mathcal{C}_\tau^k \llbracket e_1 \rrbracket, \mathcal{C}_\tau^k \llbracket e_2 \rrbracket) \\
\mathcal{C}_\tau^k \llbracket \text{switch}^{\text{Const}}(x, \overline{k_i \rightarrow e_i, e?}) \rrbracket &= \text{tail switch}^{\text{Const}}(x, \overline{k_i \rightarrow \mathcal{C}_\tau^k \llbracket e_i \rrbracket, \mathcal{C}_\tau^k \llbracket e? \rrbracket}) \\
\mathcal{C}_\tau^k \llbracket \text{switch}^{\text{ctor}}(x, \overline{(t_i, n_i) \rightarrow e_i, e?}) \rrbracket &= \text{tail switch}^{\text{ctor}}(x, \overline{(t_i, n_i) \rightarrow \mathcal{C}_\tau^k \llbracket e_i \rrbracket, \mathcal{C}_\tau^k \llbracket e? \rrbracket}) \\
\mathcal{C}_\tau^k \llbracket \text{matchFail} \rrbracket &= \text{tail matchFail} \\
\mathcal{C}_s^k \llbracket \bar{s}; \tau \rrbracket &= \mathcal{C}_s^k \llbracket \bar{s} \rrbracket (\mathcal{C}_\tau^k \llbracket \tau \rrbracket) && \text{(Stmt)} \\
\mathcal{C}_s^k \llbracket \epsilon \rrbracket (e_c) &= e_c \\
\mathcal{C}_s^k \llbracket (\text{let}_1 x = r); \bar{s} \rrbracket (e_c) &= \begin{cases} \text{letK } \kappa(x) = \mathcal{C}_s^k \llbracket \bar{s} \rrbracket (e_c) \text{ in tail } f(a, \kappa) & r = f(a) \\ \text{let}_1 x = \mathcal{C}_r \llbracket r \rrbracket \text{ in } \mathcal{C}_s^k \llbracket \bar{s} \rrbracket (e_c) & \text{otherwise} \end{cases} \\
\mathcal{C}^k \llbracket \text{let}_t x = v \text{ in } e \rrbracket &= \text{let}_1 x = \mathcal{C}_v \llbracket v \rrbracket \text{ in } \mathcal{C}^k \llbracket e \rrbracket && \text{(letVal)} \\
\mathcal{C}^k \llbracket \text{let}_1 x = f(a) \text{ in } e \rrbracket &= \text{letK } \kappa(x) = \mathcal{C}^k \llbracket e \rrbracket \text{ in tail } f(a, \kappa) && \text{(letRhs)} \\
\mathcal{C}^k \llbracket \text{let}_1 x = r \text{ in } e \rrbracket &= \text{let}_1 x = \mathcal{C}_r \llbracket r \rrbracket \text{ in } \mathcal{C}^k \llbracket e \rrbracket \quad (r \neq f(a)) \\
\mathcal{C}^k \llbracket \text{let rec } f(p) = e_f \text{ in } e \rrbracket &= \text{let rec } f(p, k_f) = \mathcal{C}^{k_f} \llbracket e_f \rrbracket \text{ in } \mathcal{C}^k \llbracket e \rrbracket && \text{(letRhs)}
\end{aligned}$$

Finally, we present the denotational semantics of CPS IR *itself*. Just as above, the semantics is given by

$$\begin{aligned}
\mathcal{E} : (\text{CExpr} \oplus \text{CTail}) &\rightarrow \rho \rightarrow \Phi \rightarrow \Phi_\kappa \rightarrow (\text{Val} \rightarrow \text{Ans}) \rightarrow \text{Ans} \\
\mathcal{R} : \text{CExpr} &\rightarrow \rho \rightarrow \text{Val}
\end{aligned}$$

where the fifth parameter i.e. of type $\text{Val} \rightarrow \text{Ans}$ is the *outermost continuation*, or the *meta-continuation*, denoted μ , which represents the toplevel continuation that is id_{Ans} or halt . We write $\rho[n \mapsto v]$ to denote the insertion⁶⁹ of mapping $n \mapsto v$ to ρ ; Readers should acknowledge that ρ, n, v are metavariables. Note that $\mathcal{E}$ takes both tails and expressions.

$${}^{70}\mathcal{R} \llbracket \text{prim}(op, \bar{x}) \rrbracket \rho = \delta(op, \rho(\bar{x})) \quad \text{(CRhs)}$$

⁶⁹insertion follows the standard behavior: if the key m exists, its value is replaced by v .

$$\begin{aligned}
\mathcal{R}[\text{proj}(x, i)]\rho &= \pi_i(\rho(x)) \\
\mathcal{R}[\text{mkPair}(x, y)]\rho &= (\rho(x), \rho(y)) \\
\mathcal{R}[\text{mkCtor}(t, \bar{x})]\rho &= (t, \rho(\bar{x})) \\
\mathcal{R}[\text{isCtor}(x, t, n)]\rho &= \begin{cases} \text{true} & \rho(x) = (t, \bar{v}) \wedge |\bar{v}| = n \\ \text{false} & \text{otherwise} \end{cases} \\
\mathcal{R}[\text{const}(k)]\rho &= k \\
\mathcal{R}[\text{alias}(y)]\rho &= \rho(y) \\
\mathcal{E}[f(p, k)]\rho\Phi\Phi_\kappa\mu &= \begin{cases} \mathcal{E}[e_f]\rho'\Phi'\Phi'_\kappa\mu & \Phi(f) = (p_f, k_f, e_f) \\ \text{Error} & \text{otherwise} \end{cases} \quad (\text{funapp}) \\
\text{where } \rho' &= \{p_f \mapsto \rho(p), k_f \mapsto \rho(k)\} \\
\Phi' &= \Phi \\
\Phi'_\kappa &= \emptyset \quad (\text{local continuations are scoped}) \\
{}^{71}\mathcal{E}[\kappa(v)]\rho\Phi\Phi_\kappa\mu &= \begin{cases} \mathcal{E}[e_\kappa]\rho[p_\kappa \mapsto \rho(v)]\Phi\Phi_\kappa\mu & \Phi_\kappa(\kappa) = (p_\kappa, e_\kappa) \\ \mu(\rho(v)) & \kappa = k_{\text{param}} \\ \text{Error} & \text{otherwise} \end{cases} \quad (\text{kontapp}) \\
\mathcal{E}[\text{halt}(v)]\rho\Phi\Phi_\kappa\mu &= \rho(v) \quad (\text{halt}) \\
\mathcal{E}[\text{ite}(c, e_1, e_2)]\rho\Phi\Phi_\kappa\mu &= \begin{cases} \mathcal{E}[e_1]\rho\Phi\Phi_\kappa\mu & \rho(c) = \text{true} \\ \mathcal{E}[e_2]\rho\Phi\Phi_\kappa\mu & \text{otherwise} \end{cases} \quad (\text{conditional}) \\
\mathcal{E}[\text{switch}^{\text{Const}}(x, \overline{k_i \rightarrow e_i}, e?)]\rho\Phi\Phi_\kappa\mu &= \begin{cases} \mathcal{E}[e_j]\rho\Phi\Phi_\kappa\mu & \exists j. \rho(x) = k_j \\ \mathcal{E}[e?]\rho\Phi\Phi_\kappa\mu & \forall i. \rho(x) \neq k_i \wedge e? \text{ exists} \\ \text{Error} & \text{otherwise} \end{cases} \quad (\text{const switch}) \\
\mathcal{E}[\text{switch}^{\text{ctor}}(x, \overline{(t_i, n_i) \rightarrow e_i}, e?)]\rho\Phi\Phi_\kappa\mu &= \begin{cases} \mathcal{E}[e_j]\rho\Phi\Phi_\kappa\mu & \rho(x) = (t_j, \bar{v}) \wedge |\bar{v}| = n_j \\ \mathcal{E}[e?]\rho\Phi\Phi_\kappa\mu & \text{no match; } e? \text{ exists} \\ \text{Error} & \text{otherwise} \end{cases} \quad (\text{ctor switch}) \\
\mathcal{E}[\text{matchFail}]\rho\Phi\Phi_\kappa\mu &= \text{MATCH-FAILURE} \quad (\text{matchfail}) \\
\mathcal{E}[\text{let}_1 x = r \text{ in } e]\rho\Phi\Phi_\kappa\mu &= \mathcal{E}[e]\rho[x \mapsto \mathcal{R}[r]\rho]\Phi\Phi_\kappa\mu \quad (\text{let1}) \\
{}^{72}\mathcal{E}[\text{letK } \kappa(p) = e_\kappa \text{ in } e]\rho\Phi\Phi_\kappa\mu &= \mathcal{E}[e]\rho\Phi\Phi_\kappa[\kappa \mapsto (p, e_\kappa)]\mu \quad (\text{letKont}) \\
\mathcal{E}[\text{let rec } f(p, k_f) = e_f \text{ in } e]\rho\Phi\Phi_\kappa\mu &= \mathcal{E}[e]\rho\Phi[f \mapsto (p, k_f, e_f)]\Phi_\kappa\mu \quad (\text{letRec}) \\
\mathcal{E}[\text{tail } \tau]\rho\Phi\Phi_\kappa\mu &= \mathcal{E}[\tau]\rho\Phi\Phi_\kappa\mu \quad (\text{tail})
\end{aligned}$$

Experienced readers should immediately notice some key properties based on the definition of this CPS IR, nevertheless, we list them as remarks below:

Remark (Non-first class continuations). refer to continuations appear only in specific syntactic positions, as listed below:

1. As the second argument to function calls: $f(p, k)$

⁷⁰where δ is the interpretation of primitive operations and π_i is projection.

⁷¹The second case handles the distinguished continuation parameter: applying it invokes the toplevel continuation μ , effectively returning from the current function.

⁷²Note that continuations are *not* values in the environment; they are stored in a separate continuation table Φ_κ , reflecting that they are not first-class.

2. As the target of continuation application: $\kappa(v)$
3. Bound by **letK**: **letK** $\kappa(p) = e$ **in** ...

This is enforced by the definition (abstract syntax) of CPS IR, where there is no CRhs that captures a continuation as a value. Some users might be disappointed as such IR could not support advanced control primitives such as `call/cc` or `shift/reset` from delimited continuation. Although such primitives can be more efficiently treated by the compiler, this form of CPS really is not widely used and can be substituted for more boilerplate code. Even If one must use it, we suggest an *internalized* version `Cont` which is more local with good maintainability⁷³ through the `Monad` interface, similar to the one from `Control.Monad.Cont` in `GHC`. Refer to `testcase examples/cont.tig` for an example.

Remark (Guaranteed tailcall). refers to the fact that all function calls in CPS are in tail position.

$\mathcal{E}[\![f(p, k)]\!]\dots$ directly invokes f without building stack frames

Non-tail calls in the source are transformed via **letK**:

$$\mathbf{let}_1 x = f(a) \mathbf{in} e \quad \rightsquigarrow \quad \mathbf{letK} \kappa(x) = \mathcal{C}^k \llbracket e \rrbracket \mathbf{in} \mathbf{tail} f(a, \kappa)$$

where the continuation κ captures the rest of the computation e .

Remark (Closure representation after CPS). Previously, we discussed the encoding and calling convention of function objects, this, is largely unchanged after CPS:

$\mathbf{C}[\textit{ptr}, \Gamma]$

where Γ is the captured environment. In CPS, function calls become:

$$\mathcal{E}\llbracket f(p, k) \rrbracket = \dots \text{ where } f \text{ is a code pointer}$$

The payload p is a pair (α, Γ) where α contains the user arguments.

Remark (CPS Module semantics). A CPS module consists of:

$$\begin{aligned} M &= (\bar{F}, F_{main}) \\ F &= (f, p, k, e) \end{aligned}$$

The semantics of a module is:

$$\mathcal{M}[(\bar{F}, F_{main})] = \mathcal{E}[e_{main}] \rho_0 \Phi_0 \oslash \text{id}_{\text{Ans}}$$

where:

$$\begin{aligned}\Phi_0 &= \{f_i \mapsto (p_i, k_i, e_i) \mid F_i \text{ of } (f_i, p_i, k_i, \cdot) \in \bar{F}\} \cup \{f_{main} \mapsto (p_m, k_m, e_m)\} \\ \rho_0 &= \{p_m \mapsto ((), \mathbf{E}[\![\!]\!])\}\end{aligned}$$

The initial payload is `(((),()),E $\square$ )`⁷⁴. Though, this behavior can be changed in the backend by modifying the driver wrapper code to pass whatever value to `main` in Common Lisp.

⁷³since CPS itself can be convoluted and counterintuitive of which the incautious use is just as bad as goto.

⁷⁴Recall, for function with parameter a of arity 1, we construct $(a, ())$ even if $a = ()$.

Before proceeding further, let us recap all the stages in the implementation of Tigris, through datatypes:

where ① : lexing, parsing ② : typechecking ③ : instance resolution, SysF elaboration
 ④ : *Lambda* IR lowering ⑤ : CPS conversion ⑥ : SBCL codegen

Informally, we describe the translation as follows:

⁷⁵as in 'simple-vector, the fastest stock array variant. Note that @x denotes the splice of x.

test-ctor i.e. `isCtor(x, tag, )` as `(eq (car x) tag)`. Note that arity is not checked at this time. Symbols appear inside CPS IR are escaped with bars `||` to preserve the case, as in

```
def sym (s : Name) : String :=
  let esc := s.foldl (init := "|") fun acc ch =>
    match ch with
    | '|' => acc ++ "\\|"
    | '\\ => acc ++ "\\\\"
    | _ => acc.push ch
  esc.push '|'
```

The caveats, as described in [section 4.1](#) (uncurrying), because type information is erased, we collect *shapes* on demand to *alleviate* this problem, where Shapes can be thought as degenerate types

```
inductive Shape where
  | unknown
  | pair
  | ctor (tag : Name) (arity : Nat)
  | fn -- function object inside value cell
deriving Inhabited, BEq, Repr
abbrev ShapeEnv := Std.HashMap Name Shape
```

where ShapeEnv associates a name with a shape. The initial shape of a name is always unknown, but will be *refined* as the code is being compiled. However we use the word *alleviate* because this method is not robust as it is local to functions and does not deal with the root cause: design flaws. Major refactor of adding type information to Lambda IR, threading through CPS transformation and codegen must be made to address the issue.

The runtime system refers to the primitive operator implementations, driver code and the definitions of low-level exceptions, e.g. the string equality `%string=`, the integer addition `%int+` and the MATCH-FAILURE condition `+NOMATCH+`⁷⁶ is implemented as

```
1 (declaim (inline %string=))
2 (defun %string= (s1 s2)
3   (declare (type simple-string s1 s2))
4   (the boolean (sb-kernel:%sp-string= s1 s2 0 nil 0 nil)))
5
6 (declaim (ftype (function (integer integer) integer) %int+) (inline %int+))
7 (defun %int+ (a b)
8   (declare (optimize (speed 3) (debug 0) (safety 0)))
9   (sb-kernel:two-arg+ a b))
10
11 (define-condition match-failure (error)
12   ((discriminant :initarg :discr :reader discriminant :type string))
13   (:report
14     (lambda (condition stream)
15       (format stream "No branch can be matched against ~A" (discriminant condition))))))
16 (defconstant +NOMATCH+ 'match-failure)
```

⁷⁶Common Lisp have *conditions*, which, is arguably more expressive and general than other languages' exception system as it encodes all kinds of control flow, not just exceptions. In fact, it can be used to implement an internalized continuation facility.

To compare against..	Use..
Objects/Structs	EQ
Symbols	EQ
NIL	EQ/NULL
T	EQ
Precise numbers	EQL
Floats	=
Char	EQL/CHAR=
List/Cons/Seq	EQUAL
String	EQUAL/EQUALP/STRING= (symbols as well)
Tree	TREE-EQUAL

Table 4: LISP equalities

Interested readers should refer to `runtime.lisp`.

Common Lisp have all kinds of equality relations. To minimize overhead on dynamically typed languages, we largely follow [Table 4](#). EQ is the most efficient as it compares memory address (i.e. physical equality rather than structural equality), which, is used most, in our case, such as comparing constructors as they are always unique, which has the same memory address. Compared to the usual encoding of constructors as integers, we achieve same performance here with better readability since the name of the constructor is the symbol itself.

Remark (Inlining runtime). The content of `runtime.lisp` is *inlined* in the compiler, through a trick used in the build script (also in Lean.) which detects whether there is a change in the file, and writes to `runtime.lean` if so:

```

1  input_file runtime.lisp where
2    path := __dir__ / "runtime.lisp"
3    text := true
4  target runtime.lean : Unit := do -- NOTE: see also `meta` (modifier)
5    let runtimep ← runtime.lisp.fetch
6    let rstrBegin := "r###\n".toUTF8
7    let rstrEnd   := "\n###".toUTF8
8    runtimep.mapM fun path =>
9      IO.FS.withFile path .read fun rt =>
10      IO.FS.withFile (__dir__ / "runtime.lean") .write fun h => do
11        let runtime ← rt.readBinToEndInto rstrBegin
12        h.putStrLn s!"/-- generated from {path} -/"
13        h.putStrLn "def runtime : String := "
14        h.write $ runtime.pushMany rstrEnd

```

FFI or *foreign function interface*, refers to calling functions external to the language. Since *Tigris* compiles to SBCL, a rather high and low-level⁷⁷ language, FFI is obtained for *free* if one is familiar with the calling convention described before as we do not have to deal with memory management or boxing/unboxing. Metaprogramming is used extensively to make the use of FFI as convenient as possible. Since *Tigris* is not able to see an externally defined function, it puts every trust it has in the user whom is responsible for

⁷⁷Common Lisp is high-level in having GC and being a dynamically typed functional language; and is low-level in having basic special operators and a bunch of low-level features such as `go`, `block`, `progn` that dates back to 1950s.

making sure the function is correctly typed, both in LISP and in the extern declaration.

However, schemes declared this way can not be checked for their *sanity*, i.e., it is possible to inhabit the types that should otherwise be empty, or types that can only be proved (inhabited) with circular argument such as $\forall\alpha. \alpha$ or $\forall\alpha, \beta. \alpha \rightarrow \beta$. Though, the former already exists because of the deprecated fixpoint combinator `rec`, as in `rec id $\alpha$` ,⁷⁸ whereas the latter could be useful, though unsafely, when specialized to $\forall\alpha. \text{Unit} \rightarrow \alpha$ (as seen in an example later).

We illustrate the use of FFI through a somewhat elaborate example:

```
extern println "%println" : forall a, a -> Unit
extern append "%string-append" : String -> String -> String
extern toString "%to-string" : forall a, a -> String
extern read "%read" : forall a, Unit -> a
infixl 65 "++" => append

let fact n =
  let rec go acc
    | 0 => acc
    | m =>
      let _ = println $ "fact "
                    ++ toString (n - m) ++ " = "
                    ++ toString (acc * m)
      in go an @@ m - 1
  in go 1 n
postfix 50 "!" => fact

let main =
  let _ = println "Enter a number:" in
  let x = read () in
  x ! -- x! is also a valid identifier. So we parenthesize it/use juxtaposition.
```

`read` is an unsafe function whose return type can be specialized to any type, in this case, `Int`. Internally, a name associated to a FFI declaration is transformed, during parsing, to its FFI name, e.g., `read` becomes `%read`. Where the implementations of these external functions are

```
(defforeign [%to-string] (x _) ;; e.g. toString : ∀a, a -> String
  (declare (ignore gamma _))
  (funcall k (princ-to-string x)))
(defforeign [%string-append] (x y) ;; e.g. string-append : String -> String -> String
  (declare (ignore gamma) (type string x y))
  (funcall k (concatenate 'string x y)))
(defforeign [%read] (_ _) ;; e.g. read (unsafe) : ∀a, Unit -> a
  (declare (ignore gamma _ _))
  (funcall k (read)))
(defforeign [%println] (x _) ;; e.g. println : ∀a, a -> Unit
  (declare (ignore gamma _))
  (princ x) (terpri)
  (funcall k nil))
```

⁷⁸while it damages the soundness of the type system, we keep it nonetheless for educational purposes. Besides, users can avoid it, since normal programming do not use `rec` at all.

Compiling and running the program results in⁷⁹

```

1  $ tigrisl examples/fact.tig --link ffi.lisp -o dist-fact.fasl && chmod +x dist-fact.fasl
2  ; wrote (...)/dist-fact.fasl
3  ; compilation finished in 0:00:00.021
4  $ ./dist-fact.fasl
5  Enter a number:
6  > 5
7  fact 0 = 5
8  fact 1 = 20
9  fact 2 = 60
10 fact 3 = 120
11 fact 4 = 120
12 120

```

For users with LISP experience, they will quickly notice the unfamiliar notation `defforeign`. This, together with other gadgets, are LISP macros to make the definition of foreign functions easier, which lives inside `ffi.lisp`. Amongst them, the most important are

- `WITH-ARGS` destructures payload = (cons α Γ) and binds each user argument to the symbols provided and a special variable `GAMMA` denoting the captured environment. Basically, it generates the prelude part of a function where payload are destructured according to the provided *lambda list*⁸⁰. This macro is not supposed to be used along; It is intended to be called by `defforeign` and users should use that instead. `with-args` is defined as

```

1  (defmacro with-args (payload arg-list &body body)
2    (let* ((alpha (gensym "ALPHA"))
3           (tmp   (gensym "TMP")))
4      (labels ((build-binds (args)
5                  (cond
6                    ((null args) '())
7                    ((null (cdr args)) `((, (car args) (car ,tmp))))
8                    (t
9                     (let ((head (car args))
10                          (rest (cdr args)))
11                       (append `((,head (car ,tmp))
12                                (,tmp (cdr ,tmp)))
13                               (if (null (cdr rest))
14                                   `((, (car rest) ,tmp))
15                                   (build-binds rest)))))))
16        `(let* ((,alpha (car ,payload))
17                (gamma (cdr ,payload))
18                (,tmp   ,alpha)
19                ,@(build-binds arg-list))
20              (declare (ignorable gamma))
21              ,@body))))

```

⁷⁹though `ffi.lisp` in this case is automatically linked. Linking it again with `--link` would make SBCL complain about duplicate definitions. `--link` here is only to demonstrate its use.

⁸⁰LISP jargon. Refers to a list that specifies a set of parameters (sometimes called lambda variables) and a protocol for receiving values for those parameters.

- DEFFOREIGN defines a tigris-callable foreign function. It is modeled after `defun` and combines `defparameter`/with-args. A foreign function definition takes the form of⁸¹

defforeign *function-name ffi-lambda-list {form}* → closure-object*

Aside from GAMMA, it additionally binds `k` i.e. the continuation, applying `k` can be thought as returning the value of the function as this is the canonical way to do so. `function-name`, or `name` in the implementation below, is the name of the foreign function, can be a string `"%println"` (preserves case), or a symbol `|%println|`. Users are highly recommended to escape the symbol (to preserve cases) as LISP by default does not distinguish cases but *Tigris* does. `defforeign` is implemented as

```

1  (defmacro defforeign (name arg-list &body body)
2    (flet ((name-to-string (x)
3              (etypecase x
4                (string x)
5                (symbol (symbol-name x)))))
6      (escape-bar (s)
7        (with-output-to-string (out)
8          (loop for c across s do
9            (case c
10              (#\| (write-string "\\|" out))
11              (#\\ (write-string "\\\\" out))
12              (t (write-char c out))))))
13      (make-closure (f)
14        `(cons C (vector
15                  (function ,f)
16                  (cons E (vector))))))
17      (let* ((ns (name-to-string name))
18             (fn-sym (intern (escape-bar ns)))
19             (cell (intern ns))
20             (clos (make-closure fn-sym)))
21        `(eval-when (:compile-toplevel :load-toplevel :execute)
22          (defun ,fn-sym (payload k)
23            (with-args payload ,arg-list ,@body))
24            (defparameter ,cell ,clos))))))

```

Note that the use of special operator `eval-when` is to make sure that any foreign functions defined in this way gets compiled and bound when `(load ..)`.

With these macros, defining foreign functions should be as easy as defining normal LISP functions.

caution! Any function output by the SBCL backend has the following compiler instructions:

```
(declare (optimize (speed 3) (safety 0) (debug 0)))
```

which removes most safety features of LISP, maximizing performance. Any error can result in a compromised LISP image (memory environment) with unreadable low-level error messages. It is best to remove this instruction when debugging.

⁸¹Here we use the same BNF syntax from X3J13/CLtL2. Unexperienced users may interpret it as: `(defforeign function-name (x y) forms)` that returns a closure object, where `(x y)` is `ffi-lambda-list` representing function parameters that may vary in length; `forms` refers to a list of forms that is the function body.

tigrisl is the CLI interface to interact with the compiler. **tigrisl** uses a simple hand-written recursive descent parser to parse command line arguments; Multiple input files are compiled in parallel with the same amount of threads launched. An invocation takes the form of

`tigrisl [flags] ifiles [-o ofiles]`

where *ifiles* and *ofiles* refers to a list of input and output files that should either

- matches one to one. As in, for all i , input file name f_i^{in} , there always exists a output file name f_i^{out} , vice versa.
- matches one to one, with residual *ifiles*. As in, for *ifiles* $f_1^{\text{in}}, \dots, f_i^{\text{in}}, f_{i+1}^{\text{in}}, \dots, f_n^{\text{in}}$ and *ofiles* $f_1^{\text{out}}, \dots, f_i^{\text{out}}$, we have $f_1^{\text{in}}, \dots, f_i^{\text{in}}$ matching $f_1^{\text{out}}, \dots, f_i^{\text{out}}$, with residual *ifiles* $f_{i+1}^{\text{in}}, f_n^{\text{in}}$ using default output naming convention i.e. appending with `.lisp`.
- any other situation results in a exception.

...and notable flags are

- `--lam`, emit raw *Lambda* IR with optimizations;
- `--cc`, emit CC'd *Lambda* IR;
- `--cps`, emit CPS IR;
- `--sysf`, emit System F IR; These four flags can be used together.
- `-nl`, `--no-lisp`, do not emit Common Lisp source.
- `-ne`, `--no-entry`, do not generate entrypoint, where entrypoint is also called driver code, that is the following:

```
(defun |__start| ()
  (format nil "~A"
    (funcall #'|main| nil #'identity)))
```

- `--legacy`, use deprecated typechecker/IR/backend. Not recommended;
- `-l`, `--link <path>`, link additional lisp source from `<path>`, can be used multiple times. Default behavior: link `ffi.lean`. Note that, internally, linking a file *f* adds a `(load f)` to the top of the object code. The order of multiple `--links` is preserved here.
- `-o <ofiles>`, *positional vararg*, anything beyond this point is considered a part of `-o`. `<ofiles>` has file extension-directed output where `*.fasl` generate FASL executables and anything else for LISP sources. FASL generation is done by concatenating every linked LISP files together with the generated object code, then feed them into the launched SBCL with *block compilation* on to generate a monolithic binary. This binary can usually run directly in shell. If that is not the case, users may also use `sbcl --script <fasl>` to run fasl files.
- `--fasl`, emit FASL, applies to all input files.

5 PP: The *dependently-typed* table printer

What's the fun when there is *Tigrisd*, the theorem prover counterpart of *Tigris*, but the implementation of them doesn't involve some dependent type practices – Actually, both CLI interfaces and REPLs use a dependently-typed table printer that lives inside the PP module, not to mention the termination proofs of some recursive functions.

The author, and this project advocates for the use of dependent types in programming and below, we'll discuss the design and implementation of this simple printer, hopefully, to provide some insights into how dependent types can help users catch more unexpected errors at compile-time, rather than runtime.

We wish the types of table are distinguished by their headers, say, the header (name, gender, age) should be different from (product name, manufacturer, serial number, price). To achieve this at compile-time, we shall rely on the typechecker. Suppose the header is a list, we may turn it into a *dynamically-sized*⁸² product *type* whose length and component type must match for the product types to be unified (Text.SString is stylized string):

```
abbrev ToProd (as : List  $\alpha$ ) (motive : Type -> Type) : Type :=
  match as with
  | [] => PUnit
  | [_] => motive  $\alpha$ 
  | _ :: x :: xs =>
    motive  $\alpha$   $\times$  ToProd (x :: xs) motive
```

```
variable (header : List Text.SString)
```

```
abbrev Row := ToProd header id
```

The above reads: $\text{ToProd } (as : \text{List } \alpha) f$ can be eliminated to one of 1) PUnit ⁸³, 2) $f \alpha$, or 3) $f \alpha \times \text{ToProd } (\text{cdr } as) f$, based on the *shape* of as :

1. where as is empty;
2. where as is a *singleton*;
3. where as contains *two or more* elements.

This effectively tells us that ToProd can *eliminate* to one of the three⁸⁴ types depending on the value of as , specifically, its shape. Readers should pay great attention to this approach, or thought process, as the pattern matching used here is interpreted differently⁸⁵ (more of a *recursor*-based interpretation),⁸⁶ which is mirrored later.

Note that *motive* is type theory jargon. It represents the target type (or a part of it) we want it to be eliminated to. Informally, it can be best compared to the function used when mapping some data structures. Also, the use of **abbrev** rather than **def** here guarantees the definition to be *reducible*, not *semireducible* or *irreducible*, allowing the automatic unfolding of the definition when using tactics such as `rfl`, short for reflexivity.

Now, we may encode an entry (a *Row*) of a table as ToProd header id . This way, we require the entry of some table to match the shape of its header, otherwise it is a type error; And the table type itself, obviously, can be defined as

```
abbrev TableOf := Array $ Row header
```

We use `Array` here because it is overridden in the runtime with *real continuous data structures* rather than a linked list `List`. Besides, it also makes sense as we want *the shape* of the entry to match its header, but there should not be any size restriction when it comes to the size of a table, i.e. the numbers of entry a table has.

⁸²In the sense that its size depends on the length of the header.

⁸³*universe-polymorphic* version of `Unit`. Though since the result lives in $\text{Type} \equiv \text{Type } 0 \equiv \text{Sort } 1$, it is effectively `Unit`.

⁸⁴To be precise, it is $2 + n$ for as of length n .

⁸⁵different than a *operational* interpretation, in the sense of regular programming.

⁸⁶And it is indeed in the form of recursor invocations inside the Lean *kernel*.

<pre> <ofiles> a list of output files written to file extension-directed output: - '*.fasl' for FASL output, regardless of --fasl - anything else for LISP output </pre>	<pre> (.str "<ofiles>", .str "a list of output files written to" .str "", .by1 "file extension-directed output:") (.str "", .str " - '*.fasl' for FASL output, regardless of" .str "", .str " - anything else for LISP output") </pre>
---	--

Figure 7: output of the table printer and its equivalent table entry

Furthermore, since we are making a *printer*, we want the output to look good, or at least, looks like a table. Here we define

```

inductive Alignment | left | center | right
inductive PadsBy | perCell | perCol
abbrev Align := ToProd header λ _ ⇒ Alignment
abbrev OverrideWidth := ToProd header λ _ ⇒ Option Nat

```

where

- Align is a tuple of the same shape as a header, denoting each column's alignment;
- PadsBy is the padding strategy. `perCell` makes every cell to have the same size as the largest one in them. This is rarely useful, but may look good if the table isn't too large; `perCol` is the one that is actually useful and more familiar to users since it applies the same strategy but local to each column.
- `OverrideWidth`, despite its name, stores the calculated width of each column. It is named `OverrideWidth` because it can be overridden (*before* the calculation) to have more manual control over the output, providing better granularity.

Based on this, we sketch the configuration of this printer as

```

structure PPSpec (header : List Text.SString) where
  align      : Align header
  width      : OverrideWidth header := 0
  header?    : Bool := true
  margin     : Nat := 3
  padsBy     : PadsBy := .perCol
  truncate   : Bool := false

```

where `margin` is *sum* of a cell's left and right margin. And `truncate`, which is a flag that toggles the skip of a cell if it is empty i.e. gets replaced by the cell to its right (also respects margin); Despite its simple nature, this is actually more interesting than one might imagine, as this could be used to create indentation, together with `margin`, to achieve some interesting effects, such as simulating tool tips, which is most useful when printing command helppages as shown in [Figure 7](#).

Now, we can actually implement this printer by first defining some helper functions that calculates the width of every column: [Figure 8](#). This is where the aforementioned thought process come into use. The type `ToProd`, can somewhat be interpreted as having multiple values (though this “value” are *types*), as it *depends* on its `List` argument, though, it remains as one type at a moment because it is statically typed. We illustrate the usage of this thought process by introducing a concept called *discriminant refinement*.

Discriminant refinement is, a jargon, though fairly straightforward, stating that the type information of a match discriminant can be refined based on the pattern matching against it. An example being `match x with | 0 => ...`⁸⁷, where Lean knows that within this branch, `x` must be `0`, or as a

⁸⁷Though, in non-dependent type context, this feature may need to be enabled manually, in the form of `match h : x with ..`, by prefixing `x` with a name `h`, for binding the hypotheses in each branch in order to for them be picked up by the typechecker.

hypothesis that becomes part of the *ambient types*: $h : x = 0$. – A very reasonable, and true take. This technique, is used fairly a lot in programming with dependent types, as is in this case: we *guide* Lean the underlying type of `ToProd` through patterns against `header`:

1. a `[]`, stating $header = []$, refines `ToProd` to `PUnit`, as stated in its definition;
2. a `[_]`, stating $header = [·]$, a singleton, or, a `List` with one element of type α , refines `ToProd` to $f\alpha$, as stated in its definition;
3. an Erlang-like `%[_ , _ | _]`, which is syntactic sugar for $· :: _ :: _$, stating $header = · :: (· :: ·)$, a list with two or more elements, refines `ToProd` to a dynamically-sized product.

Where f is the motive, in this case, the constant function that maps all types to `Option Nat`. The point of these seemingly useless patterns differs from the regular purpose of providing templates for destructuring; Instead, it is to provide additional information to the typechecker, which, if not omitted, results in a type error as Lean cannot deduce the type of `header` on its own. It is the user's responsibility to do case-by-case analysis just as in proofs. From the outsider's perspective, it is as simple as to mirror the cases in the type definition.

Remark (Common usage of discriminant refinement). Furthermore, discriminant refinement deals with surrounding environment. `foldr1` on `List` usually does not permit empty lists, applying to such is a runtime error in Haskell, for example. In Lean, we simply make this requirement an assumption and defines the function as:

```
def foldr1 (f :  $\alpha \rightarrow \alpha \rightarrow \alpha$ ) (as : List  $\alpha$ ) (h : as  $\neq []$ ) :  $\alpha$  :=
  match as with
  | []  $\Rightarrow$  h rfl >.elim -- or simply remove this branch.
  | [x]  $\Rightarrow$  x
  | x :: y :: xs  $\Rightarrow$  f x (foldr1 f (y :: xs) nofun)
```

where h is refined to $[] \neq []$ in the empty branch where we manually *proved false* and used *ex falso* to match the expected type. Though, this branch can simply be removed as Lean can automatically proves it false. Similiarly `nofun` is a shorthand used here to prove $y :: xs \neq []$, but the proof itself automatically handled by Lean. Users calling this function must prove that as is nonempty through actual proofs or simply introducing a hypothesis using `if h : as  $\neq []$  then ...`. Though, we shall stop here as this is beyond the scope of this text, which is not a tutorial.

Getting back to implementing table printer, we found ourself having several polymorphic helpers in the sense that they are independent of `header` and we may use it to implement some operations like `toString`, $(\cdot \cup \cdot)$, `max` and `0` where it denotes the zero element of some algebraic structure. From there, implementing printers becomes trivial.

This concludes the description of *Tigris*.

```

def ovToStr (ov : OverrideWidth header)
  : String := match header with
  | [] => "" | [_] => toString ov
  | %[_ , _ | _] => toString ov.1 ++ ovToStr ov.2

def zeroOverride (header : List Text.SString)
  : OverrideWidth header := match header with
  | [] => () | [_] => none
  | %[_ , y | ys] => (none, zeroOverride (y :: ys))

def maxN (h1 h2 : OverrideWidth header)
  : OverrideWidth header := match header with
  | [] => () | [_] => max h1 h2
  | %[_ , _ | _] => (max h1.1 h2.1, maxN h1.2 h2.2)

def merge (h1 h2 : OverrideWidth header)
  : OverrideWidth header := match header with
  | [] => () | [_] => max h1 h2
  | %[_ , _ | _] => (h2.1 <▷ h1.1, merge h1.2 h2.2)

instance : ToString $ OverrideWidth header := <ovToStr>
instance : Union $ OverrideWidth header := <merge>
instance : Max $ OverrideWidth header := <maxN>
instance : Zero $ OverrideWidth header := <zeroOverride header>

```

Figure 8: Helper functions used to calculate column width

6 *Tigrisd*: a lightweight theorem prover

Tigrisd is a experimental dependently-typed theorem prover that demonstrates clear, modular and exemplary theorem prover design. *Tigrisd*, together with *Tigris*, is a bundled solution serving educational purposes that hopes to become useful in programming language/theorem proving education. *Tigrisd* is deeply influenced and takes much inspiration in terms of design/implementation from [pi-forall](https://github.com/sweirich/pi-forall)⁸⁸

While currently *Tigrisd* is written in Haskell, because of the latter’s strong and vibrant ecosystem with libraries like Unbound providing useful methods to manipulate the lambda calculus and the AST so that we do not have to reinvent the wheel.⁸⁹ Though, we may someday rewrite it in Lean. The point being that at least there is a working prototype.

Nomenclature The word *term*, when used along, includes what would be considered a *type* in regular programming languages. When *term* and *type* are used together, sometimes the former includes latter, sometimes it does not, readers should be able to deduce the exact meaning of it; Unconditionally, *type* is used exclusively to reference types, no matter dependent or not.

6.1 The basics

The syntax of *Tigrisd* is very similar to *Tigris*, with a few exceptions, especially in relation to its dependently-typed nature. The implementation of this syntax also uses monadic parsing, in the form of parser combinators, specifically, the *layout* combinators, which use off-side rules (i.e. indentation aware). This decision is made this way because proofs are generally more verbose than programs. Some of the connectives can be omit if we use indentation-aware parsing, which, is more closer to a human-readable form. We list some notable syntax changes below.

Indexed family has the form

```

<dtype-decl> ::= inductive <dtype-name> <dtype-ibinder>* : <dtype-univ>
               "where"? ("|" <dtype-branch>)*
<dtype-branch> ::= <dtype-ctor> : sepBy1(">", <dtype-binder>)

```

⁸⁸<https://github.com/sweirich/pi-forall>

⁸⁹as we’ve sort of did so in *Tigris*, but it is all inlined and not as a standalone package, which, can be difficult to use here.

```

<dtype-ctor>      ::= <upper-ident>
<dtype-name>      ::= <upper-ident>
<dtype-binder>     ::= <dtype-tbinder> | <dtype-ibinder>
<dtype-tbinder>    ::= (<ident> : <term>)
<dtype-ibinder>    ::= {<ident> : <term>}
<dtype-univ>      ::= Type

```

An concrete index family example in *Tigrisd* is

```

inductive Vector (α : Type) (n : Nat) : Type where
| Nil : Vector α 0
| Cons : {m : Nat} -> α -> Vector α (Succ m)

```

With type irrelevant binders wrapped in curly brackets $\{\}$ i.e. not used in programming. A constructor's result type is checked against the type that is being defined to see if their *head* matches, in the case, they must be an application of `Vector`. Implicitly, two constraints are produced by this inductive family, identified by the variant⁹⁰:

- in the `Nil` branch, n must equate to `Zero`;
- in the `Cons` branch, n must equate to `Succ m` where m is bound to the first field of `Cons`.

This example defines a `List` that encodes its length in the type, similar to the `Vector` in `Lean`. We can verify this with unnamed bindings, defined with **example**:

```

example: {α : Type} -> Vector α 0 = Nil
example: Vector Unit 2 = Cons 1 () (Cons 0 () Nil)

```

function declaration now takes the form of:

```

<tm-lam> ::= fun <dtype-binder>+ => <tm>
<tm-let> ::= let <ident> <dtype-binder>* : <tm> => <tm>

```

The `let`-binding syntactic sugar described in *Tigris* applies here:

```

let map {α : Type} {β : Type} {n : Nat} (f : α -> β) (as : Vector α n) : Vector β n =
  match as with
  | Nil      => Nil
  | Cons m x xs => Cons m (f x) (map α β m f xs)

```

Note that currently *Tigrisd* does not perform implicit inference or implicit synthesis, which requires users to provide every relevant binders. *Tigrisd* implements a simple bidirectional typechecker with limited inference ability.

Readers should now pay great attention to this new property:

function takes types just the same as taking values, since types are terms as well.

Which, yields some interesting rules that previously, as in *Tigris*, requires extensions, such as skolem variables. Consider replacing the recursive call of `map` with `xs`, i.e. `Cons m (f x) xs`, which, immediately yields a type error because α and β are distinctive parameters. The typechecker cannot establish that `Vector α n` equals `Vector β n`, unless the equality $h : \alpha = \beta$ ⁹¹ is proven, only after which said types can be proved equal through α -equivalence; In this case, it is not and it can't be, as α and β are arbitrary types and there is not any assumption that entails h .

⁹⁰The word *variant* here is used literally, which, has no connection to variant values.

⁹¹Equality itself is a big topic. In this case, we are talking about definitional equality. Detailed information on this is described later.

pattern matching and product elimination has been reduced to the primitive form

- `match e with ..` only accepts the variant pattern;
- `let (x, y) = a in b` is exclusively used for product (pair) elimination.

Tigrisd implements a lightweight version of discriminant refinement as mentioned above, we shall describe this later.

Type/Proof Irrelevance refers to the absent of proofs and types as they have no runtime representation and thus are erased. Therefore, it is crucial for the typechecker to make sure there is no unexpected erasure of meaningful values. *Irrelevance* is the general word that describe this rule, as it may apply to not just types or proofs (see below).

modules are fancy names for files. A *Tigrisd* source file automatically constitutes of a module, with the module name being its filename. Modules $M_1, \dots, M_i$ can be imported using the directive `import M_1, .., M_i`; which may only appear at the top of a file.

Builtin commands are either proofs or commands used in interactive sessions (e.g. REPL), they are

- `sorry` (proof): can prove anything, which is obviously unsound. In practice, it is used to stub out the proofs that is to be typed out later. Any use of this command results a warning from the typechecker, with every definition depending on it being marked irrelevant.
- `show` (cmd): is used to pretty print the typing context.
- `rfl` (proof): is short for reflexivity, or $a = a$, for some term a . Some might say this is the root of all equality proofs as one is essentially substituting one side of the equality that eventually becomes the other side, which then can be *discharged* with `rfl`. `rfl` is more nuanced than it seems: It respects α -equivalence and β -equivalence.
- `exfalso tm` (proof): or *ex falso quodlibet* (EFQ), is the famous principle of explosion. `exfalso` expects a term proved false, which then can be used to prove anything, or “construct” a value of any type. The system becomes unsound if some term can be proved false without assumptions.

Universe , or level, is 1 in *Tigrisd*, which is also *impredicative*, in the sense that **Type** : **Type**, instead of **Type** 1, reading **Type** lives in itself, or the **Type** of types is still **Type**. Although 1-level MLTT is unsound, as proven by Girard⁹², we adopt it nonetheless for its simplicity. Similarly, bounded recursion, termination checking or totality checking is nonexistent in *Tigrisd*.

6.2 A formal description of *Tigrisd*

Unlike *Tigris*, which can be considered *stable*⁹³, while *Tigrisd* already have a working prototype with desired features, huge refactors and design changes may still happen, such as rewriting it in Lean, implementing *implicit argument synthesis* to reduce the verbosity of the proofs, and more. Because of this, we omit the implementation notes here and instead jump straight to the formal description of *Tigrisd*, where the core calculus and the type system is most unlikely to change.

⁹²We do not show why it is unsound, as that is mathematicians’ job. Readers only have to know that the reason is similar to why ZF sets need an axiom of regularity; Otherwise there could be a Russell-style paradox.

⁹³Readers must acknowledge that “stable” here means that the chance to have a code refactoring, or any major change to the design of the language is quite unlikely. Not that *Tigris* self is stable: it is absolutely not and is not suitable for production environment.

Core calculus & context is given by

tm, a, b, A, B	$::=$	terms
	Type	universe
	x	
	$\lambda x. a$	lambda abstraction
	$a b$	application
	$(x:A) \rightarrow B_x$	Π /product
	$(a : A)$	ascription
	sorry	stub
	show	show ctx
	$(x : A) \times B_x$	Σ /coproduct
	(a, b)	products
	let $(x, y) = a$ in b	pair elim
	$a = b$	Eq
	rfl	reflexivity
	$b \triangleright a$	cast
	exfalse a	ex falso
	$\lambda\{x\}. a$	irr lambda
	$a \{b\}$	irr app
	$\{x:A\} \rightarrow B_x$	irr Pi
	let $x = a$ in b	let

$context, \Gamma$	$::=$	contexts
	$\emptyset$	empty
	$\Gamma, x:A$	cons
	$x:A$	singleton
	$\Gamma, x:^e A$	irr cons
	$x:^e A$	irr single
	Γ^e	demote
	$\Gamma, x = a$	cons eq hypo

where the construction of context and typing is given by

Γ (Context construction)

$$\frac{\text{G-NIL}}{\vdash \emptyset} \qquad \frac{\text{G-CONS} \quad \Gamma \vdash A : \text{Type} \quad \vdash \Gamma}{\vdash \Gamma, x:A}$$

$\Gamma \vdash a : A$ (Typing)

$$\begin{array}{c} \text{T-VAR} \\ \frac{x:A \in \Gamma}{\Gamma \vdash x:A} \end{array} \quad \begin{array}{c} \text{T-LAMBDA} \\ \frac{\Gamma, x:A \vdash a : B[x' \mapsto x]}{\Gamma \vdash \lambda x. a : (x':A) \rightarrow B_{x'}} \end{array} \quad \begin{array}{c} \text{T-IMPRED-UNIV} \\ \frac{}{\Gamma \vdash \text{Type} : \text{Type}} \end{array} \quad \begin{array}{c} \text{T-APP} \\ \frac{\Gamma \vdash a : (x:A) \rightarrow B_x \quad \Gamma \vdash b : A}{\Gamma \vdash a b : B[x \mapsto b]} \end{array}$$

$$\begin{array}{c} \text{T-PI} \\ \frac{\Gamma \vdash A : \text{Type} \quad \Gamma, x:A \vdash B : \text{Type}}{\Gamma \vdash (x:A) \rightarrow B_x : \text{Type}} \end{array} \quad \begin{array}{c} \text{T-SIGMA} \\ \frac{\Gamma \vdash A : \text{Type} \quad \Gamma, x:A \vdash B : \text{Type}}{\Gamma \vdash (x:A) \times B_x : \text{Type}} \end{array} \quad \begin{array}{c} \text{T-CAST} \\ \frac{\Gamma \vdash a : A \quad \Gamma \vdash A = B}{\Gamma \vdash a : B} \end{array}$$

$\frac{\text{T-PAIR} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : B[x \mapsto a]}{\Gamma \vdash (a, b) : (x : A) \times B_x}$	$\frac{\text{T-LETPAIR} \quad \Gamma \vdash a : (x : A_1) \times A_{2x} \quad \Gamma, x : A_1, y : A_2 \vdash b : B \quad \Gamma \vdash B : \text{Type}}{\Gamma \vdash \text{let } (x, y) = a \text{ in } b : B}$	$\frac{\text{T-LET} \quad \Gamma \vdash a : A \quad \Gamma, x : A, x = a \vdash b : B \quad \Gamma \vdash B : \text{Type}}{\Gamma \vdash \text{let } x = a \text{ in } b : B}$
$\frac{\text{T-EQ} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash a = b : \text{Type}}$	$\frac{\text{T-REFL} \quad \Gamma \vdash a = b}{\Gamma \vdash a = b : \text{Type}} \quad \Gamma \vdash \mathbf{rfl} : a = b$	$\frac{\text{T-SUBST} \quad \Gamma \vdash a : A[x \mapsto a_1][y \mapsto \mathbf{rfl}] \quad \Gamma \vdash b : a_1 = a_2}{\Gamma \vdash b : A[x \mapsto a_2][y \mapsto b] \triangleright a}$
	$\frac{\text{T-EXFALSO} \quad \Gamma \vdash A : \text{Type} \quad \Gamma \vdash a : \text{True} = \text{False}}{\Gamma \vdash \mathbf{exfalse} a : A}$	

Readers should note that some concrete types such as `Bool` and `Unit`, is omitted. These types basically the same rules as in *Tigris*. With irrelevant being the same, in terms of typing:

$\boxed{\Gamma \vdash a : A}$			(Irrelevant Typing)
$\frac{\text{T-EVAR} \quad x :^{\text{Rel}} A \in \Gamma}{\Gamma \vdash x : A}$	$\frac{\text{T-ELAMBDA} \quad \Gamma, x :^{\text{Irr}} A \vdash a : B \quad \Gamma^{\text{Rel}} \vdash A : \text{Type}}{\Gamma \vdash \lambda\{x\}. a : \{x : A\} \rightarrow B_x}$	$\frac{\text{T-EAPP} \quad \Gamma \vdash a : \{x : A\} \rightarrow B_x \quad \Gamma^{\text{Rel}} \vdash b : A}{\Gamma \vdash a \{b\} : B[x \mapsto b]}$	$\frac{\text{T-EPI} \quad \Gamma \vdash A : \text{Type} \quad \Gamma, x :^{\text{Rel}} A \vdash B : \text{Type}}{\Gamma \vdash \{x : A\} \rightarrow B_x : \text{Type}}$

The bidirectional type system *Tigrisd* implementation is (check/infer)

$\boxed{\Gamma \vdash a \Leftarrow B}$			(type checking)
$\frac{\text{C-LAMBDA} \quad \Gamma, x : A \vdash a \Leftarrow B}{\Gamma \vdash \lambda x. a \Leftarrow (x : A) \rightarrow B_x}$	$\frac{\text{C-SWITCH-MODE} \quad a \text{ not applicable} \quad \Gamma \vdash a \Rightarrow A \quad \Gamma \vdash A \xRightarrow{\text{impl}} B}{\Gamma \vdash a \Leftarrow B}$	$\frac{\text{C-WHNF} \quad A \notin \mathbf{WHNF} \quad \Gamma \vdash a \Leftarrow nf}{\Gamma \vdash \mathbf{WHNF} A \rightsquigarrow nf}$	
$\frac{\text{C-PAIR} \quad \Gamma \vdash a \Leftarrow A \quad \Gamma \vdash b \Leftarrow B[x \mapsto a]}{\Gamma \vdash (a, b) \Leftarrow (x : A) \times B_x}$	$\frac{\text{C-LETPAIR-RL} \quad \Gamma \vdash z \Rightarrow (x : A_1) \times A_{2x} \quad \Gamma, x : A_1, y : A_2 \vdash b \Leftarrow B[z \mapsto (x, y)]}{\Gamma \vdash \text{let } (x, y) = z \text{ in } b \Leftarrow B}$		
$\frac{\text{C-LETPAIR-LR} \quad \Gamma \vdash z \Rightarrow (x : A_1) \times A_{2x} \quad \Gamma, x : A_1, y : A_2, z = (x, y) \vdash b \Leftarrow B}{\Gamma \vdash \text{let } (x, y) = z \text{ in } b \Leftarrow B}$	$\frac{\text{C-LET} \quad \Gamma \vdash a \Rightarrow A \quad \Gamma, x : A, x = a \vdash b \Leftarrow B}{\Gamma \vdash \text{let } x = a \text{ in } b \Leftarrow B}$	$\frac{\text{C-REFL} \quad \Gamma \vdash a \xRightarrow{\text{impl}} b}{\Gamma \vdash \mathbf{rfl} \Leftarrow a = b}$	
$\frac{\text{C-SUBST-L} \quad \Gamma \vdash y \Rightarrow B \quad \Gamma \vdash \mathbf{WHNF} B \rightsquigarrow (x = a_2) \quad \Gamma \vdash a \Leftarrow A[x \mapsto a_2][y \mapsto \mathbf{rfl}]}{\Gamma \vdash y \Leftarrow A \triangleright a}$	$\frac{\text{C-SUBST-R} \quad \Gamma \vdash y \Rightarrow B \quad \Gamma \vdash \mathbf{WHNF} B \rightsquigarrow (a_1 = x) \quad \Gamma \vdash a \Leftarrow A[x \mapsto a_1][y \mapsto \mathbf{rfl}]}{\Gamma \vdash y \Leftarrow A \triangleright a}$	$\frac{\text{C-EXFALSO} \quad \Gamma \vdash a \Rightarrow A \quad \Gamma \vdash \mathbf{WHNF} A \rightsquigarrow (a_1 = a_2) \quad \Gamma \vdash \mathbf{WHNF} a_1 \rightsquigarrow \text{True} \quad \Gamma \vdash \mathbf{WHNF} a_2 \rightsquigarrow \text{False}}{\Gamma \vdash \mathbf{exfalse} a \Leftarrow B}$	

$\boxed{\Gamma \vdash a \Rightarrow A}$			(type inference)
$\frac{\text{I-VAR} \quad x : A \in \Gamma}{\Gamma \vdash x \Rightarrow A}$	$\frac{\text{I-APP-NONWHNF} \quad \Gamma \vdash a \Rightarrow (x : A) \rightarrow B_x \quad \Gamma \vdash b \Leftarrow A}{\Gamma \vdash a \ b \Rightarrow B[x \mapsto b]}$	$\frac{\text{I-APP} \quad \Gamma \vdash a \Rightarrow A \quad \Gamma \vdash \mathbf{WHNF} \ A \rightsquigarrow (x : A_1) \rightarrow B_x \quad \Gamma \vdash b \Leftarrow A_1}{\Gamma \vdash a \ b \Rightarrow B[x \mapsto b]}$	
$\frac{\text{I-PI} \quad \Gamma \vdash A \Leftarrow \mathbf{Type} \quad \Gamma, x : A \vdash B \Leftarrow \mathbf{Type}}{\Gamma \vdash (x : A) \rightarrow B_x \Rightarrow \mathbf{Type}}$	$\frac{\text{I-IMPRED-UNIV}}{\Gamma \vdash \mathbf{Type} \Rightarrow \mathbf{Type}}$	$\frac{\text{I-ASCRIBE} \quad \Gamma \vdash A \Leftarrow \mathbf{Type} \quad \Gamma \vdash a \Leftarrow A}{\Gamma \vdash (a : A) \Rightarrow A}$	$\frac{\text{I-SIGMA} \quad \Gamma \vdash A \Leftarrow \mathbf{Type} \quad \Gamma, x : A \vdash B \Leftarrow \mathbf{Type}}{\Gamma \vdash (x : A) \times B_x \Rightarrow \mathbf{Type}}$
$\frac{\text{I-LET} \quad \Gamma \vdash a \Rightarrow A \quad \Gamma, x : A, x = a \vdash b \Rightarrow B}{\Gamma \vdash \mathbf{let} \ x = a \ \mathbf{in} \ b \Rightarrow B[x \mapsto a]}$	$\frac{\text{I-EQ} \quad \Gamma \vdash a \Rightarrow A \quad \Gamma \vdash b \Rightarrow B \quad \Gamma \vdash A \xRightarrow{\text{impl}} B}{\Gamma \vdash (a = b) \Rightarrow \mathbf{Type}}$	$\frac{\text{I-SUBST} \quad \Gamma \vdash b \Rightarrow B \quad \Gamma \vdash \mathbf{WHNF} \ B \rightsquigarrow (a_1 = a_2) \quad \Gamma \vdash a \Rightarrow A}{\Gamma \vdash b \Rightarrow A \triangleright a}$	

Readers paying attention will notice that some inference rules use definitional (also judgemental in MLTT, and typal in 1-level TT) equality, such as rules **C-SWITCH-MODE**, **I-APP**, and **C-WHNF**. Note that these rules only specifies *what it means to be “equal”*, They are not computational rules nor used for implementation purposes. We define the equality as follows:

$\Gamma \vdash A = B$		<i>(Defintional/Judgemental/Typal equality)</i>		
$\frac{\text{E-REFL}}{\Gamma \vdash A = A}$	$\frac{\text{E-SYM} \quad \Gamma \vdash A = B}{\Gamma \vdash B = A}$	$\frac{\text{E-TRANS} \quad \Gamma \vdash A_1 = A_2 \quad \Gamma \vdash A_2 = A_3}{\Gamma \vdash A_1 = A_3}$	$\frac{\text{E-ASCRIBE} \quad \Gamma \vdash a_1 = a_2}{\Gamma \vdash (a_1 : A) = a_2}$	$\frac{\text{E-SUBST-CONGR} \quad \Gamma \vdash a_1 = a_2}{\Gamma \vdash b[x \mapsto a_1] = b[x \mapsto a_2]}$
$\frac{\text{E-BETA}}{\Gamma \vdash (\lambda x. a) b = a[x \mapsto b]}$		$\frac{\text{E-PI-CONGR} \quad \Gamma \vdash A_1 = A_2 \quad \Gamma, x : A_1 \vdash B_1 = B_2}{\Gamma \vdash (x : A_1) \rightarrow B_{1x} = (x : A_2) \rightarrow B_{2x}}$		
$\frac{\text{E-SIGMA-CONGR} \quad \Gamma \vdash A_1 = A_2 \quad \Gamma, x : A_1 \vdash B_1 = B_2}{\Gamma \vdash (x : A_1) \times B_{1x} = (x : A_2) \times B_{2x}}$		$\frac{\text{E-APP-CONGR} \quad \Gamma \vdash a_1 = a_2 \quad \Gamma \vdash b_1 = b_2}{\Gamma \vdash a_1 b_1 = a_2 b_2}$	$\frac{\text{E-LAM-CONGR} \quad \Gamma, x : A_1 \vdash b_1 = b_2}{\Gamma \vdash \lambda x. b_1 = \lambda x. b_2}$	
$\frac{\text{E-PAIR-ELIM}}{\Gamma \vdash \mathbf{let} (x, y) = (a_1, a_2) \mathbf{in} b = b[x \mapsto a_1][y \mapsto a_2]}$		$\frac{\text{E-PAIR-CONGR} \quad \Gamma \vdash a = a' \quad \Gamma \vdash b = b'}{\Gamma \vdash \mathbf{let} (x, y) = a \mathbf{in} b = \mathbf{let} (x, y) = a' \mathbf{in} b'}$		
$\frac{\text{E-VAR} \quad x = a \in \Gamma}{\Gamma \vdash x = a}$	$\frac{\text{E-LET-BETA}}{\Gamma \vdash \mathbf{let} x = a \mathbf{in} b = b[x \mapsto a]}$	$\frac{\text{E-LET} \quad \Gamma \vdash a_1 = a_2 \quad \Gamma, x : A, x = a_1 \vdash b_1 = b_2}{\Gamma \vdash \mathbf{let} x = a_1 \mathbf{in} b_1 = \mathbf{let} x = a_2 \mathbf{in} b_2}$		
$\frac{\text{E-TYPALSUBST-BETA}}{\Gamma \vdash \mathbf{rfl} \triangleright a = a}$		$\frac{\text{E-TYPALSUBST-CONGR} \quad \Gamma \vdash a = a' \quad \Gamma \vdash b = b'}{\Gamma \vdash b \triangleright a = b' \triangleright a'}$		$\frac{\text{E-TYPALEQ-CONGR} \quad \Gamma \vdash a_1 = b_1 \quad \Gamma \vdash a_2 = b_2}{\Gamma \vdash (a_1 = a_2) = (b_1 = b_2)}$

$$\begin{array}{c}
\text{E-EXFALSO-CONGR} \\
\frac{\Gamma \vdash a = a'}{\Gamma \vdash \mathbf{exfalse} \ a = \mathbf{exfalse} \ a'} \\
\\
\text{E-EAPP-CONGR} \\
\frac{\Gamma \vdash a_1 = a_2}{\Gamma \vdash a_1 \ \{b_1\} = a_2 \ \{b_2\}} \\
\\
\text{E-ELAM-CONGR} \\
\frac{\Gamma, x: \mathbf{!}^{\text{irr}} A \vdash a_1 = a_2}{\Gamma \vdash \lambda\{x\}. a_1 = \lambda\{x\}. a_2} \\
\\
\text{E-EPI-CONGR} \\
\frac{\Gamma \vdash A_1 = A_2 \quad \Gamma, x: \mathbf{Rel} \ A_1 \vdash B_1 = B_2}{\Gamma \vdash \{x: A_1\} \rightarrow B_{1x} = \{x: A_2\} \rightarrow B_{2x}}
\end{array}$$

Weak Head Normal Form or the judgement $\Gamma \vdash \mathbf{WHNF} \ tm \rightsquigarrow nf$ appeared above reads: under context Γ , tm reduces to nf , which is in *weak head normal form*⁹⁴. Before introducing WHNF, let us recall the definition of a *value*. As mentioned in *Tigris*, a value v is in normal form if $\langle v, \sigma \rangle \Downarrow \langle v \rangle$ where σ denotes a state (if any). In *Tigrisd*, v is defined as

$$\begin{array}{lcl}
v & ::= & \text{values} \\
| & & \text{Type} \\
| & & \lambda x. a \\
| & & (x: A) \rightarrow B_x \\
| & & (x: A) \times B_x \\
| & & (a, b) \\
| & & a = b \\
| & & \mathbf{rfl}
\end{array}$$

WHNF has similar form to v and contains v , but is generalized to quantify more forms. Readers should pay attention to the phrase “cannot be reduced further” as this is precisely what WHNF quantifies. This additional part that WHNF quantifies, is called *neutral form*. A neutral term mainly contains a sequence of applications, with its head being a variable. Although we knew that the arguments to this variable may be redexes⁹⁵ themselves, this application itself cannot be reduced further. Therefore, it belongs in WHNF. In this calculus, the neutral terms are given by

$$\begin{array}{lcl}
\text{neutral, } ne & ::= & \text{neutral} \\
| & & x \\
| & & ne \ a \\
| & & \mathbf{let} \ (x, y) = ne \ \mathbf{in} \ a \\
| & & ne \triangleright a
\end{array}$$

For example, the term $(x : \text{Type}) \times (\lambda x. \text{Nat}) \ \text{Unit}$ is in WHNF, because, though its RHS is β -reducible, this term itself is not: It is a coproduct; Or the neutral term $\mathbf{add} \ 1 \ 3$, which, judging from its apperance, should reduce to 4 but we are unsure of that yet as we did not *unfold* the definition of $\mathbf{add}$, therefore, this term is also irreducible in WHNF. We introduce this concept because it concerns with reduction strategies, or normalization of forms (as mentioned in the footnote), which in turn, matters a lot when implementing equality as we want equality check to not only cover terms that *look exact*⁹⁶, but also several basic equivalence in the lambda calculus, which though possible by a naive evaluator that performs *full reduction*, is not good enough and can potentially be harmful. Therefore, we want to set *standards* for terms (i.e. terms that should not be eagerly reduced) in order to *control reduction strategies* of properly constructed evaluators that respect these standards. WHNF is exactly the standard we are looking for.

Specifically,

⁹⁴this term supposedly came from Peyton Jones (SPJ), for its relations to reduction/evaluation strategies.

⁹⁵reducible expressions

⁹⁶as in syntactic equality (w/o α -renaming)

- WHNF allow us to *defer*⁹⁷ the unfolding of definitions until needed, similar to a call-by-need evaluation strategy. This, intuitively, mimics the *symbolic* computational thought process of a normal human; Consider the equation $f\ 3 = f\ (1 + 2)$ where f represents a *lengthy computation*. Both sides are in WHNF, where a naive evaluator may spend time reducing all the application of f . A WHNF-respecting evaluator can instead compare $3, (1 + 2)$ because both sides share the same *head* f . *Deciding* this equality is simple enough that we do not need to unfold f at all. In fact, this is just a special case of the *congruence* relation of functions.
- However, the equality relation in *Tigris*d is *semidecidable*, as in sometimes it is possible for a computation to diverge because totality is not guaranteed. WHNF *alleviates* this problem by – as mentioned above – deferring unfolding as even if f diverges, the equality still holds, proven by *Tigris*d. Though it does *not* entail that no unfolding is taken place – As to prove $1 + 1 = 2$, we must unfold $+$, and in the case $+$ diverges, the reduction loops no matter what, leading to the eventual crash of the system when it runs out of stack/heap spaces; Though, the REPL implementation has similar feature to that of *Tigris*, allowing users to interrupt with $\langle C-C \rangle$.

Furthermore, readers with relatively little prover knowledge may draw intuition from Lean’s `rfl` tactic/term and Coq’s `reflexivity` tactic, both of which instructs the typechecker to perform normalization to decide equality, which is arguably much more stronger than *Tigris*d’s, implemented in similar way, if not more complex, of making rules to avoid eager reductions. Moreover, they have some form of *normalization by evaluation* (NBE) which compiles expressions to forms suitable for running, such as (OCaml) bytecode, if using Coq. It is about the granularity of these rules and the performance of normalization that set us apart.

We now define the *reduction*⁹⁸ rules to WHNF and finally, the implementation of equality in *Tigris*d:

$\Gamma \vdash \mathbf{WHNF}\ a \rightsquigarrow nf$			<i>(WHNF normalization)</i>
WHNF-VAR $\frac{x = a \in \Gamma}{\Gamma \vdash \mathbf{WHNF}\ a \rightsquigarrow nf}$	WHNF-LAM $\frac{}{\Gamma \vdash \mathbf{WHNF}\ \lambda x. a \rightsquigarrow \lambda x. a}$	WHNF-PI $\frac{}{\Gamma \vdash \mathbf{WHNF}\ (x:A) \rightarrow B_x \rightsquigarrow (x:A) \rightarrow B_x}$	
		WHNF-LAM-BETA $\frac{\Gamma \vdash \mathbf{WHNF}\ a \rightsquigarrow (\lambda x. a')}{\Gamma \vdash \mathbf{WHNF}\ (a'[x \mapsto b]) \rightsquigarrow nf}$	
WHNF-TYPE $\frac{}{\Gamma \vdash \mathbf{WHNF}\ \text{Type} \rightsquigarrow \text{Type}}$		$\frac{}{\Gamma \vdash \mathbf{WHNF}\ a\ b \rightsquigarrow nf}$	
WHNF-LETPAIR $\frac{\Gamma \vdash \mathbf{WHNF}\ a \rightsquigarrow (a_1, a_2)}{\Gamma \vdash \mathbf{WHNF}\ (b[x \mapsto a_1][y \mapsto a_2]) \rightsquigarrow nf}$		WHNF-PROD-CONGR $\frac{\Gamma \vdash \mathbf{WHNF}\ a \rightsquigarrow ne}{\Gamma \vdash \mathbf{WHNF}\ \text{let } (x, y) = a \text{ in } b \rightsquigarrow \text{let } (x, y) = ne \text{ in } b}$	
WHNF-DEF $\frac{x = a \in \Gamma}{\Gamma \vdash \mathbf{WHNF}\ a \rightsquigarrow v}$	WHNF-LET-BETA $\frac{\Gamma \vdash \mathbf{WHNF}\ (b[x \mapsto a]) \rightsquigarrow v}{\Gamma \vdash \mathbf{WHNF}\ (\text{let } x = a \text{ in } b) \rightsquigarrow v}$	WHNF-TYPAEQ-REFL $\frac{}{\Gamma \vdash \mathbf{WHNF}\ (a = b) \rightsquigarrow (a = b)}$	

⁹⁷do note the difference between *deferred* evaluation and *lazy* evaluation. Usually the latter encapsulate the former.

⁹⁸we emphasize *reduction* because this transformation does not change the “meaning” of a term.

$$\begin{array}{c}
\text{WHNF-REFL} \\
\hline
\Gamma \vdash \mathbf{WHNF} \mathbf{rfl} \rightsquigarrow \mathbf{rfl}
\end{array}
\qquad
\begin{array}{c}
\text{WHNF-TYPALSUBST} \\
\Gamma \vdash \mathbf{WHNF} b \rightsquigarrow rf \\
\Gamma \vdash \mathbf{WHNF} a \rightsquigarrow nf \\
\hline
\Gamma \vdash \mathbf{WHNF} (b \triangleright a) \rightsquigarrow nf
\end{array}
\qquad
\begin{array}{c}
\text{WHNF-TYPALSUBST-CONGR} \\
\Gamma \vdash \mathbf{WHNF} b \rightsquigarrow ne \\
\hline
\Gamma \vdash \mathbf{WHNF} (ne \triangleright a) \rightsquigarrow (ne \triangleright a)
\end{array}$$

$$\boxed{\Gamma \vdash a \stackrel{\text{impl}}{=} b}$$

(equality implementation)

$$\begin{array}{c}
\text{IMPLEQ-REFL} \\
\hline
\Gamma \vdash a \stackrel{\text{impl}}{=} a
\end{array}
\qquad
\begin{array}{c}
\text{IMPLEQ-LAM-CONGR} \\
\Gamma \vdash a_1 \stackrel{\text{impl}}{=} a_2 \\
\hline
\Gamma \vdash \lambda x. a_1 \stackrel{\text{impl}}{=} \lambda x. a_2
\end{array}
\qquad
\begin{array}{c}
\text{IMPLEQ-APP-CONGR} \\
\Gamma \vdash ne_1 \stackrel{\text{impl}}{=} ne_2 \\
\Gamma \vdash b_1 \stackrel{\text{impl}}{=} b_2 \\
\hline
\Gamma \vdash ne_1 b_1 \stackrel{\text{impl}}{=} ne_2 b_2
\end{array}$$

$$\begin{array}{c}
\text{IMPLEQ-PI-CONGR} \\
\Gamma \vdash A_1 \stackrel{\text{impl}}{=} A_2 \\
\Gamma \vdash B_1 \stackrel{\text{impl}}{=} B_2 \\
\hline
\Gamma \vdash (x: A_1) \rightarrow B_{1x} \stackrel{\text{impl}}{=} (x: A_2) \rightarrow B_{2x}
\end{array}
\qquad
\begin{array}{c}
\text{IMPLEQ-WHNF-EXT} \\
\Gamma \vdash \mathbf{WHNF} a \rightsquigarrow nf_1 \\
\Gamma \vdash \mathbf{WHNF} b \rightsquigarrow nf_2 \\
\Gamma \vdash nf_1 \stackrel{\text{impl}}{=} nf_2 \\
\hline
\Gamma \vdash a \stackrel{\text{impl}}{=} b
\end{array}$$

Which, reveals an important property of WHNF:

Theorem 1 (WHNF equality invariant). Two terms are equal if and only if their WHNF reductions are equal.

From left to right, this property is guaranteed by the invariant of reduction; From right to left, this property is shown by rule **IMPLEQ-WHNF-EXT**.

7 Summary

Regarding *Tigris*, the work is somewhat completed. Regarding *Tigrisd*, while the working prototype is available, its implementation notes and rules of dependent pattern matching and usage examples are omitted in this report, but I'm working on delivering it in the final thesis.

此外, 格式部分并不符合要求⁹⁹, 语言部分也较为粗糙, 不够精炼, 甚至可能有打错的部分. 但作为中期报告以及论文初稿的核心内容 (除去上面 *Tigrisd* 中的部分) 来说应当是充分的.

⁹⁹之后会将本文内容誊抄到 GitHub 上师兄师姐制作的学校本科生论文模板中.